

XML
PRESS

Schematron

A Language for Validating XML



Erik Siegel

Schematron

Table of Contents

1. Preface

- 1.1. Who is this book for?
- 1.2. How to use this book
- 1.3. Using and finding the code examples
- 1.4. The oXygen IDE
- 1.5. Contact information
- 1.6. Acknowledgements

2. Introduction to Schematron

- 2.1. What is Schematron?
- 2.2. Why Schematron?
- 2.3. The history of Schematron in a nutshell
- 2.4. The Schematron standard
 - 2.4.1. A critical note on the standard
- 2.5. The Schematroll
- 2.6. An illustrative example
 - 2.6.1. The example document
 - 2.6.2. Running the examples
 - 2.6.3. Checking the value of the `code` attribute
 - 2.6.4. Improving the message
 - 2.6.5. Using variables

3. Schematron in context

- 3.1. Validating XML documents
 - 3.1.1. Restricting XML freedom
 - 3.1.2. How to define XML formats
 - 3.1.3. Validating XML documents
- 3.2. The main schema languages
 - 3.2.1. Document Type Definition (DTD)
 - 3.2.2. W3C XML Schema
 - 3.2.2.1. W3C XML Schema version 1.1
 - 3.2.3. RELAX NG
 - 3.2.4. Schematron

4. Applying Schematron

- 4.1. Creating Schematron schemas
- 4.2. IDE-based Schematron validation

- 4.3. Raw Schematron validation
- 4.4. Validation results
 - 4.4.1. SVRL
 - 4.4.2. XVR
- 4.5. Schematron validation processors
 - 4.5.1. The “skeleton” XSLT Schematron processor
 - 4.5.2. The SchXslt Schematron processor
 - 4.5.2.1. Running SchXslt from the command line
- 5. Schematron basics
 - 5.1. Setting up a Schematron schema
 - 5.2. Patterns, rules, assertions, and reports
 - 5.2.1. The Schematron processing algorithm
 - 5.2.2. Rule processing
 - 5.2.3. Assert and Report processing
 - 5.2.3.1. Using reports instead of assertions
 - 5.2.3.2. The message texts
 - 5.3. More meaningful messages: `<value-of>`
 - 5.4. Declaring and using variables: `<let>`
 - 5.4.1. Ground rules for variables
 - 5.4.2. Variable usage example
 - 5.5. Declaring namespaces
- 6. Advanced Schematron
 - 6.1. Providing multiple messages: `<diagnostics>`
 - 6.1.1. Multiple messages
 - 6.1.2. Localization of messages
 - 6.2. Selecting what patterns are active: `<phase>`
 - 6.3. Reusing rules: abstract rules
 - 6.3.1. Abstract rules in external documents
 - 6.3.2. Alternative to abstract rules
 - 6.4. Reusing patterns: Abstract patterns
 - 6.5. Including documents: `<include>`
- 7. Query language binding and using XSLT
 - 7.1. Introduction to query language binding
 - 7.2. Using XSLT in Schematron
 - 7.2.1. Using XSLT keys
 - 7.2.2. Using XSLT functions
 - 7.2.3. Using other XSLT features

8. Additional features

- 8.1. Messages with markup: `<emph>`, `<span>`, and `<dir>`
- 8.2. Flags
- 8.3. Properties
- 8.4. Adding structured comments: `<title>` and `<p>`
- 8.5. Validating documents referenced by XInclude
- 8.6. Specifying a role: the `role` attribute
- 8.7. Specify a different location: the `subject` attribute
- 8.8. The `<name>` element
- 8.9. The `icon`, `see`, and `fpi` attributes

9. Schematron examples and recipes

- 9.1. Validating a Schematron schema
- 9.2. Handling a default namespace
- 9.3. Using Schematron for schema validation
- 9.4. Checking multiple identifier references
- 9.5. Validating processing instructions and comments
- 9.6. Validating doubled elements in mixed content

A. XPath technology primer

- A.1. XML as a tree
 - A.1.1. Basic trees: documents, elements, and text
 - A.1.2. Attributes in the tree
 - A.1.3. Representing mixed content
- A.2. Basic tree navigation
 - A.2.1. Using navigation expressions
 - A.2.2. Basic navigation to attributes
 - A.2.3. Multiple selections: sequences
- A.3. Tree navigation and the context item
- A.4. Some special operators
 - A.4.1. The context item single dot `.` operator
 - A.4.2. The parent double dot `..` operator
 - A.4.3. The `*` and `@*` wildcard operators
 - A.4.4. The “search” `//` operator
- A.5. Predicates in tree navigation
- A.6. Expressions on simple data
 - A.6.1. The XPath function library
 - A.6.2. XPath data types

- A.6.2.1. Explicit data typing and data type conversions
 - A.6.3. Numerical expressions
 - A.6.4. String expressions
 - A.6.5. Boolean expressions and comparisons
 - A.6.5.1. Testing sequences
 - A.6.5.2. Testing string values
 - A.6.6. Working with dates, times and durations
 - A.6.6.1. Additional date, time, and duration functions
- B. An introduction to namespaces
 - B.1. Why namespaces?
 - B.2. Namespace names
 - B.3. Declaring namespaces
 - B.3.1. Defining a default namespace
 - B.3.2. Defining and using namespace prefixes
 - B.3.3. The XML namespace
 - B.4. Namespaces in Schematron
- C. Schematron reference
 - C.1. XML structure overview notation
 - C.2. The Schematron namespace
 - C.2.1. Using other namespaces
 - C.3. Root element: `<schema>`
 - C.4. Declaring a namespace: `<ns>`
 - C.5. Defining validation phases: `<phase>`
 - C.5.1. Attaching a single pattern to a phase: `<active>`
 - C.6. Creating validation patterns: `<pattern>`
 - C.6.1. Rules in validation patterns: `<rule>`
 - C.6.1.1. Defining assertions: `<assert>`
 - C.6.1.2. Defining reports: `<report>`
 - C.6.1.3. Referencing an abstract rule: `<extends>`
 - C.6.2. Parameters for abstract patterns: `<param>`
 - C.7. Defining diagnostic messages: `<diagnostics>`
 - C.7.1. Diagnostic message: `<diagnostic>`
 - C.8. Defining additional properties: `<properties>`
 - C.8.1. Defining a property: `<property>`
 - C.9. Message markup: mixed contents

C.9.1. Writing direction: `<dir>`

C.9.2. Emphasis: `<emph>`

C.9.3. Retrieving the name of a node: `<name>`

C.9.4. Spanning text: `<span>`

C.9.5. Value of an XPath expression: `<value-of>`

C.10. Standard attributes and elements

C.10.1. The `flag` attribute

C.10.2. The `fpi` attribute

C.10.3. The `icon` attribute

C.10.4. The `role` attribute

C.10.5. The `subject` attribute

C.10.6. The `see` attribute

C.10.7. The `xml:lang` attribute

C.10.8. The `xml:space` attribute

C.10.9. The `<let>` element

C.10.10. The `<p>` element

C.10.11. The `<title>` element

C.10.12. The `<include>` element

D. SVRL reference

D.1. SVRL in the Schematron standard

D.2. The SVRL namespace

D.3. Root element: `<schematron-output>`

D.4. Namespace declarations: `<ns-prefix-in-attribute-values>`

D.5. Active patterns: `<active-pattern>`

D.6. Fired rules: `<fired-rule>`

D.7. Failed asserts: `<failed-assert>`

D.7.1. Diagnostic reference: `<diagnostic-reference>`

D.7.2. Property reference: `<property-reference>`

D.8. Successful reports: `<successful-report>`

D.9. Mixed text: `<text>`

D.9.1. Writing direction: `<dir>`

D.9.2. Emphasis: `<emph>`

D.9.3. Spanning text: `<span>`

E. Schematron QuickFix

F. Additional resources

F.1. Schematron

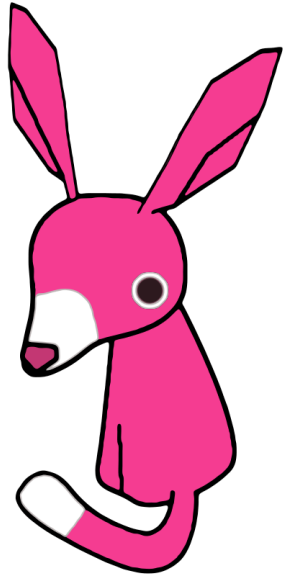
F.2. XPath

- F.3. XML Schema
- F.4. RELAX NG
- F.5. XSLT
- F.6. XQuery
- F.7. XProc
- F.8. Other information
- G. Copyright and Legal Notices

Schematron

A Language for Validating XML

Erik Siegel



XML
PRESS

Chapter 1. Preface

Computer systems process, read, exchange, and spit out data like the contents of this book, your tax form, electronic health records, social media messages, and much more. All such computer data must be in some kind of well-defined format, so it can be understood and processed correctly.

Computer data formats can be organized into families, according to the ground rules they follow. This is analogous to natural languages. In the western world we have a limited character set (A-Z, sometimes a few more), and we write from left to right. Other language families, for example Chinese, have a much more extended character set and a different writing orientation. If you speak a western language but don't understand French, you can't comprehend text in French, but you can still recognize constructs from the Western language family, such as characters, words, sentences, and paragraphs. For Chinese that will be much more difficult.

Computer data format families work the same way. Examples of data format families include EDI, CSV, JSON, and XML. Each has its own syntax rules, application domains, pros, cons, and fan club. And just as a French person and a Swedish person cannot easily communicate, even though their languages are in the same family, a computer system that talks XML language A cannot understand XML language B. People and computers both can perceive familiar language constructs, but they cannot understand what the other person/computer is saying unless they know the specific language being used.

In the natural language world we can sometimes partly solve that communication problem with a little improvisation, such as sign language or gestures. However, for computers that's not so easy. What goes in and what comes out must be correct and understandable. The process of establishing this correctness for computer data is called *validation*.

Validation is usually done in phases:

- First the data is checked to see whether it follows the language-family *ground rules*.

- Then checks are done to determine whether it's in the **correct language** and follows the **grammar** of that language.
- And, optionally, **business rules** are checked to determine whether the data makes sense.

[Table 1.1](#) compares these phases for natural and computer languages:

Table 1.1 – Comparison of natural versus computer language validation

Check	Natural language	Computer language
Language family ground rules	Does it use only allowed characters ? Are these characters organized into words ? Is punctuation used correctly? Are sentences organized into paragraphs , etc.?	Does it uses only allowed characters? Are data elements delimited correctly? Are things grouped correctly, etc.?
Language grammar check	Are the words spelled correctly ? Are the verbs conjugated correctly ? Is the word order as expected for this language, etc.?	Is the data recognizable? Are all data elements there? Are numbers written as numbers, dates as dates, etc. Are all required data elements present, etc.?
Business rules check	Does it have an <u>introductory paragraph</u> ? Do numbers, if any, make sense? Is the <u>author's name spelled correctly</u> , etc.?	Is this date before that date ? Is the sum of this set of numbers not more than some threshold? Is the value of this data element present in that external database table , etc.?

The rules for computer data checks can be expressed in special formal computer languages called **validation languages**. Validation languages allow the computer system to validate whether the data it consumes or produces is correct and *valid*. Schematron, the subject of this book, is about validating XML, so let's focus on that.

- For XML, checking the **ground rules** of the language family is an absolute precondition for any subsequent checks. An XML document that passes this stage is called **well-formed**.
- To check **grammar** you can choose from several XML validation languages. The most common ones are DTD, W3C XML Schema, and RELAX NG. If this stage is passed, an XML document is called **valid**.
- **Schematron** is a validation language that checks **business rules**. A set of checks written in the Schematron language is called a *Schematron schema*. There's no official name for a document that passes this stage, but let's call it **Schematron valid**.

This process of validating XML documents is explained in much more detail in [Chapter 3](#).

A Schematron schema consists (mostly) of **assertions**. A simple example would be that, somewhere in an XML document, a given start date must always be before a given end date. You can write such an assertion in the Schematron language and then validate that it's true for your data. If not, some error message is produced.

This book explains how to create Schematron schemas and use them to check XML data for compliance with your business rules.

1.1. Who is this book for?

This book is for anyone who wants to learn Schematron or expand their existing knowledge of the language.

I assume you have at least a basic knowledge of XML. That is, you know roughly what documents, elements, and attributes are, and you have seen data formatted like [Example 1.1](#) before:

Example 1.1 – An example XML document (schematron-book-code/data/invoices.xml)

```
<invoices date="2020-11-11" total="10101.33">
  <invoice date="2020-10-04" total="10000.50" id="12345"/>
  <invoice date="2020-10-16" total="100.83" id="56789"/>
</invoices>
```

Or like [Example 1.2](#):

Example 1.2 – Another example XML document (schematron-book-code/data/text.xml)

```
<body>
  <p class="greeting">Hello <b>Schematron</b>!</p>
</body>
```

If the examples above look like complete gibberish, this book is probably not for you. But if you know your way around XML at this level, read on.

In writing this book, I assumed that Schematron is used by a wide variety of people with a broad range of backgrounds. Some will be [XSLT](#) or [XQuery](#) [programmers](#). Some will previously have written schemas in other languages like W3C XML Schema, or RELAX NG. Others will be less tech-savvy or newer to the XML world. This book tries to explain Schematron for both XML aficionados and people who are less experienced. I assume a basic familiarity with XML. A little programming experience helps but is not strictly necessary. If your experience fits this profile, basic Schematron should be no problem.

For those that want to go beyond basics, there's also a lot to be gained. Schematron schemas can do amazing things, like combining data from many sources or even digging into databases. You can mix it with programming languages like XSLT, unleashing all the power these languages provide. This book will teach you how to do this.

If, after reading the above, you think Schematron might be interesting but are still a little unsure, please jump ahead to [Section 2.6, “An illustrative example”](#). This will provide you with an example of basic Schematron usage. Hopefully this will give you a good picture.

1.2. How to use this book

There are several ways you can use this book. Here are a few suggestions:

- For those that are new to the XML world or have limited programming experience, this book contains two introductions on important topics:
 - To create Schematron schemas, you need at least a basic knowledge of [XPath](#). If you don't know what this is and expressions like `/data/entry` or `//invoice[1]/@id` don't make sense to you, please read the technology primer, [Appendix A](#), first.
 - [Appendix B](#) introduces another tricky subject in the XML world: [namespaces](#). Even if you have nothing to do with Schematron, there's a good chance you need to know something about namespaces when working with XML.
- If you want to know more about [validating documents in general](#), using Schematron and other languages, read [Chapter 2](#) and [Chapter 3](#).
- If you want to know [how to apply Schematron](#), how to set a Schematron schema to work, read [Chapter 4](#).
- [Chapter 5](#) explains the basics of Schematron. This will provide you with enough information to get going and write simple, but nonetheless useful, Schematron schemas.
- [Chapter 6](#), [Chapter 7](#), and [Chapter 8](#) go beyond the basics to teach you to create reusable schema components, [integrate other programming languages](#), and use other nifty features.
- [Chapter 9](#) contains [more advanced examples and recipes](#).

- For those that need to see the nitty-gritty details of the language, please refer to the references in [Appendix C](#) and [Appendix D](#).
- SQF (Schematron QuickFix), a not widely supported but nonetheless valuable extension to Schematron, is explained in [Appendix E](#).

1.3. Using and finding the code examples

The code examples in this book are at <https://github.com/xatapult/schematron-book-code>, a public GitHub repository. Feel free to download or clone the repository and use the code and data in any way you like. The examples that come from this repository contain a path between parentheses in the example's title, like [Example 1.3](#).

Example 1.3 – Example document from the book code repository (schematron-book-code/data/parcels-valid.xml)

```
<parcels max-weight="100" delivery-date="2021-10-10">
  <parcel weight="50" date="2021-09-29">
    <contents>A large quantity of mouth masks</contents>
  </parcel>
  <parcel weight="45" date="2021-09-12">
    <contents>Toys for children</contents>
  </parcel>
</parcels>
```

For example, if you've cloned the repository in `/work/data`, [Example 1.3](#) can be found in `/work/data/schematron-book-code/data/parcels-valid.xml`.

1.4. The oXygen IDE

Applying Schematron, especially when you're just beginning, is easiest using an IDE (Integrated Development Environment) that natively supports Schematron. Such an IDE helps you write schemas. You can run validations interactively and see the results in the user interface. As far as I know, the only IDE currently available (2022) that supports Schematron this way is **oXygen**.

oXygen is not free software. However, it offers a trial license, so you can use it for a limited period of time to get acquainted. For more information, please visit <https://www.oxygenxml.com>.

Using oXygen is by far the easiest way to follow the examples in this book and learn about Schematron. Therefore, I mention oXygen frequently. This may look like I own shares in the company, but I'm a paying customer myself. There simply seems to be no competition with regards to Schematron. If I'm wrong (maybe you just created the most awesome interactive Schematron validation IDE yourself...), please drop me an email and let me know.

If you can't or don't want to use oXygen, you can also run a Schematron validation from the command line. See [Section 4.5.2.1, “Running SchXslt from the command line”](#).

1.5. Contact information

This book was written by Erik Siegel (Xatapult, <http://www.xatapult.com>). You can reach me at erik@xatapult.nl.

I would definitely like to hear from you. Whatever you have to say about this book, please drop me an email. Knowing that there are people who use what I have written keeps me motivated.

1.6. Acknowledgements

I would like to thank all people that reviewed (parts of) this book and provided me with valuable feedback. In alphabetical order: Paul van Aalstede, Jennifer Flint, Tony Graham, Rick Jelliffe, Pieter Lamers, Pieter Masereeuw, Birgit Orthofer, and Andrew Sales. Also, many thanks to my editor and publisher, Richard Hamilton. His attention to detail, both technical and editorial, made this a much better book. Thanks to all of you for your time and effort!

Chapter 2. Introduction to Schematron

This chapter provides a high-level overview of Schematron. The last section, [Section 2.6, “An illustrative example”](#), contains an example of Schematron usage to introduce you to the language itself.

2.1. What is Schematron?

Here’s an overview of Schematron’s main high-level characteristics:

- Schematron is a formal schema language in which you can express rules for XML documents.
- There are two types of rules:
 - **Assertions:** when the condition for an assertion fails, an error message is issued.
 - **Reports:** when the condition for a report holds, a report message is issued.

In practice, you will use assertions more often than reports.

- In Schematron you define all the error and report messages in your own words.
- Schematron is expressed in XML: a Schematron schema is an XML document.
- Schematron allows you to specify the underlying language for its expressions. In practice, XPath is the only language supported.
- Schematron can, by design, incorporate constructs from other programming languages. However, most public implementations support XSLT only.

2.2. Why Schematron?

There are a number of reasons why Schematron is such a useful tool in the XML toolbox. Here are the most important ones:

- Schematron is a relatively simple but powerful validation language. Basic Schematron (covered in [Chapter 5](#)) already has a wide field of application and is relatively easy to master.
- Schematron can go way beyond the validations of the “classic” validation languages like DTD, W3C XML Schema, and RELAX NG. It allows you to do extensive checks on XML structures and data that are not possible in other languages. Anything you can express as an XPath test can be used for validation purposes. More experienced users can take advantage of XSLT features such as keys and functions.
- In Schematron you define all the error or report messages yourself. For other validation languages you’re at the mercy of the validation processor’s implementer, and this often results in technically correct but, for users, obscure messages. In Schematron this is completely under your control. Messages can be enriched with computed text from or about the validated document by using XPath expressions.
- Since the messages are under your control, Schematron is often used to partially take over validations normally done by other validation languages. Messages can be tailored to the user’s knowledge level or context. So instead of:

The content of element 'section' is not complete. One of '{para}' is expected

You could tell the user:

A section in a report must have at least one paragraph of text

2.3. The history of Schematron in a nutshell

Schematron came to life in 1999 in Taiwan with a presentation called “From Grammars to Schematron” by Rick Jelliffe.^[1] This eventually resulted, in 2001, in Schematron 1.5, a version based on XSLT 1.0. You could use it anywhere an XSLT processor was available, which was one of the reasons for its success.

After this, standardization was taken over by ISO (International Organization for Standardization). This led to the release, in 2006, of ISO

Schematron, which added new features such as variables and abstract patterns.

Development continued and newer versions of XPath and XSLT were incorporated. Through a succession of versions we arrived at what is, in 2022, the most current one: *ISO/IEC 19757-3, Third edition, 2020-06: Information technology - Document Schema Definition Languages (DSDL) - Part 3: Rule-based validation using Schematron*. This is the version used in this book.

2.4. The Schematron standard

The June 2020 Schematron version can be obtained from the ISO website at <https://www.iso.org/standard/74515.html>. In 2022 it cost 158 Swiss francs. If you buy it you'll get a PDF of 38 pages, describing the Schematron standard.^[2] This includes many pages with listings of RELAX NG and other schemas for Schematron itself and its reporting language SVRL.^[3]

Is it necessary to buy this standard for writing Schematron schemas? My advice would be not to spend your money on it. This book and/or what you can find on the web will give you more than enough information and examples to get going. The standard itself is a formal and terse document that doesn't add any value for most users.

The only reason I can think of to buy it is if you plan to write a new Schematron processor (or, of course, another Schematron book), in which case you need access to all the minor details of the language definition. But for day-to-day use, don't bother.

2.4.1. A critical note on the standard

In my dealings with Schematron for this book I had a close look at the standard itself, probably closer than most users will ever look. And this gave me the uneasy impression that the Schematron standard was put together with insufficient attention to detail. And details are important: consistent standards lead to consistent implementations. Most inconsistencies I encountered had to do with the Schematron reporting

language (SVRL) and how this should be produced. I will say some more about this in [Appendix D](#).

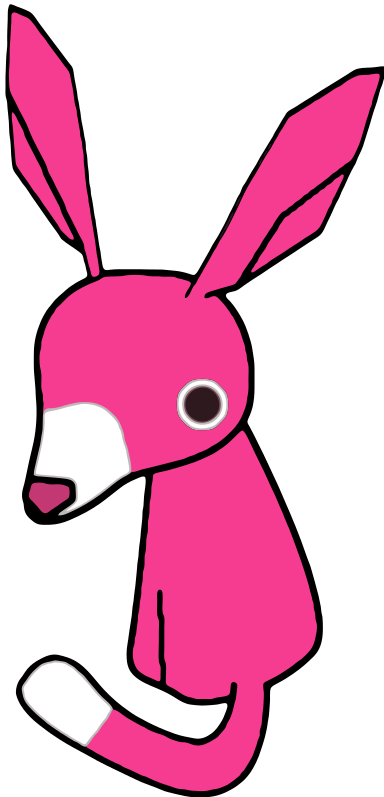
I'm not the only one that thinks the Schematron standard can be improved. A group of Schematron enthusiasts (Tony Graham, David Maus, Andrew Sales, and me) started the “Schematron enhancement proposals” initiative: a list of points for improvement. You can read our proposal on GitHub at <https://github.com/Schematron/schematron-enhancement-proposals>.

As a result of our efforts, ISO re-opened its standardization process for Schematron in September 2022. It will probably take about three years to complete this process.

2.5. The Schematroll

What would a standard be without a mascot? And for Schematron, that's Schematroll:

Figure 2.1. The notorious Schematroll



Schematroll was drawn by Cody Chang. It's a cross between two marsupials, the Bilby and the Bettong. You can use it for free on any Schematron-related product. It has its own GitHub repository: <https://github.com/Schematron/schematroll>.

2.6. An illustrative example

This section provides you with a first illustrative example of Schematron. The example is a simple Schematron schema that checks an inventory list in an XML document for errors.

Schematron allows you to specify the underlying language for its expressions. In practice, XPath is the only language supported. There is nothing overly complex here, but to understand the examples, you need a basic understanding of XPath. If you are new to XPath (or think your XPath knowledge needs a booster), please read [Appendix A](#) first. Another tricky subject I touch upon is namespaces, to which you can find an introduction in [Appendix B](#).

2.6.1. The example document

Schematron is about validating XML documents. This set of Schematron examples is based on the following inventory list example document ([Example 2.1](#)):

Example 2.1 – An example inventory list XML document (`schematron-book-code/data/example-1.xml`)

```
<inventory-list deocode="IMP">
  <article code="IMP0001">
    <name>Bolts</name>
    <description>Nuts to secure things with</description>
  </article>
  <article code="IMP0002">
    <name>Nuts</name>
    <description>Bolts to turn on the nuts</description>
  </article>
  <article code="EXP0234">
    <name>Bananas</name>
    <description>Delicious ripe bananas</description>
  </article>
</inventory-list>
```

The rule imposed on this type of document is that the `code` attribute on an `<article>` element must start with the department code. This department code can be found in the `deptcode` attribute on the root element. So the first two articles are valid, but the third one isn't.

Of course, this example is small, and you could easily validate it by hand. But what if it contained thousands of articles instead of just three? Or if you had many documents instead of just one. Automated validation then becomes necessary, which is where Schematron comes to the rescue.

2.6.2. Running the examples

If you want you can follow along and try the example Schematron schemas yourself. To do this:

- Download the example code belonging to this book as described in [Section 1.3, “Using and finding the code examples”](#).
- You need some means to actually run the schemas:
 - The easiest way is to get hold of the oXygen IDE. See [Section 1.4, “The oXygen IDE”](#) for more information on this.
 - You can run Schematron from the command line. See [Section 4.5.2.1, “Running SchXslt from the command line”](#) for more information.

2.6.3. Checking the value of the `code` attribute

Let's start by checking the value of the `code` attribute on the `<article>` elements in [Example 2.1](#). The root element of the inventory list has an attribute for its department code called `deptcode`. We need to verify that the value of each `code` attribute begins with this department code. For [Example 2.1](#), the first two articles are valid, but the third one isn't. [Example 2.2](#) validates this.

Example 2.2 – A Schematron schema for checking the code attributes of [Example 2.1](#) (`schematron-book-code/examples/introduction/example-1.sch`)

```

<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <pattern>
    <rule context="article">
      <assert test="starts-with(@code, /inventory-list/@depcode)">
        The article code must start with the right prefix
      </assert>
    </rule>
  </pattern>

</schema>

```

In a nutshell:

- All `<article>` elements in the document are checked, one by one:

```
<rule context="article">
```

- The assertion that their code attributes start with the correct prefix is verified:

```
<assert test="starts-with(@code, /inventory-list/@depcode)">
```

- If so, nothing happens, but if this isn't the case this message is issued:

```
The article code must start with the right prefix
```

When validating [Example 2.1](#) against the Schematron schema in [Example 2.2](#), the third `<article>` element will raise an error, as expected:

```
The article code must start with the right prefix
```

This immediately illustrates one of the major benefits of Schematron validations: the message is in the code, it is part of the schema, and you specify the text.

Let's examine how [Example 2.2](#) is constructed in more detail. References to where you can find more information are at the end of this section.

- The root element of a Schematron schema is always `<schema>`.
- This, and all other Schematron elements, must be in the Schematron namespace. This is taken care of by the `xmlns="http://purl.oclc.org/dsdl/schematron"` "attribute"^[4] that defines this as the default namespace.

- The `queryBinding="xslt3"` attribute tells the Schematron processor what underlying programming language(s) you can use. It's called the "query language binding." The value `xslt3` means you can use certain XSLT 3.0 constructs (though this example doesn't), but also, more importantly, you can use XPath 3.1 expressions.

For instance, the `context` and `test` attributes in [Example 2.2](#) contain XPath expressions. By setting the query binding to `xslt3`, you can use the most current version of XPath (as of 2022).

Other query language bindings

When trying this example, your Schematron processor might complain about the `xslt3` query language binding. In that case try the value `xslt2` first. If that still doesn't work, try `xpath31`, `xpath3`, or `xpath2`.

In the unlikely case that none of these values work in your environment (maybe you're using a proprietary Schematron processor), it will be difficult to follow and try the examples. You will need to find out which query language(s) work with your processor or use one of the processors suggested in this book. More information about query language binding in [Chapter 7](#).

-
- A Schematron schema always consists of (one or more) patterns in `<pattern>` elements.
 - A pattern consists of rules in `<rule>` elements. A rule has a `context` attribute that defines what part of the document this rule is about. For [Example 2.2](#) this is the `<article>` element:

```
context="article"
```

The effect is that this rule will be held against all `<article>` elements in the document.

A pattern can consist of multiple rules, but only the first one for which the `context` attribute matches is executed.

- The most common constructs in rules are assertions, in `<assert>` elements. An assertion checks something, as expressed in its `test` attribute. In [Example 2.2](#) it checks whether the value of the `article's`

code attribute starts with the value of the `depcode` attribute on the root element:

```
starts-with(@code, /inventory-list/@depcode)
```

If so, this expression returns `true`, the assertion holds, all is fine, and no message is issued. However, if the assertion fails, the expression returns `false` and the text in the `<assert>` element is issued as an error message.

For more information:

- XPath in general: [Appendix A](#).
- Namespaces in general: [Appendix B](#).
- Basic schemas: [Section 5.1, “Setting up a Schematron schema”](#).
- Patterns, rules, and asserts: [Section 5.2, “Patterns, rules, assertions, and reports”](#).
- Query language binding: [Chapter 7](#).

2.6.4. Improving the message

The message issued by [Example 2.2](#) can be improved. For instance, it would be nice if it would tell us what article it is about. For this, Schematron allows you to insert values from your document into your messages, using the `<value-of>` element (see [Example 2.3](#)).

Example 2.3 – Using the name of the article in the messages ([schematron-book-code/examples/introduction/example-2.sch](#))

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <pattern>
    <rule context="article">
      <assert test="starts-with(@code, /inventory-list/@depcode)">
        The article code must start with the right prefix
        for <value-of select="name"/>
      </assert>
    </rule>
  </pattern>

</schema>
```

The `<value-of select="name">` element inserts the value of the article's `<name>` element in the message, resulting in:

The article code must start with the right prefix for Bananas

There's no limit how far you can go dressing up your messages. For instance, [Example 2.4](#) tries to make things absolutely clear.

Example 2.4 – Using even more information in the message ([schematron-book-code/examples/introduction/example-2b.sch](#))

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="article">
      <assert test="starts-with(@code, /inventory-list/@depcode)">
        The article code (<value-of select="@code"/>) must start with the right
        prefix (<value-of select="/inventory-list/@depcode"/>)
        for <value-of select="name"/>
      </assert>
    </rule>
  </pattern>
</schema>
```

This will result in:

The article code (EXP0234) must start with the right prefix (IMP) for Bananas

For more information: [Section 5.3, “More meaningful messages: <value-of>”](#).

2.6.5. Using variables

The last subject for this first example is variables. A variable allows you to compute some value only once and store it using a name. This name can then be used to retrieve the value in subsequent expressions. Variables are useful for efficiency reasons,^[5] but also for improving code readability and maintainability.

We use the value of the root's `depcode` attribute (`/inventory-list/@depcode`) in the assertion. But assertions are checked against all `article` elements, so this value is retrieved multiple times. For our simple input document in

[Example 2.1](#) there are only three of these, but what if these documents get bigger? It seems more efficient to retrieve the department code only once.

To do this, define a variable that holds the department code and then use it later in the assertion, as shown in [Example 2.5](#).

Example 2.5 – Using a variable to store the department code ([schematron-book-code/examples/introduction/example-3.sch](#))

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <let name="department-code" value="/inventory-list/@deptcode"/>
  <pattern>
    <rule context="article">
      <assert test="starts-with(@code, $department-code)">
        The article code (<value-of select="@code"/>) must start with the right
        prefix (<value-of select="$department-code"/>)
        for <value-of select="name"/>
      </assert>
    </rule>
  </pattern>
</schema>
```

The `<let>` element defines a variable called `department-code` and fills it with the value of the `deptcode` attribute of the root element.^[6] Whenever you need this value you can reference it using the name of the variable prefixed with a `$` character: `$department-code`.

You can define and use variables on almost any level in a Schematron schema. For instance, you could create a variable for the `code` attribute. This is not strictly necessary, but you might consider it to increase code comprehensibility, as shown in [Example 2.6](#).

Example 2.6 – Using a second variable to store the article code ([schematron-book-code/examples/introduction/example-3b.sch](#))

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <let name="department-code" value="/inventory-list/@deptcode"/>
  <pattern>
    <rule context="article">
      <let name="article-code" value="@code"/>
```

```
    <assert test="starts-with(@code, $department-code)">
      The article code (<value-of select="$article-code"/>) must start with the
right
      prefix (<value-of select="$department-code"/>)
      for <value-of select="name"/>
    </assert>
  </rule>
</pattern>

</schema>
```

For more information see [Section 5.4, “Declaring and using variables: <let>”](#).

[1] This initial presentation and many other background stories are still available at <https://www.schematron.com>.

[2] When this book talks about “the standard” or “the specification,” it’s referring to this document.

[3] See [Section 9.1, “Validating a Schematron schema”](#) on how to obtain these schemas.

[4] Although it looks like one, it’s not really an attribute but a namespace declaration. See [Appendix B](#) for details.

[5] Code efficiency is a bit of a tricky subject. Modern compilers are very good in optimizing code and might already do the work for you. Nonetheless, it certainly doesn’t hurt to optimize code yourself if you can.

[6] Strictly speaking we’re not storing the attribute’s value but creating a variable with the attribute as its value. I will discuss this later.

Chapter 3. Schematron in context

Schematron is an XML validation language. It expresses rules and limitations for the contents of XML documents. But Schematron is not the only way to validate XML, and to fully understand and appreciate it, it's important to know about the world surrounding it. This chapter dives into the broader world of XML document validation: what it is, why we need it, what kinds there are, and what to use when.

3.1. Validating XML documents

This section serves as an introduction to the general subject of validating XML documents: Why do we need to validate anyway? How can we do this?

3.1.1. Restricting XML freedom

The XML language specification sets a number of general syntax rules for its documents. Here are some (informal) examples:

- The main building blocks for XML documents are elements (`<para>`, `<bank-account>`, `<MeasurementArea>`, etc.). They're written using angled brackets.
- Elements must be properly opened and closed (`<para>...</para>`, `<Empty/>`, etc.).
- Elements must be properly nested (`<p>...<b>...</b>...</p>`, not `<p>...<b>...</p>...</b>`).
- Elements can have attributes (`<p class="header">`, `<Measurement unit="cm" value="12.45">`, etc.).
- An XML document must have a single root element.
- The character set for element and attribute names is defined: letters, digits, dots, hyphens, and underscores. Names must start with a letter or underscore.
- The names of elements and attributes are case-sensitive (`<email>` is not the same element as `<EMail>` or `<EMAIL>`).

However, within these rules you have complete freedom. When creating an XML document from scratch, you can call some element `<business-email>`,

<businessEMail>, <BusinessEMail>, or just <BE>. It's up to you. You can choose which elements appear inside which elements (nesting), whether to use attributes, and so on. That's a lot of freedom!

Assume two separate developers have been asked to create XML documents with customer information. One comes up with [Example 3.1](#).

Example 3.1 – Some way to convert customer information into XML

```
<CUSTOMER ID="12345" NAME="Johnny Coder" EMAIL="jc@acme.com" COMPANY="ACME"/>
```

The other, with clearly more verbose preferences, encodes the same information as [Example 3.2](#).

Example 3.2 – Another way to convert customer information into XML

```
<customer>
  <identification-code>12345</identification-code>
  <full-person-name>Johnny Coder</full-person-name>
  <business-email-address>jc@acme.com</business-email-address>
  <company-working-for>ACME</company-working-for>
</customer>
```

We can bicker about what is the right approach (which XML designers do), but it's obvious that systems written for [Example 3.1](#) won't communicate with systems designed for [Example 3.2](#). Although the information carried is the same and they're both XML, the formats used are different and therefore incompatible.

When systems are isolated, for instance when they're only used in-company, these differences don't matter. But as soon as the format is used for communicating information outside of system or company boundaries, it becomes important that all systems involved use the same definition, the same format. Therefore, we need a way to define an XML format so it can be widely used.

This same situation, on a larger scale, occurs with official XML standards. After usually long and detailed discussions, committees establish the names, meaning, and usage of elements and attributes. For example, DocBook (the standard used to write this book) defines that a paragraph is surrounded by a

<para> element. And that means not <Para>, <PARA>, or <p>. Another example is the Schematron standard, which defines that a Schematron schema starts with the <schema> element, not <schematron> or <SCHEMA>. If these rules were not there, nobody would know what was meant, and it would be impossible to process the XML the way it was intended.

So XML design gives freedom, but to make it usable we have to restrict that freedom. That's what XML schema languages are about.

3.1.2. How to define XML formats

To align our non-communicating developers of [Example 3.1](#) and [Example 3.2](#), or the users of an important XML standard, we need to define, to lay down, the XML format. There are several approaches to this:

- **Informal:** Provide one or more examples and maybe a little written documentation. This happens a lot and is usually good enough for situations where the document format is simple, the user community small, and conformance not very critical. But as soon as one of these preconditions goes away, it becomes a different story.
- **Less informal:** Document, for humans, exactly how the XML must be formatted. This is already considerably better, but it has several drawbacks:
 - You need to find a documentation format that communicates the intent. For XML, with its elements, attributes, nesting, and so on, this is not particularly easy. There's no agreed upon common way to do this.
 - You need to actually write the documentation, a rather tedious job according to most people. Once written, it must be maintained and kept up to date with the inevitable changes ahead, a chore not to be underestimated.
 - Most importantly, there's no way to automate checking whether a document conforms to the specification. Manual checking that a long document conforms to the specification, or checking a document against a complex specification, is humanly impossible: too much detail. Automation quickly becomes a necessity.

Still, XML formats are often defined in prose, but mostly for documentation purposes. For instance, this book contains appendices detailing XML formats, including Schematron itself ([Appendix C](#)).

When documentation (the passing of intent and meaning from one human to the other) is important enough, it's a good idea to provide it.

- **Formal:** Write a specification in a formal language that a computer can use to establish whether an XML document conforms or not. This process is called *validation*. There are several validation languages, Schematron being one of them.

3.1.3. Validating XML documents

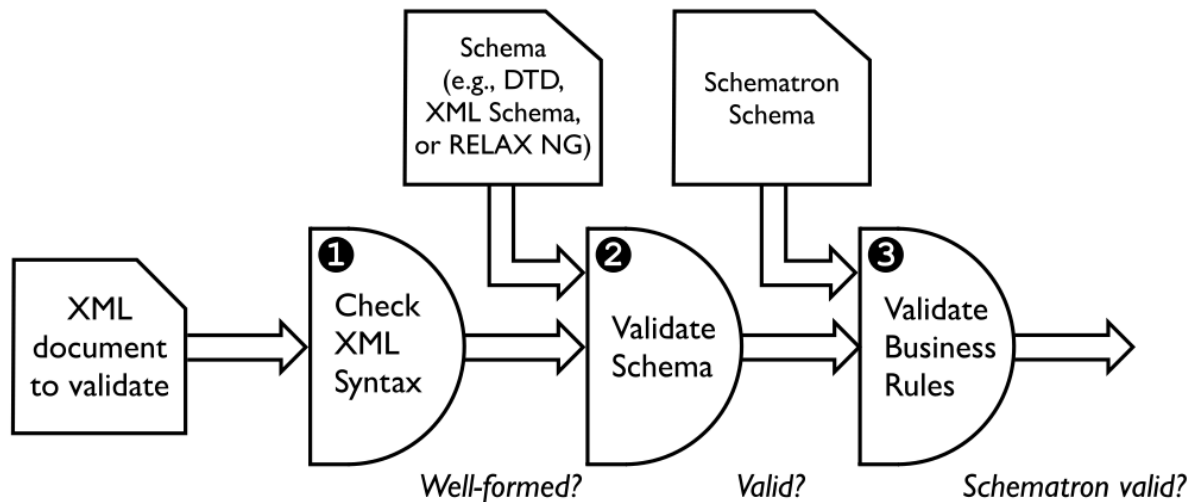
Validation can be defined as: formally establishing whether an XML document conforms to the set of rules for its document type:

- Are all element and attribute names correct?
- Is the nesting of elements okay?
- Is all necessary information there?
- Is there unexpected information?
- Does the information conform to business rules?
- ...

Every document type that needs to undergo validation has its own rule set. The XML contents for a book are different from, let's say, a tax form, or a patient record. We should be able to verify, in an automated way, whether documents conform to the rules for their intended document type.

Validation of XML documents is usually done in two or three stages ([Figure 3.1](#)).

Figure 3.1. The three stages for a full XML document validation



1. The first threshold is: does the document conform to the basic syntax rules of XML? Are the angle brackets, the element and attribute names, the nesting, and so on, OK? If not, you won't get far because no XML processing software will be able to parse and use it.

When a document passes this stage, it's called *well-formed*. Well-formedness is a requirement for all XML document types, regardless of their meaning.

2. Once a document passes stage 1, and is therefore well-formed, you can check it against the rule-set for its document type. A formalized version of such a rule-set is called a *schema*.

When a document passes all checks prescribed by the schema, it is called *valid*.

There are several flavors for schemas. The most important ones are DTD, XML Schema, and RELAX NG. More about this in [Section 3.2, "The main schema languages"](#).

Schemas check things like element and attribute names, nesting, whether required elements and attributes are there, etc. They have limited support for checking more complicated business rules.

3. Once a document passes stage 1 and 2, you can check it against the business rules for this type of document. For example, this date must be before that one, the sum of the weights of the parcels must not exceed some threshold, etc.

The main language for defining business rules for an XML document type is *Schematron*. A rule-set expressed in the Schematron language is called a *Schematron schema*. There's no official term for a document that passes Schematron defined checks, but in this book it is called *Schematron valid*.

Now if only the world would be as structured as suggested by the neatly numbered validation stages of [Figure 3.1](#). Stage 1 will always be there but there are many possible variations in applying stage 2 and stage 3:

- You don't necessarily need to pass stage 2 before doing a Schematron validation in stage 3. They're more or less independent. However, in most applications, a document is first confirmed valid (stage 2) before Schematron validation (stage 3) is performed.

The reason for performing stage 2 before stage 3 is that Schematron schemas often rely on documents being valid. For instance, suppose two dates you need to compare are in required attributes. The schema (stage 2) checks whether these attributes are there and are formatted as valid dates. The Schematron schema (stage 3) can then compare them, knowing they're present and formatted correctly.

- You could create a Schematron schema that partly takes on the role of a schema. Most schema checks can be expressed in a Schematron schema. An example of this can be found in [Section 9.3, "Using Schematron for schema validation"](#).

The main reason why this is sometimes done is messaging. Messages defined in a Schematron schema are completely under your control and can be made as user-friendly and informative as required. For schema validation this is not the case: you have to live with the messages the schema processor throws at you.

- With the advent of W3C XML Schema version 1.1, you can add limited business-rules checking to schemas, although this capability is not widely used. For instance, the “this date must be before that one” test can be validated using a version 1.1 W3C XML Schema. An example of this can be found in [Section 3.2.2.1, “W3C XML Schema version 1.1”](#).

A good reason to avoid this feature, and use Schematron instead, is the rather user-unfriendly messages produced by the currently available W3C XML Schema validators.

- And then there’s a large grey area of things that can be checked almost equally well by both schemas and Schematron schemas. For instance, both can check the following:
 - References from one element to another using identifiers.
 - Uniqueness of values (for example, identifiers).
 - Whether values of elements and attributes conform to specific rules (like enumerations, regular expressions, length, minimums and maximums, etc.).

3.2. The main schema languages

For those that are new to the world of validation, this section contains a short introduction to the main languages: DTDs, W3C XML Schema, RELAX NG, and, of course, Schematron. Apart from Schematron, I don’t teach these languages; I provide an overview and show a few examples. [Appendix F](#) lists additional resources if you want to dive further into any of these languages.

I use the XML document in [Example 3.3](#) as an example of a document to validate. It contains data about parcels, their weights, and shipment dates:

Example 3.3 – Valid example document used for introducing the main Schema languages (schematron-book-code/data/parcels-valid.xml)

```
<parcels max-weight="100" delivery-date="2021-10-10">
  <parcel weight="50" date="2021-09-29">
    <contents>A large quantity of mouth masks</contents>
  </parcel>
  <parcel weight="45" date="2021-09-12">
    <contents>Toys for children</contents>
  </parcel>
</parcels>
```

Here is an informal description of some rules you could apply to documents like the one shown in [Example 3.3](#):

1. Must have a structure as in [Example 3.3](#), and all attributes in the example are required.
2. There must be one or more `<parcel>` elements.
3. The content description (in the `<content>` element) must be at least five characters long.
4. The sum of the weights of the separate parcels (in the `weight` attributes) must not exceed the maximum weight, as specified on the root element in the `<max-weight>` attribute.
5. The date as set on a parcel (in the `date` attribute) must be at least 10 days before the delivery date specified on the root element in the `delivery-date` attribute.

Rules 1 through 3 are examples of rules that can be checked by regular schemas. Rules 4 and 5 are business rules, which can be checked using Schematron.

To try out the examples, you need validation software. It's easiest to use an IDE that has validation built in, for instance oXygen (see [Section 1.4, “The oXygen IDE”](#)). If you can't or don't want to use oXygen, you can also run a Schematron validation from the command line. See [Section 4.5.2.1, “Running SchXslt from the command line”](#).

3.2.1. Document Type Definition (DTD)

DTD, Document Type Definition, is an ancient validation language that goes way back to the previous century and XML's predecessor, SGML. DTDs are not written in XML but in a language of its own that looks a little bit like XML but isn't. [Example 3.4](#) shows the structural rules for [Example 3.3](#), expressed as a DTD.

Example 3.4 – A DTD for validating [Example 3.3](#) (schematron-book-code/examples/context/parcels.dtd)

```
<!ELEMENT parcels (parcel)+>
<!ATTLIST parcels
  xmlns CDATA #FIXED ''
  delivery-date CDATA #REQUIRED
  max-weight CDATA #REQUIRED>
```

```

<!ELEMENT parcel (contents)>
<!--ATTLIST parcel
  xmlns CDATA #FIXED ''
  date CDATA #REQUIRED
  weight CDATA #REQUIRED-->
<!--ELEMENT contents (#PCDATA)-->
<!--ATTLIST contents
  xmlns CDATA #FIXED ''-->

```

DTDs have several drawbacks, including their non-XML format, no support for namespaces, no data type support, and more. However, they are still used a lot.

More background information and examples can be found on the Wikipedia page for DTD: https://en.wikipedia.org/wiki/Document_type_definition.

3.2.2. W3C XML Schema

A much more modern approach to schema validation is the W3C XML Schema language. This language is expressed in XML and contains features to support namespaces, data types, reusable components, and more.

[Example 3.5](#) shows the structural rules for [Example 3.3](#), expressed as a W3C XML Schema.

Example 3.5 – The W3C XML Schema for validating [Example 3.3](#) (schematron-book-code/examples/context/parcels.xsd)

```

<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
  elementFormDefault="qualified">
  <xs:element name="parcels">
    <xs:complexType>
      <xs:sequence>
        <xs:element maxOccurs="unbounded" name="parcel">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="contents">
                <xs:simpleType>
                  <xs:restriction base="xs:string">
                    <xs:minLength value="5"/>
                  </xs:restriction>
                </xs:simpleType>
              </xs:element>
            </xs:sequence>
            <xs:attribute name="weight" type="xs:positiveInteger"
              use="required"/>
            <xs:attribute name="date" type="xs:date" use="required"/>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
    </xs:element>
  </xs:schema>

```

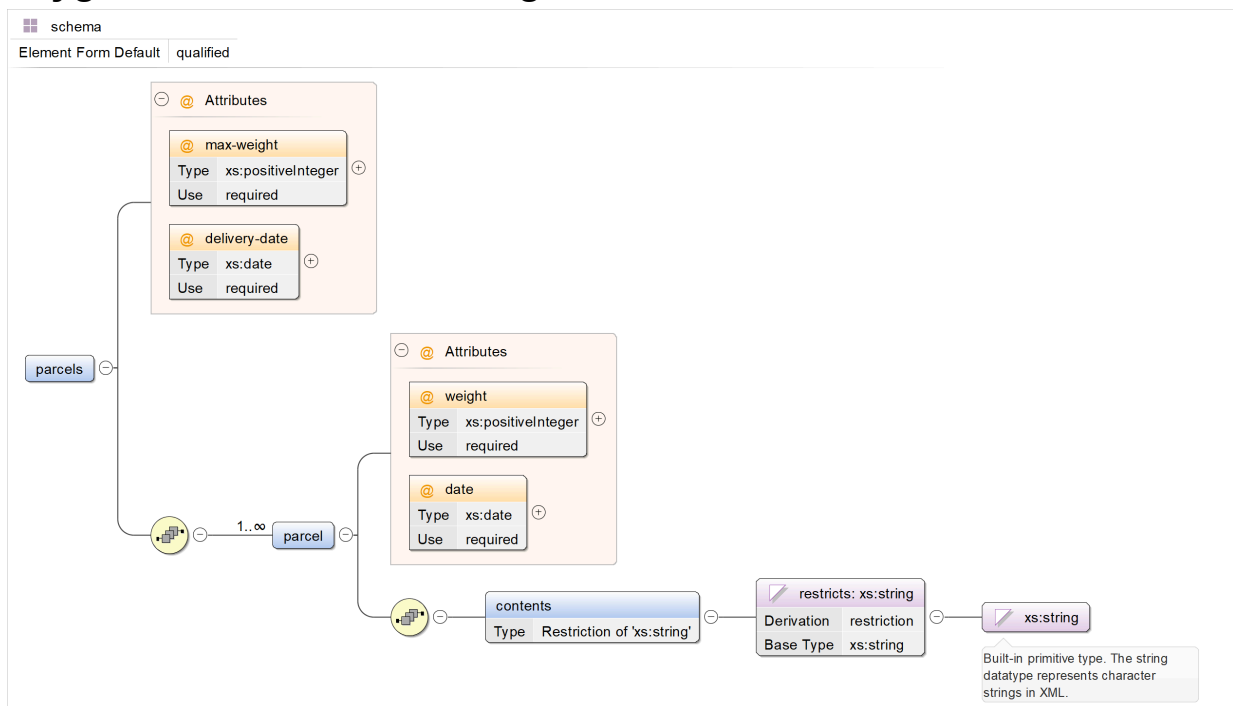
```

</xs:sequence>
<xs:attribute name="max-weight" type="xs:positiveInteger"
  use="required"/>
<xs:attribute name="delivery-date" type="xs:date" use="required"/>
</xs:complexType>
</xs:element>
</xs:schema>

```

[Example 3.5](#) also highlights a frequent criticism of XML Schema: it's a very complicated language. It's difficult to create and maintain XML Schemas without some kind of user interface support. For instance, oXygen allows you to draw W3C XML Schemas as a diagram and translates this into the underlying XML. [Figure 3.2](#) shows how the user interface in oXygen represents the XML Schema in [Example 3.5](#).

Figure 3.2. The W3C XML Schema for validating [Example 3.3](#) in oXygen's XML Schema design view



More background information and examples can be found on the XML Schema Wikipedia page:

[https://en.wikipedia.org/wiki/XML_Schema_\(W3C\)](https://en.wikipedia.org/wiki/XML_Schema_(W3C)).

3.2.2.1. W3C XML Schema version 1.1

[Example 3.5](#) uses W3C XML Schema version 1.0, which is currently the most commonly used version of the standard. The newer version 1.1 has limited support for business rule checking. For instance, [Example 3.6](#) shows a version 1.1 XML Schema that adds a rule about total weight to [Example 3.5](#).

Example 3.6 – The W3C XML Schema version 1.1 for validating [Example 3.3](#) (schematron-book-code/examples/context/parcels-v1.1.xsd)

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
  xmlns:vc="http://www.w3.org/2007/XMLSchema-versioning"
  elementFormDefault="qualified"
  vc:minVersion="1.1">
  <xs:element name="parcels">
    <xs:complexType>
      <xs:sequence>
        <xs:element maxOccurs="unbounded" name="parcel">
          <xs:complexType>
            <xs:sequence>
              <xs:element name="contents">
                <xs:simpleType>
                  <xs:restriction base="xs:string">
                    <xs:minLength value="5"/>
                  </xs:restriction>
                </xs:simpleType>
              </xs:element>
            </xs:sequence>
            <xs:attribute name="weight" type="xs:positiveInteger"
              use="required"/>
            <xs:attribute name="date" type="xs:date" use="required"/>
          </xs:complexType>
        </xs:element>
      </xs:sequence>
      <xs:attribute name="max-weight" type="xs:positiveInteger"
        use="required"/>
      <xs:attribute name="delivery-date" type="xs:date" use="required"/>

      <!-- The sum of all parcels weights must not exceed maximum weight: -->
      <xs:assert test="sum(parcel/@weight) le xs:double(@max-weight)"/>

    </xs:complexType>
  </xs:element>
</xs:schema>
```

Let's validate an invalid version of [Example 3.3](#) against this schema. For instance, in [Example 3.7](#) the total weight of the parcels exceeds the maximum weight (as specified by the `max-weight` attribute on the `<parcels>` element).

Example 3.7 – Invalid version of [Example 3.3](#) (schematron-book-code/data/parcels-invalid-2.xml)

```
<parcels max-weight="100" delivery-date="2021-10-10">
  <parcel weight="50" date="2021-09-29">
    <contents>A large quantity of mouth masks</contents>
  </parcel>
  <parcel weight="55" date="2021-10-05">
    <contents>Toys for children</contents>
  </parcel>
</parcels>
```

Validating [Example 3.7](#) against the schema in [Example 3.6](#) results in (the exact message depends on which validation processing software you're using, this one is produced by Xerces):

Example 3.8 – The error message when validating [Example 3.7](#) with the schema of [Example 3.6](#)

```
Assertion evaluation ('sum(parcel/@weight) le xs:double(@max-weight)')
for element 'parcels' on schema type '#AnonType_parcels' did not succeed.
```

I hope you agree with me that that's not a particularly user-friendly message. Most users, with the exception of the schema designer, wouldn't be able to make much sense of it. Avoiding technical mumbo-jumbo like this is one of Schematron's strengths.

3.2.3. RELAX NG

RELAX NG is a language developed in response to the complexity of XML Schema. It also tries to fix some of its shortcomings. [Example 3.9](#) shows the structural rules for [Example 3.3](#), expressed in RELAX NG.

Example 3.9 – The RELAX NG Schema for validating [Example 3.3](#) (schematron-book-code/examples/context/parcels.rng)

```
<grammar ns="" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns="http://relaxng.org/ns/structure/1.0"
  xmlns:doc="http://relaxng.org/ns/compatibility/annotations/1.0"
  datatypeLibrary="http://www.w3.org/2001/XMLSchema-datatypes">
  <start>
    <element name="parcels">
      <attribute name="delivery-date">
        <data type="date"/>
      </attribute>
      <attribute name="max-weight">
        <data type="positiveInteger"/>
      </attribute>
      <oneOrMore>
        <element name="parcel">
          <attribute name="date">
```

```

        <data type="date"/>
      </attribute>
      <attribute name="weight">
        <data type="positiveInteger"/>
      </attribute>
      <element name="contents">
        <text/>
      </element>
    </oneOrMore>
  </element>
</start>
</grammar>

```

I think we can agree that [Example 3.9](#) is easier to understand than its W3C XML Schema counterpart in [Example 3.5](#). Even if you don't know any RELAX NG, the basics are easy to understand.

RELAX NG also has a non-XML notation, called RELAX NG Compact Syntax. [Example 3.10](#) is the exact same schema as [Example 3.9](#) expressed in RELAX NG Compact Syntax.

Example 3.10 – The RELAX NG Compact Syntax schema for validating [Example 3.3](#) (`schematron-book-code/examples/context/parcels.rnc`)

```

default namespace = ""
namespace doc = "http://relaxng.org/ns/compatibility/annotations/1.0"
namespace xsi = "http://www.w3.org/2001/XMLSchema-instance"
start =
  element parcels {
    attribute delivery-date { xsd:date },
    attribute max-weight { xsd:positiveInteger },
    element parcel {
      attribute date { xsd:date },
      attribute weight { xsd:positiveInteger },
      element contents { text }
    }+
  }

```

You can find more background information and examples on the RELAX NG Wikipedia page: https://en.wikipedia.org/wiki/RELAX_NG.

3.2.4. Schematron

This overview of the main schema languages concludes with the subject of this book: Schematron. As I described earlier, Schematron is a language for checking the business rules of an XML document. Let's assume our XML

survived the scrutiny of a validation, whether against a DTD, W3C XML Schema, or RELAX NG schema. What's left to check is:

- The sum of the weights of the separate parcels (in the `weight` attributes) must not exceed the maximum weight, as specified on the root element in the `max-weight` attribute.
- The date as set on a parcel (in the `date` attribute) must be at least 10 days before the delivery date, as specified on the root element in the `delivery-date` attribute.

[Example 3.11](#) shows a Schematron schema for checking these rules.

Example 3.11 – The Schematron schema for validating [Example 3.3](#) (schematron-book-code/examples/context/parcels.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="/*">
      <assert test="sum(parcel/@weight) le xs:double(@max-weight)">
        The total weight is too high
      </assert>
    </rule>
  </pattern>
  <pattern>
    <rule context="parcel">
      <assert test="(xs:date(*/@delivery-date) - xs:date(@date)) gt
        xs:dayTimeDuration('P10D')">
        The parcel's date must be more than 10 days before the delivery date
      </assert>
    </rule>
  </pattern>
</schema>
```

The syntax for Schematron and how a Schematron schema works are the subject of this book. What [Example 3.11](#) clearly shows is one of Schematron's strengths: defining your own messages. Both error messages are defined in the schema, and that's exactly how they will appear.

Chapter 4. Applying Schematron

This chapter is about applying Schematron. Given an XML document and corresponding Schematron schema, how do you perform a Schematron validation? How do you get it singing and dancing? As you'll learn, there are several ways to go about this.

This chapter starts simple but gets increasingly more complicated and technical. If you're just interested in applying Schematron, you can stop at [Section 4.3, “Raw Schematron validation”](#) and continue with [Chapter 5](#).

The examples in this chapter use the parcel document introduced in [Chapter 3](#) and the accompanying Schematron schema originally shown in [Example 3.11](#). The schema is repeated in [Example 4.1](#) and the parcel document is repeated in [Example 4.2](#).

Example 4.1 – The Schematron schema to validate [Example 4.2](#) with (a repeat of [Example 3.11](#)) (schematron-book-code/examples/context/parcels.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="/*">
      <assert test="sum(parcel/@weight) le xs:double(@max-weight)">
        The total weight is too high
      </assert>
    </rule>
  </pattern>
  <pattern>
    <rule context="parcel">
      <assert test="(xs:date(/*/@delivery-date) - xs:date(@date)) gt
        xs:dayTimeDuration('P10D')">
        The parcel's date must be more than 10 days before the delivery date
      </assert>
    </rule>
  </pattern>
</schema>
```

Example 4.2 – Valid example document, used for explaining how to apply Schematron (a repeat of [Example 3.3](#)) (schematron-book-code/data/parcels-valid.xml)

```
<parcels max-weight="100" delivery-date="2021-10-10">
  <parcel weight="50" date="2021-09-29">
    <contents>A large quantity of mouth masks</contents>
  </parcel>
  <parcel weight="45" date="2021-09-12">
    <contents>Toys for children</contents>
  </parcel>
</parcels>
```

4.1. Creating Schematron schemas

Creating a Schematron schema is just like creating any other XML document: you open a new document in your favorite editor and start typing. Use Notepad, Emacs, VI, Visual Studio Code, oXygen, Eclipse, or whatever you like: an XML document is just a text file with lots of angle brackets.

Having said that, it is nice to get at least some support on the XML level and preferably also the Schematron level:

- XML-aware editors help you create well-formed documents. They can help you keep open and close tags aligned and make sure your document follows XML syntax rules.
- XML-schema-aware editors also validate documents against a particular XML schema. They can tell you what elements and attributes are valid at any point in your document to make it easier to create a valid XML document. You can find a list of XML-schema-aware editors at https://en.wikipedia.org/wiki/Comparison_of_XML_editors.
- A Schematron-aware editor can do everything a schema-aware editor can do and, at the same time, validate your document against a Schematron schema. The only editor that I know of that does this (in 2022) is oXygen.

If you don't have (or want) the luxury of a Schematron-aware editor, I would advise you to start by using a template file, for instance the one in [Example 4.3](#).

Example 4.3 – Empty Schematron schema template (schematron-book-code/templates/empty-schema.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern/>
</schema>
```

Using a template will, at least, save you the effort of correctly typing the namespace URI and the root element. Depending on your environment and Schematron processor you might need a different value for the `queryBinding` attribute. More about this in [Chapter 7](#).

Since Schematron schemas are XML documents, it's entirely possible to validate these themselves. More about this in [Section 9.1, “Validating a Schematron schema”](#).

4.2. IDE-based Schematron validation

Suppose you have a Schematron schema (either written yourself or acquired) and some XML document(s) you want to validate. The easiest way to do this is by using an IDE (Integrated Development Environment) that supports Schematron validation. As far as I know, the only editor currently (in 2022) capable of doing this is oXygen.

To show you what this looks like, let's validate a Schematron-invalid version of the parcels document in [Example 4.2](#) (see [Example 4.4](#)).

Example 4.4 – A Schematron-invalid version of the parcels document in [Example 4.2](#) (schematron-book-code/data/parcels-schematron-invalid.xml)

```
<parcels max-weight="100" delivery-date="2021-10-10">
  <parcel weight="50" date="2021-10-08">
    <contents>A large quantity of mouth masks</contents>
  </parcel>
  <parcel weight="55" date="2021-09-12">
    <contents>Toys for children</contents>
  </parcel>
</parcels>
```

[Example 4.4](#) is invalid because:

- The sum of the parcel weights, in the `parcel/@weight` attributes, is 105, which is more than the maximum weight of 100, as specified in the `/parcels/@max-weight` attribute.
- The date of the first parcel in its `parcel/@date` attribute (2021-10-08) is less than 10 days before the delivery date in the `/parcels/@delivery-date` attribute (2021-10-10).

To validate this document using oXygen:^[7]

- Open the Schematron schema (schematron-book-code/examples/context/parcels.sch).
- Open the invalid document (schematron-book-code/data/parcels-schematron-invalid.xml).
- With the document (not the schema!) having focus, from the menu choose: **Document > Validate > Validate with....**
- Provide a reference to the Schematron schema in the **URL:** input box. Because this file is already open, you can easily select it by pressing the down arrow at the end of the input box. oXygen determines the schema type for you. On my machine the result looks like [Figure 4.1](#).

Figure 4.1. Selecting a schema in oXygen

 Validate with

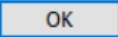
URL: 

Schema type:

☐ Embedded Schematron rules

Public ID:

Schematron phase:

- Press **OK** and the validation starts. As both the document and the schema are very small, this won't take long.

oXygen underlines the places where there is an error according to the schema:

Figure 4.2. The invalid parcels document validated by oXygen

```
1 <parcels max-weight="100" delivery-date="2021-10-10">
2   <parcel weight="50" date="2021-10-08">
3     <contents>A large quantity of mouth masks</contents>
4   </parcel>
5   <parcel weight="55" date="2021-09-12">
6     <contents>Toys for children</contents>
7   </parcel>
8 </parcels>
```

The oXygen results window (usually at the bottom of your screen) provides the error messages. Please notice that these messages come straight from the Schematron schema.

Figure 4.3. The results window for [Figure 4.2](#)

Results		
	Info	Description - 2 items
		Resource
parcels-schematron-invalid.xml, schema "parcels.sch" (2 items)		
	-	E [ISO Schematron] The total weight is too high
	-	E [ISO Schematron] The parcel's date must be more than 10 days before the delivery date

Validation messages also appear when you move the cursor. For instance, if you position the cursor over the `max-weight="100"` attribute, the following message appears:

Figure 4.4. Validation message shown at the bottom of the editing window



For IDE-based Schematron validation, this is it: open a document, validate it against the appropriate Schematron schema, study the results, fix, and repeat.

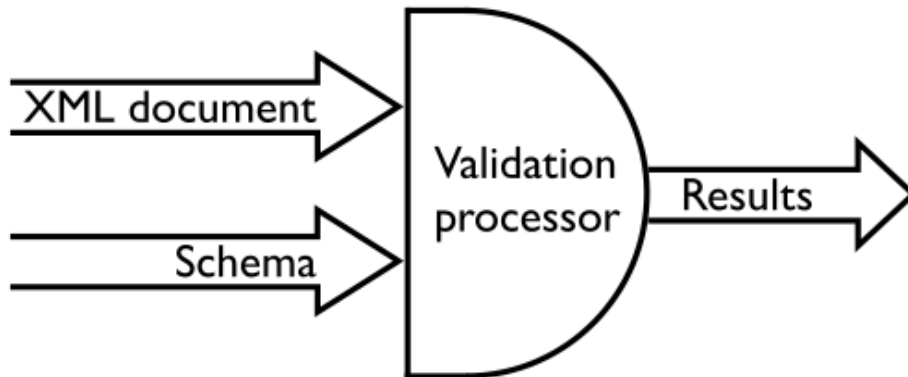
4.3. Raw Schematron validation

If you perform validation in an IDE, it shows you the validation results in a user-friendly way. However, the actual validation (the process that generates the validation results) is hidden from view. It's like driving a car and not looking under the hood: fine when you just need to drive from A to B, but sometimes you want to know a little more.

This section looks under the hood of Schematron validation and explains how it's actually done. It covers validation processors, XSLT transformations, and other more technically oriented information. If you're not interested in this level of detail, you can skip this section and go straight to the next chapter, where I explain the basics of the Schematron language.

Schematron validation (or any type of XML validation) is performed by a processor that takes an XML document and a schema and produces a validation result (see [Figure 4.5](#)).

Figure 4.5. A schematic representation of a validation processor



There are cases where you need or want to use this validation machinery directly. Maybe you're a seasoned command-line jockey and think clicking GUI menus is only for rookies. Or you need to incorporate validation as an automated step in a pipeline or other process. Or you have no IDE available and need to perform a validation directly.

Validation processors are often Java or .NET components, so they can be incorporated into bigger systems and tool chains. For instance, you can do W3C XML Schema validation from the command line or in a tool chain with the [Saxon EE \(Enterprise Edition\)](#) processor (the commercial version, not the free Home Edition).

While there are processors that can validate Schematron, current (in 2022) processors are rather complicated and rely on pre-compilation and an underlying XSLT processor. I'll dive into this, but first I need to discuss the results of a validation and how to represent them.

4.4. Validation results

Let's take a closer look at the results of a validation. What could a validation tell us about the document it validated?

1. As a bare minimum, just "valid" or "invalid."
2. Since that's not very helpful, processors usually specify what's wrong and the location of the problem (line/column number).
3. A validation processor might also produce a machine-readable report, most likely another XML document, that you can use in a tool chain, for instance to produce a customized report.

An IDE can only give you result 1 and result 2. The messages are presented in a list and the problem locations are marked in some way (often with red squiggly lines).

When you use a raw validation processor, from the command line or otherwise, you usually see result 2 as output on your console. Result 1, valid or invalid, is often reflected as the tool's process execution status.

Result 3, the validation results in machine-readable format, is rarer. Schematron however has defined such a format. It's called SVRL (Schematron Validation Reporting Language), an XML vocabulary for reporting the results of a Schematron validation. And although the part of the standard that describes SVRL is informative (meaning processor builders don't have to implement it), most existing Schematron validation processors produce SVRL.

4.4.1. SVRL

SVRL stands for Schematron Validation Reporting Language. It's an XML-based format that reports the results of a Schematron validation. You can find an SVRL reference in [Appendix D](#).

For example, let's validate [Example 4.4](#) again, using the SchXslt validation processor (see [Section 4.5.2, "The SchXslt Schematron processor"](#)). [Example 4.5](#) shows the result.^[8]

Example 4.5 – The SVRL generated when validating the document in [Example 4.4](#) with the schema from [Example 3.11](#).

```
<schematron-output xmlns="http://purl.oclc.org/dsdl/svrl">
  <active-pattern documents=".../parcels-schematron-invalid.xml"/>
  <fired-rule context="/*"/>
  <failed-assert location="/Q{}parcels[1]"
    test="sum(parcel/@weight) le xs:double(@max-weight)">
    <text>The total weight is too high</text>
  </failed-assert>
  <active-pattern documents=".../parcels-schematron-invalid.xml"/>
  <fired-rule context="parcel"/>
  <failed-assert location="/Q{}parcels[1]/Q{}parcel[1]"
    test="(xs:date(//*[@delivery-date] - xs:date(@date)) gt
xs:dayTimeDuration('P10D'))">
    <text>The parcel's date must be more than 10 days before the delivery date</text>
  </failed-assert>
  <fired-rule context="parcel"/>
</schematron-output>
```

- An SVRL document uses the <http://purl.oclc.org/dsdl/svrl> XML namespace.

- An SVRL document is basically a linear list of what the Schematron processor did. A pattern that becomes active produces an <active-pattern> element, a rule that fires produces a <fired-rule> element (I'll get to what patterns and rules are in [Chapter 5](#)).
- Most interesting are the two <failed-assert> elements, which contain error messages.

You can post-process SVRL to format the validation results to your liking.

4.4.2. XVRl

A new kid on the “validation results in machine-readable format” block is XVRl (eXtensible Validation Report Language). This format, introduced by the XProc 3.0^[9] community group, is an attempt to create a unified vocabulary for validation reports for all validation languages. The validation types incorporated in XProc 3.0, including Schematron, can report their results in XVRl. [Example 4.6](#) shows what [Example 4.5](#) looks like, expressed in XVRl.

Example 4.6 – The XVRl equivalent of [Example 4.5](#)

```
<report xmlns="http://www.xproc.org/ns/xvrl">
  <metadata>
    <timestamp>...</timestamp>
    <document href=".../parcels-schematron-invalid.xml"/>
    <schema href=".../parcels.sch" schematypens="http://purl.oclc.org/dsdl/schematron"/>
  </metadata>
  <detection severity="error">
    <location xpath="/Q{}parcels[1]"/>
    <message> The total weight is too high</message>
  </detection>
  <detection severity="error">
    <location xpath="/Q{}parcels[1]/Q{}parcel[1]"/>
    <message> The parcel's date must be more than 10 days before
      the delivery date</message>
  </detection>
</report>
```

Because XVRl is a new language, not specifically Schematron related, and currently implemented only by XProc 3.0, I won't say more about it here. See <https://spec.xproc.org/master/head/xvrl/> or the XVRl GitHub repository <https://github.com/xproc/xvrl> to learn more.

4.5. Schematron validation processors

There are several Schematron validation processors, but only two are relevant enough to earn a separate description: “skeleton” XSLT and SchXslt. I use the SchXslt processor in this book.

If you're interested in other processors (for instance those written in/for XQuery or Java), see the software section in the README.md file at <https://github.com/Schematron/awesome-schematron>.

4.5.1. The “skeleton” XSLT Schematron processor

Until recently, the official Schematron processor was the “skeleton” XSLT implementation of Schematron. It was originally created by Rick Jelliffe, released in 2000, and maintained by a number of people. You can find it at <https://github.com/Schematron/schematron>.

However, its GitHub repository was archived and made read-only in October 2020. Maintenance has stopped. Its home page advises you to use the SchXslt processor instead.

Nonetheless, this processor is still used a lot. Given its rather recent archiving (I'm writing this early 2022), it's probably still safe to use. If your software tool chain already contains this processor, there seems to be no urgent need to replace it. But if you have a choice, use the SchXslt processor.

The operation of the skeleton XSLT Schematron processor is similar to that of the SchXslt processor. You use a chain of XSLT stylesheets to produce an XSLT version of your Schematron schema. You then use this generated stylesheet to validate the document(s) against the schema. This principle is discussed in detail in the next section.

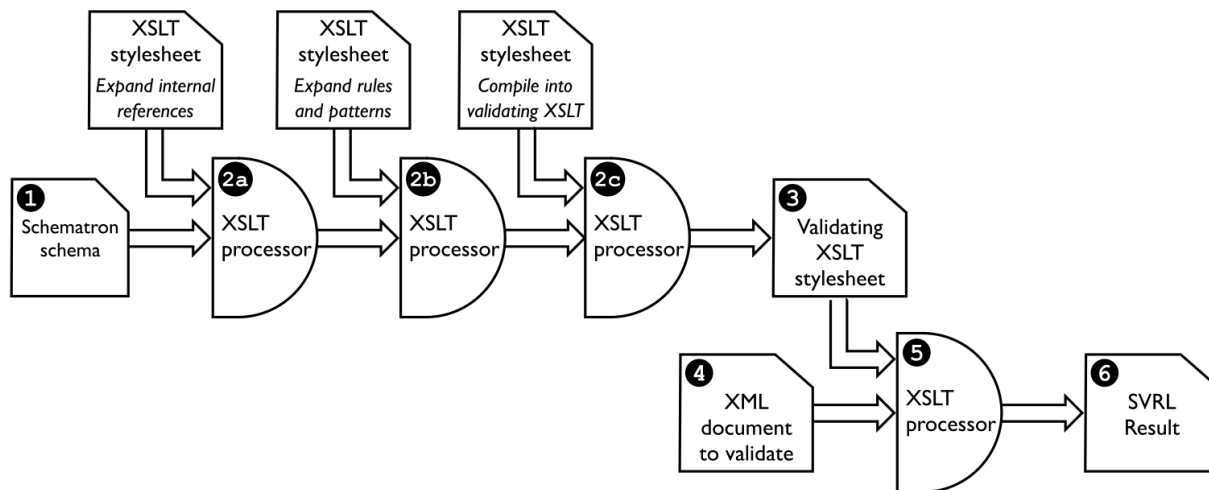
4.5.2. The SchXslt Schematron processor

At the XML Prague conference in 2019, David Maus introduced a different way of doing Schematron validation in his talk, “Ex-post rule match selection: A novel approach to XSLT-based Schematron validation.”^[10] This was the beginning of the rise of the SchXslt Schematron processor.

In 2020, SchXslt became the official Schematron processor and development/maintenance of the skeleton was stopped. You can find SchXslt on GitHub: <https://github.com/schxslt/schxslt>.

The SchXslt processor uses the basic approach of the original skeleton processor, but produces different, more efficient, code. [Figure 4.6](#) shows how it works.

Figure 4.6. Schematic representation of the operation of the SchXslt processor



1. The process starts with the Schematron schema.
2. This schema is transformed by a chain of (currently three) XSLT transformations. These transformations are supplied by, and are part of, SchXslt:[\[11\]](#)
 - 2.a. The first transformation takes the original Schematron schema and expands all external references (<include> and <extends href=..."> elements).
 - 2.b. The second transformation expands the abstract rules and patterns.
 - 2.c. The third and last transformation compiles this into a validating XSLT stylesheet.
3. The result of this chain is a generated XSLT stylesheet.
4. Then the XML document we want to validate comes into play.
5. This document is transformed using the generated XSLT stylesheet.
6. The result of this transformation is the SVRL representation of the validation result.

SchXslt offers support so you can use it in several environments, including XProc, Ant, BaseX, and eXist. It can also be used from the command line, as described in the next section.

4.5.2.1. Running SchXslt from the command line

This final section of this chapter explains how to use SchXslt from the command line. This description is from early 2022. It's likely that details will change over time, so be careful.

- Have Java installed on your machine.
- Go to <https://github.com/schxslt/schxslt> and select the latest release (on the right-hand side of the screen).
- Download the command line jar file: `schxslt-cli.jar`.
- Open a command line window, navigate to the directory where `schxslt-cli.jar` is stored, and issue the command:

```
java -jar schxslt-cli.jar
```

This will provide you with an overview of the command line options.

- To perform a validation, issue the command:

```
java -jar schxslt-cli.jar -d {inputfile} -s {schemafilename} -o {svrl-outputfile} -v
```

The output file is an XML file that logs the entire validation process and contains any error messages. The `-v` option displays any error messages found. Each error message includes an XPath expression that shows where the error was.

[7] The instructions below are for version 24.0 or older, the exact commands/behavior may change with newer versions. oXygen has more ways/commands for performing Schematron and other validation. Since this is not an oXygen tutorial these won't be covered here.

[8] The SVRL output shown in [Example 4.5](#) (and the SVRL examples to come) is somewhat sanitized. Some irrelevant sections, like metadata, are left-out, and the file paths are shortened to an ellipsis (...).

[9] XProc is an XML-based programming language for processing documents in pipelines: chaining conversions and other steps together to achieve the desired results. See <https://xproc.org>.

[10] This talk is still online: <https://www.youtube.com/watch?v=1xJ6F9M1k1k>. A textual version can be found in the 2019 conference proceedings

(<https://www.xmlprague.cz/archive/>).

^[11] SchXslt also contains an XSLT stylesheet that combines these three steps into one.

Chapter 5. Schematron basics

In this chapter you will learn how to write a basic Schematron schema. It explains concepts like patterns, rules, assertions, and reports and describes how to tie these concepts together into a consistent, meaningful whole. I also describe the algorithm a Schematron processor follows when it processes a schema.

To get the most out of this chapter, you need a basic knowledge of XPath. [Appendix A](#) provides an overview of XPath that covers what you need to know.

I suggest that while you read this chapter, you try out some of these examples and run Schematron validations of your own or variations of the given examples.^[12] [Chapter 4](#) contains information on how to perform Schematron validations.

To experiment, it's easiest to use an IDE, for instance oXygen (see [Section 1.4, “The oXygen IDE”](#)). But you can also perform validations from the command line (see [Section 4.5.2.1, “Running SchXslt from the command line”](#)).

5.1. Setting up a Schematron schema

Let's start with a basic, but nonetheless valid, Schematron schema that does absolutely nothing (see [Example 5.1](#)).

Example 5.1 – An empty Schematron schema (`schematron-book-code/templates/empty-schema.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern/>
</schema>
```

Here are a few things to consider when looking at [Example 5.1](#):

- The root element of a Schematron schema is always `<schema>`, and there must be at least one `<pattern>` element. More about patterns in the next

section.

- The next thing to notice is that the XML vocabulary for Schematron is in a namespace (namespaces are explained in [Appendix B](#)):
`http://purl.oclc.org/dsdl/schematron`

You can usually forget about this, as long as you make sure all your Schematron XML is in this namespace. An

`xmlns="http://purl.oclc.org/dsdl/schematron"` on the root element, as in [Example 5.1](#), will take care of this.

- I also added a `queryBinding="xslt3"` attribute on the `<schema>` root element. The Schematron language allows you to specify the language used to define expressions. This is called the “query language binding.” The `queryBinding` attribute defines this.

In [Example 5.1](#), the value of the `queryBinding="xslt3"` attribute (`xslt3`) tells the Schematron processor to use XSLT 3.0 (and because of that XPath 3.1). You can forget about this for now, but if you who want to know more, [Chapter 7](#) is devoted to this topic.

The `queryBinding` attribute is optional, but if you omit it the default binding `xslt` is used. This means that your expressions are limited to what you can do in XSLT 1.0 (and because of that, XPath 1.0). Given the current (in 2022) state of the XML standards, which have advanced significantly from this version, that’s severely limiting.

Using other query language bindings

Your Schematron processor might complain about the `xslt3` query binding. In that case try the value `xslt2` first. If that still doesn’t work, try `xpath31`, `xpath3` or `xpath2`. Most of the examples in this book work fine with these settings too.

If none of these values work in your environment,^[13] it will be difficult to follow and try the examples in this book. I suggest you use one of the processors discussed in the book to learn Schematron. Afterwards, it should be easier for you transfer what you know to whatever query binding language your processor uses.

5.2. Patterns, rules, assertions, and reports

The core parts of Schematron are patterns, rules, assertions, and reports:

- A Schematron schema consists of patterns: `<pattern>` elements.
- Each pattern consists of rules: `<rule>` elements. A rule contains a query (using the query language specified in the `queryBinding` attribute), and when that query matches something, for example an element or attribute, in your XML instance, the rule is triggered. If there is more than one rule in a pattern, only the first rule triggered is executed.
- Each rule consists of assertions and/or reports: `<assert>` and/or `<report>` elements. When a rule gets executed, all assertions and reports in that rule are checked.

Let's illustrate this with a simple example. Suppose you need to validate XML documents in which there are `<ID>` elements that hold some identifier. These elements can occur in several places (see [Example 5.2](#)).

Example 5.2 – An XML document that contains `<ID>` elements (`schematron-book-code/data/data-with-ids.xml`)

```
<DATA>
  <ARTICLE>
    <NAME>BOOK</NAME>
    <ID>ABC12345</ID>
  </ARTICLE>
  <ARTICLE>
    <NAME>TOY</NAME>
    <ID>XYZ123456</ID>
  </ARTICLE>
</DATA>
```

For this example, let's say that identifiers must be 9 characters long and that any identifier that starts with the letter x must be reported as special.

[Example 5.3](#) contains a Schematron schema that performs this validation.

Example 5.3 – A Schematron schema that checks the length of the `<ID>` elements of [Example 5.2](#) (`schematron-book-code/examples/basics/check-data-with-ids.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="ID">
```

```

    <assert test="string-length(.) eq 9">
      An ID must be 9 characters long!
    </assert>
    <report test="starts-with(., 'X')">
      Special identifier found!
    </report>
  </rule>
</pattern>
</schema>

```

- `<pattern>` elements usually have no attributes (see [Chapter 6](#) for a few exceptions).

Every pattern in the schema is applied to every node in the document being validated.

- A `<rule>` element must have a `context` attribute. The contents of this attribute is an “XSLT match pattern.” The one in [Example 5.3](#), `context="ID"`, fires on every `<ID>` element.

When a rule fires, the node it triggered on becomes the context item, and the assertions and reports in that rule are processed.

Only the *first* rule triggered in a pattern fires. Any other rule in that pattern will not be executed on that context item.

- An `<assert>` or `<report>` element must have a `test` attribute. Its contents must be an XPath boolean expression. An assertion fails when the expression is `false`, a report succeeds when it’s `true`.

All assertions and reports in the rule that fires are processed.

A failed assertion or successful report results in a validation message, which comes from the text of the contents of the `<assert>` or `<report>` element.

Given this, when you validate [Example 5.2](#) against the schema in [Example 5.3](#), there will be two validation messages:

1. On the `<ID>ABC12345</ID>` element: **An ID must be 9 characters long!**
2. On the `<ID>XYZ123456</ID>` element: **Special identifier found!**

5.2.1. The Schematron processing algorithm

This section provides an overview of the algorithm a Schematron processor uses to apply a Schematron schema to an XML instance. Here are the steps:

1. Read the XML document and represent it internally using a tree-shaped data structure in the Schematron processor's memory. All parts of the XML document (elements, attributes, text, etc.) are represented as nodes in that tree.
2. The Schematron processor starts "walking" the tree and, one by one, visits every node. This includes not only elements and attributes but also the document node, text, comments, and processing instructions.
3. For every node in the tree, the processor loops over every `<pattern>` element in the Schematron schema.
4. In each pattern, it finds the first rule (`<rule>` element) whose `context` attribute matches the node being visited (the context node). In this case, *first* means "top-most" (or, in official XML terminology, "first in document-order"). It's not an error if no rule matches.
5. If a matching rule is found, process all assertions and reports (`<assert>` and `<report>` elements) inside that rule. It's not an error if no assertion or report fires.

Here is the algorithm expressed in a pseudo programming language (the numbers refer to the corresponding numbers in the list above):

Example 5.4 – The Schematron algorithm in a pseudo programming language

1. Read the input document and represent it as a tree
2. For every node in the input document
 3. For every pattern in the schema
 4. For every rule in the pattern
 - * If (the rule matches) Then
 5. For every assert and report in the rule
 - * Process asserts/reports
 - * Stop checking the rules for this pattern
 - Else
 - * Check next the rule in the pattern

The order of things

The official Schematron specification states that the order in which nodes are visited in step 2 of the algorithm above is implementation-defined. This means that a processor could jump at random through the nodes in the tree, as long as it visits them all. In practice, of course, processors will walk through the tree in an orderly, predictable fashion, most likely in depth-first order.[\[14\]](#)

The specification also states that the order in which patterns and assertions/reports are handled (step 3 and 5) is implementation-defined. Again, this will usually be predictable, most likely the order in which the patterns, assertions, and reports appear in the schema.

The bottom line is that you can't rely on the order in which a Schematron processor reads a document, processes patterns, or outputs messages. Another processor (or even the next version of the same processor) might do things differently. You can, however, rely on a processor to process the rules within a pattern in the order in which they appear in the schema.

5.2.2. Rule processing

Let's dwell a little longer on step 4 of the algorithm presented in the previous section: the processing of rules (<rule> elements). Suppose you have a Schematron schema for an imaginary publisher's catalog. Inside is a pattern that looks like [Example 5.5](#).

Example 5.5 – A pattern with three rules

```
<pattern>
  <rule context="book">
    ... (asserts and reports for book elements)
  </rule>
  <rule context="magazine">
    ... (asserts and reports for magazine elements)
  </rule>
  <rule context="*">
    ... (asserts and reports for other elements)
  </rule>
</pattern>
```

Now, suppose the Schematron processor is checking a <magazine> element. This matches the second rule in the pattern (context="magazine") and, therefore, causes all assertions and reports for magazines to be checked.

If you're not fully aware of the Schematron processing algorithm, you might presume that not only the second but also the third rule will execute. It's obvious, isn't it? context="*" means all elements, <magazine> is an element, so, ergo, this rule is processed.

Fortunately we know better. We read the rules inside a pattern as an if/then/else statement:

Example 5.6 – The algorithm for processing rules in a pseudo programming language

```
* If (it's a book element) Then
  ... (process the asserts and reports for book elements)
Else If (it's a magazine element) Then
  ... (process the asserts and reports for magazine elements)
Else
  ... (process the asserts and reports for other elements)
```

Or, if you come from an XSLT background, as an `<xsl:choose>`:

Example 5.7 – The algorithm for processing rules represented as an XSLT `<xsl:choose>` element

```
<xsl:choose>
  <xsl:when test="(it's a book element)">
    ... (process the asserts and reports for book elements)
  </xsl:when>
  <xsl:when test="(it's a magazine element)">
    ... (process the asserts and reports for magazine elements)
  </xsl:when>
  <xsl:otherwise>
    ... (process the asserts and reports for other elements)
  </xsl:otherwise>
</xsl:choose>
```

However, what if you want `<book>` and `<magazine>` elements to be checked both by the assertions and reports for those specific elements *and* the assertions and reports for any element? The solution is easy: Put the generic rules in a pattern of their own. Since all patterns are processed for all nodes, this will make sure the generic assertions/reports are also processed.

[Example 5.8](#) does this.

Example 5.8 – Using a pattern for generic rules

```
<pattern>
  <rule context="book">
    ... (asserts and reports for book elements)
  </rule>
  <rule context="magazine">
    ... (asserts and reports for magazine elements)
  </rule>
</pattern>
<pattern>
```

```
<rule context="*">
  ... (generic asserts and reports, for all elements)
</rule>
</pattern>
```

5.2.3. Assert and Report processing

The previous sections mention “assertions and reports being processed.” What does that mean?

Assertions and reports are tests performed on the contents of the XML document being validated:

- An assertion (`<assert test="...">` element) checks whether the condition in the `test` attribute is `false`. If so it reports a validation error.

An assertion flags something that is invalid.

- A report (`<report test="...">` element) does the reverse: it generates a message when the condition in the `test` attribute is `true`.

A report flags specific situations that require attention (or are interesting, peculiar, wonderful, celebration-worthy, or whatever).

report = not assert?

Programmers may be frowning now: so a report is equivalent to the boolean *not* of an assertion (or the other way around)? Yes, that’s correct. For instance, take this report:

```
<report test="...">...</report>
```

The following will produce the same message:

```
<assert test="not(...)">...</assert>
```

Then why are there two elements which, from a programming point of view, have almost exactly the same functionality? The difference is of course in their meaning, their goal, not in the technical implementation.

You’ll also find this difference reflected in the resulting SVRL: a triggered assertion results in a `<failed-assert>` element, a report in a `<successful-report>` element.

Schematron is a language with which you can assert whether a document is Schematron valid. It's therefore more natural to think in assertions when writing such a schema. As far as I've seen, most Schematron schemas use assertions only. Because of this I will focus on the `<assert>` element for the explanations and examples to come. However, be aware that everything said about the `<assert>` element also holds for the `<report>` element, with a negated test as the main difference.

Let's illustrate this with another example. Suppose you have a Schematron schema whose sole purpose is to check whether all `type` attributes, anywhere in the document, have a value of either `normal` or `special` and produce an understandable error message when this isn't the case. The simple Schematron schema in [Example 5.9](#) does this.

Example 5.9 – A simple Schematron schema that checks the value of every `type` attribute (`schematron-book-code/examples/basics/check-type-1.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="@type">
      <assert test="(. eq 'normal') or (. eq 'special')">
        The type must be normal or special
      </assert>
    </rule>
  </pattern>
</schema>
```

If you run this schema against the document in [Example 5.10](#), the `type` attribute on the fourth `<thing>` element triggers this error message: **The `type` must be normal or special**

Example 5.10 – A document with an invalid `type` attribute according to the Schematron schema in [Example 5.9](#). (`schematron-book-code/data/things-with-types.xml`)

```
<things>
  <thing name="thing 1" type="normal"/>
  <thing name="thing 2" type="special"/>
  <thing name="thing 3" type="special"/>
  <thing name="thing 4" type="vintage"/>
</things>
```

The SVRL validation report looks like [Example 5.11](#).

Example 5.11 – The SVRL generated when validating the document in [Example 5.10](#) with the schema from [Example 5.9](#).

```
<schematron-output xmlns="http://purl.oclc.org/dsdl/svrl">
  <active-pattern documents=".../things-with-types.xml"/>
  <fired-rule context="@type"/>
  <fired-rule context="@type"/>
  <fired-rule context="@type"/>
  <fired-rule context="@type"/>
  <failed-assert location="/Q{}things[1]/Q{}thing[4]/@Q{}type"
    test="(. eq 'normal') or (. eq 'special')">
    <text>The type must be normal or special</text>
  </failed-assert>
</schematron-output>
```

Back to the schema in [Example 5.9](#). Let's analyze in detail what's happening:

- The (only) rule in the (only) pattern has a `context="@type"` attribute. This means that this rule will trigger on every `type` attribute in the document.
- When this rule encounters a `type` attribute, it triggers the assertion and the `type` attribute becomes the context item. This means that you can reference it with a dot (`.`) in the test expression, as this example does.
- The `test` attribute on an `<assert>` or `<report>` element must contain a valid XPath boolean expression. This expression is evaluated and the result (`true` or `false`) is used by the Schematron processor:
 - An assertion will trigger when the value of the test is `false`.
 - A report will trigger when the test is `true`.
- The `test` attribute in the example contains the value:

```
(. eq 'normal') or (. eq 'special')
```

This checks whether the value of the context item is either `normal` or `special` and returns `true` if this is the case.

- Now look back to [Example 5.10](#). The first three `<thing>` elements have a `type` attribute that conforms to the rules. So the expression in the `test` attribute will evaluate to `true` and no message will be produced.

But the value of the `type` attribute on the fourth `<thing>` has an invalid type, `vintage`, which causes the test expression to evaluate to `false`. The Schematron processor uses the contents of the `<assert>` element as message text, resulting in the error message: **The type must be normal or special.**

5.2.3.1. Using reports instead of assertions

Could you do the same thing using reports instead of assertions? Yes, and [Example 5.12](#) shows how.

Example 5.12 – A variant of [Example 5.9](#), but now using `report` instead of `assert`. (`schematron-book-code/examples/basics/check-type-2.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="@type">
      <report test="not((. eq 'normal') or (. eq 'special'))">
        The type must be normal or special
      </report>
    </rule>
  </pattern>
</schema>
```

The expression in the `<assert>` element's test expression is almost identical to [Example 5.9](#). The only difference is that the test is negated using the `not()` function.

The validation result is almost the same. The only difference in the SVRL report is that it will tell you a report was detected instead of an assert (see [Example 5.13](#)).

Example 5.13 – The SVRL generated when validating the document in [Example 5.10](#) with the schema from [Example 5.12](#).

```
<schematron-output xmlns="http://purl.oclc.org/dsdl/svrl">
  <active-pattern documents=".../things-with-types.xml"/>
  <fired-rule context="@type"/>
  <fired-rule context="@type"/>
  <fired-rule context="@type"/>
  <fired-rule context="@type"/>
  <successful-report location="/Q{}things[1]/Q{}thing[4]/@Q{}type"
    test="not((. eq 'normal') or (. eq 'special'))">
    <text>The type must be normal or special</text>
```

```
</successful-report>
</schematron-output>
```

This might seem to be almost the same, but the meaning is different. In most situations assertions are used to catch errors, and reports are used to flag situations that need disclosure.

Following this line of thought, a report might look more like [Example 5.14](#).

Example 5.14 – Variant of [Example 5.12](#), using a `<report>` element as intended. (schematron-book-code/examples/basics/check-type-2b.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="@type">
      <report test="(. eq 'normal') or (. eq 'special')">Type OK!</report>
    </rule>
  </pattern>
</schema>
```

If you run this schema against [Example 5.10](#), the result will be three **Type OK!** messages, because there are three `<thing>` elements where the value of the `type` attribute is either `normal` or `special`. The resulting SVRL looks like [Example 5.15](#).

Example 5.15 – The SVRL generated when validating the document in [Example 5.10](#) with the schema from [Example 5.14](#).

```
<schematron-output xmlns="http://purl.oclc.org/dsdl/svrl">
  <active-pattern documents=".../things-with-types.xml"/>
  <fired-rule context="@type"/>
  <successful-report location="/Q{}things[1]/Q{}thing[1]/@Q{}type"
    test="(. eq 'normal') or (. eq 'special')">
    <text>Type OK!</text>
  </successful-report>
  <fired-rule context="@type"/>
  <successful-report location="/Q{}things[1]/Q{}thing[2]/@Q{}type"
    test="(. eq 'normal') or (. eq 'special')">
    <text>Type OK!</text>
  </successful-report>
  <fired-rule context="@type"/>
  <successful-report location="/Q{}things[1]/Q{}thing[3]/@Q{}type"
    test="(. eq 'normal') or (. eq 'special')">
    <text>Type OK!</text>
  </successful-report>
</schematron-output>
```

5.2.3.2. The message texts

The texts produced by the assertions and reports are completely up to you. Of course, it helps a lot if you try to write them from the user's perspective. Don't leave your users staring puzzled at messages they don't understand. Making messages clear and helpful is a worthy goal.

The Schematron standard contains two, in my opinion rather peculiar, statements about this:

- For assertions (standard section 5.4.2): “The natural-language assertion shall be a positive statement of a constraint.”
- For reports (standard section 5.4.12): “The natural-language assertion shall be a positive statement of a found pattern or a negative statement of a constraint.”

In other words: the Schematron standard prescribes how to formulate messages. A message like `Invalid value` is considered wrong and should be reformulated to something positive, for instance `The value must be between x and y`. The original designer of Schematron (Rick Jelliffe) says about this: “Schematron is a Schema language *because* it requires you to state positively what is supposed to be found in the document.” It's all about what assertions and reports actually *are*, what they're intended to be.

Of course, in practice it's entirely up to you how you formulate your messages. No Schematron processor will ever refuse a Schematron schema because the messages aren't stated in a positive way. Personally, I see this as a philosophical issue with limited practical value. Some examples in this book use positive message texts, some don't. Yes, messages should be clear and helpful, but how to formulate them is, in my opinion, not something a standard should be concerned with. You have to make up your own mind.

5.3. More meaningful messages: <value-of>

So far, the messages in the examples have been rather simple, consisting of a fixed piece of text. However, it would be nice, and it is often required, to have more meaningful messages that, for example, detail exactly what's wrong and identify the location of the error.

This section describes the most important, and most frequently used, message construct for this: the `<value-of>` element. There are other elements you can use to improve message quality, such as adding emphasis or defining the reading direction. I don't describe these other elements here, but you can find a description in [Section 8.1, “Messages with markup: `<emph>`, `<span>`, and `<dir>`”](#).

You can improve messages by using information from the document. For instance, a message that reads **The type must be normal or special,**
not vintage reports both the offending type value and the correct values. This allows you to search the document for the word “vintage” and repair the incorrect instances.

Schematron uses the `<value-of>` element to do this. This element inserts the result of an XPath expression in a message. The expression is in the (mandatory) `select` attribute:

```
<value-of select="..." />
```

The result of the expression in the `select` attribute, converted to a string, is inserted into your message. [Example 5.16](#) puts the `<value-of>` element to work.

Example 5.16 – Using a `<value-of>` element to enhance a message. **([schematron-book-code/examples/basics/check-type-3.sch](#))**

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="@type">
      <assert test="(. eq 'normal') or (. eq 'special')">
        The type must be normal or special, not <value-of select="..." />
      </assert>
    </rule>
  </pattern>
</schema>
```

The `select` attribute of the `<value-of>` element references the context item, just like the `test` attribute of the `<assert>` element. This context item, set by the encompassing `<rule>` element, and referenced with the dot (`.`) operator, is the same in both cases.

When run against the document in [Example 5.10](#), the error message becomes **The type must be normal or special, not vintage**. The generated SVRL is shown in [Example 5.17](#).

Example 5.17 – The SVRL generated when validating the document in [Example 5.10](#) with the schema from [Example 5.16](#).

```
<schematron-output xmlns="http://purl.oclc.org/dsdl/svrl">
  <active-pattern documents="../../../things-with-types.xml"/>
    <fired-rule context="@type"/>
    <fired-rule context="@type"/>
    <fired-rule context="@type"/>
    <fired-rule context="@type"/>
    <failed-assert location="/Q{}things[1]/Q{}thing[4]/@Q{}type"
      test="(. eq 'normal') or (. eq 'special')">
      <text>The type must be normal or special, not vintage</text>
    </failed-assert>
</schematron-output>
```

Let's make this message even more specific: The input document from [Example 5.10](#) also has a `name` attribute on every `<thing>` element:

```
<thing name="thing 4" type="vintage"/>
```

You can do this by including the value of the `name` attribute in the error message, for instance: **The type of thing 4 must be normal or special, not vintage**. However, the context item is the `type` element, and you need to reach the `name` attribute. To reach that attribute you need to work with the XML document tree:

- Climb up in the tree to the `type` attribute's parent, the `<thing>` element, using the parent double-dot operator (`..`).
- Descend from the `<thing>` parent element to its child `name` attribute: `@name`.
- Concatenate these two parts using the slash (`/`) operator: `../@code`

[Example 5.18](#) shows the result when you add this to the schema.

Example 5.18 – Adding the value of the `name` attribute to the message (schematron-book-code/examples/basics/check-type-4.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
```

```

<rule context="@type">
  <assert test="(. eq 'normal') or (. eq 'special')">
    The type of <value-of select="../@name"/> must be normal or special,
    not <value-of select="."/>
  </assert>
</rule>
</pattern>
</schema>

```

Where has the context item gone?

Looking at resulting the XPath expression `../@name`, you might wonder where the context item has gone. All this tree climbing and descending must start at the context item (`.`). Shouldn't the expression be `../@name` instead?

Actually, both expressions, `../@name` and `../@name`, are correct. Starting at the context item is implicit for XPath expressions that do not start with a slash. So you can write `../@name`, but the starting `./` is superfluous (and not using it has the priceless benefit of saving two keystrokes).

The code in [Example 5.18](#) is fine but a bit clumsy with all this climbing up and down in the tree to get to the value of an attribute. Another way of doing this is to make the rule fire on the parent `<thing>` element and from there test and, if necessary, report the values of the attributes. [Example 5.19](#) does this.

Example 5.19 – Using the `<thing>` element as target for the rule. (`schematron-book-code/examples/basics/check-type-5.sch`)

```

<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="thing">
      <assert test="(@type eq 'normal') or (@type eq 'special')">
        The type of <value-of select="@name"/> must be normal or special,
        not <value-of select="@type"/>
      </assert>
    </rule>
  </pattern>
</schema>

```

Both [Example 5.18](#) and [Example 5.19](#) result in the same messages.

5.4. Declaring and using variables: `<let>`

Adding a variable to your Schematron schema allows you to store some value and re-use it in rules, assertions, reports, and messages. This value

can be anything you can access using an XPath expression: a constant, something from the document that is validated, a computed result, etc.

For a first example, let's revisit [Example 5.19](#). Here the schema determines the value of the `type` attribute twice: Once on the `<assert>` element and once when creating the message (the second `<value-of>` element).

For a simple schema this kind of duplication of effort isn't a big deal, but it does violate the software engineering's DNNRY (Do Not Repeat Yourself) principle. It would be better to retrieve the value only once. This will become more important when schemas and XPath expressions get bigger, more complicated, and harder to maintain.

[Example 5.20](#) shows an enhanced version of [Example 5.19](#), using a variable as intermediate storage for the value of the `type` attribute.

Example 5.20 – Using a variable for a thing's type (schematron-book-code/examples/basics/check-type-6.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="thing">
      <let name="thing-type" value="@type"/>
      <assert test="($thing-type eq 'normal') or ($thing-type eq 'special')">
        The type of <value-of select="@name"/> must be normal or special,
        not <value-of select="$thing-type"/>
      </assert>
    </rule>
  </pattern>
</schema>
```

The `<let>` element creates a variable called `thing-type` and fills this with the `type` attribute. In both the `<assert>` and the second `<value-of>` element we refer to this variable with `$thing-type`.

5.4.1. Ground rules for variables

Declaring and using variables is, in most cases, simple and intuitive. You normally don't have to be aware of the exact rules. If you think you can do without them, feel free to skip this section and look at the example in [Section 5.4.2, “Variable usage example”](#) for inspiration. But if you get

stuck, or are an inquisitive XPath connoisseur, read on. The following assumes we're using an XSLT or XPath type query language binding.

- You declare a variable using the `<let>` element: `<let name="..." value="..." />`
- You can declare variables as children of the `<schema>`, `<pattern>`, and `<rule>` elements.
- You reference a variable using the prefix `$` (as in XSLT):

Example 5.21 – Declaring and using a variable

```
<let name="thing-type" value="@type"/>
<assert test="...">... <value-of select="$thing-type"/> ...</assert>
```

- The (required) `name` attribute of the `<let>` element holds the name of the variable. It must follow the rules of an XML element/attribute name (the technical term is a *qualified name*). For example: `name-of-thing`, `totalWeight`, `delivery_date`, `top3-listings`, etc.

Variables in a namespace

Being a qualified name also means you can use (declared) namespace prefixes for your variables, like `local:first-date`, `ns1:value`, etc. However, this capability is rarely used and of interest only when your schema becomes really complicated. There are no examples of variables-in-a-namespace in this book.

The namespace for a variable is no different from any other namespace, which means it must be declared before it can be used. See [Section 5.5, “Declaring namespaces”](#) for information about declaring namespaces.

-
- The `value` attribute of the `<let>` element holds the XPath expression that results in the value for the variable. For instance:

Example 5.22 – Some examples XPath expressions in the `<let>` element's `value` attribute

```
<let name="max-weight" value="50"/>
<let name="nr-of-things" value="count(/*/thing)"/>
<let name="name-of-thing" value="../@name"/>
```

Omitting the `value` attribute

The `value` attribute is optional, but the rules for what happens if you omit it are a bit weird. The Schematron standard says: “If no `value` attribute is specified, the value of the attribute is the element content of the `<let>` element.”

That sounds reasonable until you find out that the only things allowed inside a `<let>` element are elements that are not in the Schematron namespace.

So this is not allowed:

```
<let name="max-weight">50</let>
```

But this is:

```
<let name="max-weight"><weight xmlns="http://some/name/space">50</weight></let>
```

The data type of the resulting `max-weight` variable becomes `document-node()` (so an XML document of its own), with the children of the `<let>` element as the contents.

Using the `<let>` element without a `value` attribute is not useful for most schemas. I strongly advise against using it, unless you really know what you’re doing.

- The context item used when evaluating the XPath expression can be one of two things:
 - Outside of `<rule>` elements, it is the document node of the document being validated.
 - Inside `<rule>` elements, it is the node matched by the rule’s `context` attribute. This is the same context item used for assertions, reports, and messages. You can see an example in [Example 5.20](#).
- A variable must be declared before it’s used.
- The scope of a variable is the element (and its children) it’s declared in.
- You cannot re-declare a variable within the same element. The re-declaration of `count` in [Example 5.23](#) is illegal.

Example 5.23 – Example of invalid variable re-declaration

```
<rule context="thing">  
  <let name="count" select="6"/>  
  <let name="count" select="7"/>  
  ...  
</rule>
```

- However, you can re-declare a variable in a child element (see [Example 5.24](#)).

Example 5.24 – Example of validly re-declaring a variable in a child element

```
<pattern>
  <let name="count" select="6"/>
  <rule context="thing">
    <let name="count" select="7"/>
    ...
  </rule>
</pattern>
```

Shadowing

Re-declaring, as done in [Example 5.24](#), is also known as *shadowing*: You do not change a variable's value, you only shadow its previous value with a new one. The new value is in effect only inside the element it is declared in and has no effect outside the scope of that element.

I do not recommend shadowing variables. It obfuscates code, making it harder to follow logic and determine where a value comes from. If you want to create maintainable code, invent a new descriptive name for each variable.

- The last thing about variables we need to talk about is their data type. XSLT programmers might already have noticed that you cannot explicitly assign a type to a variable, as you can in XSLT with the `as` attribute. This means that the data type of a variable is determined solely by the result of the expression in the `select` attribute.

For example, look back to [Example 5.20](#). What is the data type of the `thing`-type variable here? Since the XPath expression refers to an attribute, it's `attribute()` (and not `xs:string` as you might expect).

[Example 5.25](#) is a variant of [Example 5.20](#) with additional message text that tells us about `$thing`-type.

Example 5.25 – Finding out about `$thing`-type's data type ([schematron-book-code/examples/basics/check-type-6b.sch](#))

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="thing">
```

```

    <let name="thing-type" value="@type"/>
    <assert test="($thing-type eq 'normal') or ($thing-type eq
'special')">
      The type of <value-of select="@name"/> must be normal or special,
      not <value-of select="$thing-type"/>.
      $thing-type is attribute:
      <value-of select="$thing-type instance of attribute()"/>.
      Local name of $thing-type:
      <value-of select="local-name($thing-type)"/>.
    </assert>
  </rule>
</pattern>
</schema>

```

The <assert> element's message has been extended with two observations about \$thing-type: Whether it's an attribute (\$thing-type instance of attribute()) and the attribute's name (local-name(\$thing-type)). And indeed, running this Schematron schema against the document in [Example 5.10](#) reports:

```

The type of thing 4 must be normal or special, not vintage.
$thing-type is attribute: true.
Local name of $thing-type: type.

```

There are sometimes situations where you need to be explicit about the data type of a variable. More about this in [Section A.6.2.1, “Explicit data typing and data type conversions”](#). That section also tells you how to force a variable to a specific data type, even though Schematron has no direct means for doing this.

5.4.2. Variable usage example

To provide you with an illustrative example of variable usage, let's return to the parcels document presented in [Example 4.2](#). The following example works with dates and durations (for background information see [Section A.6.6, “Working with dates, times and durations”](#)).

The first attempt to formulate a Schematron schema for this was in [Example 4.1](#). [Example 5.26](#) enhances this example using variables.

Example 5.26 – Example of using variables in a Schematron schema (schematron-book-code/examples/basics/check-parcels.sch)

```

<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

```

```

<!-- 1 - Define the number of days a parcel's date must be before the
       delivery date and turn this into a duration: -->
<let name="delivery-day-limit" value="10"/>
<let name="delivery-day-limit-as-duration"
    value="xs:dayTimeDuration('P' || $delivery-day-limit || 'D')"/>

<!-- 2 - Grab important values from the root element and make sure they
       have the right data type: -->
<let name="max-weight" value="xs:double(//*[@max-weight])"/>
<let name="delivery-date" value="xs:date(//*[@delivery-date])"/>

<pattern>
  <rule context="/*">
    <!-- 3 - Compute the total parcels weight, store this in a variable
           and use this in the assert's test and in the message: -->
    <let name="total-parcels-weight" value="sum(parcel/@weight)"/>
    <assert test="$total-parcels-weight le $max-weight">
      The total weight (<value-of select="$total-parcels-weight"/>) is
      more than the allowed maximum (<value-of select="$max-weight"/>)
    </assert>
  </rule>
</pattern>

<pattern>
  <rule context="parcel">
    <!-- 4 - Get the parcel's date and make sure it has the right
           data type: -->
    <let name="parcel-date" value="xs:date(@date)"/>
    <assert
      test="($delivery-date - $parcel-date) gt
            $delivery-day-limit-as-duration">
      The parcel's date (<value-of select="$parcel-date"/>) must be
      more than <value-of select="$delivery-day-limit"/> days
      before the delivery date (<value-of select="$delivery-date"/>)
    </assert>
  </rule>
</pattern>

</schema>

```

Let's look at the four numbered sections in [Example 5.26](#) in more detail:

1. One of the rules we set for our parcels document was that a parcel's date must be at least 10 days before the delivery date.

Variables for magic values

From a maintenance point of view it's not a good idea to use hard-coded "magic" values (like 10) in your code.^[15] This is especially true when the value is used multiple times. When the value changes, and it will, it's easy to forget to change the value everywhere it's used. Even if you use such a value only once, it's still better to define it as a variable in some central location (like here, at the top of the document) with a

name that tells you what the value represents. You can then easily find and update the value when it changes.

[Example 5.26](#) defines a variable `delivery-day-limit` with value 10 and a second variable `delivery-day-limit-as-duration` of type `xs:dayTimeDuration`. The value of this second variable is constructed by first converting the value of `delivery-day-limit` into a valid duration string (P10D) and then converting it into a duration using the type name `xs:dayTimeDuration()` as a function.

2. The input document's root element holds two important values in attributes. We need these values in our assertions later on. It's therefore a good idea to grab these values only once and put them in variables at a global level. While we're doing that we also make sure they get the right data type (`xs:double` for `max-weight` and `xs:date` for `delivery-date`).
3. The first pattern checks the total maximum weight. It computes this and stores it in the variable (`total-parcels-weight`) then uses it in the following assertion. Notice that the message includes the values of some of our variables to make it more clear and informative.
4. The second pattern reads the date value from the context `<parcel>` element into a variable (`parcel-date`) and uses this in its assertion. Again we use variables to make the message clearer.

Using variables in this example is not only better from a software engineering point of view, but probably also for efficiency. In particular, the computations required to calculate the global variables defined in 1 and 2, are done just once. For this simple example it doesn't matter much, but with large documents or complex schemas, it could make a difference.

Let's see how this works. The document in [Example 5.27](#) is deliberately invalid.

Example 5.27 – Invalid parcels document to illustrate the workings of the schema in [Example 5.26](#) (`schematron-book-code/data/parcels-invalid-2.xml`)

```
<parcels max-weight="100" delivery-date="2021-10-10">
  <parcel weight="50" date="2021-09-29">
    <contents>A large quantity of mouth masks</contents>
  </parcel>
```

```
<parcel weight="55" date="2021-10-05">  
  <contents>Toys for children</contents>  
</parcel>  
</parcels>
```

If you validate [Example 5.27](#) using the schema in [Example 5.26](#), you get the following errors:

```
The total weight (105) is more than the allowed maximum (100)  
The parcel's date (2021-10-05) must be more than 10 days before the delivery date  
(2021-10-10)
```

5.5. Declaring namespaces

There is a good chance that you will encounter namespaces during your Schematron adventures. For instance, you will see them when you need to validate documents that use an international standard like XHTML (web pages), DocBook (text), QTI (questions and tests), or HL7 (healthcare messages). Also, company- and product-specific XML documents often use namespaces.

This is a topic that often causes confusion. If you don't feel comfortable with this subject, please read [Appendix B](#) first.

In a Schematron schema, namespaces and their prefixes must be declared before you use them. You declare a namespace using the `<ns uri="..." prefix="..." />` element. All namespace declarations must be direct children of the root `<schema>` element. For instance, [Example 5.28](#) declares the XHTML namespace and assigns it the prefix `xh`.

Example 5.28 – Declaring the XHTML namespace and assigning it the prefix `xh`

```
<ns uri="http://www.w3.org/1999/xhtml" prefix="xh" />
```

After this you can refer to the XHTML namespace with the prefix `xh` (see [Example 5.29](#))

Example 5.29 – Declaring the XHTML namespace with prefix `xh` and using it in a rule

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <ns uri="http://www.w3.org/1999/xhtml" prefix="xh"/>

  <pattern>
    <rule context="xh:title">
      <assert test="string-length(.) le 35">Title too long!</assert>
    </rule>
  </pattern>

</schema>
```

Not `xmlns`

If you've used namespaces before, you're probably used to declaring them with `xmlns` "attributes" (strictly speaking they're not attributes, but let's not worry about that now). So you would expect Schematron to define the XHTML namespace like this:

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3"
  xmlns:xh="http://www.w3.org/1999/xhtml">
  ...
</schema>
```

Although it's perfectly all right and valid to write this, you will not be able to use the `xh` prefix in Schematron's XPath expressions as you would like to. The Schematron processor will report that you haven't declared the `xh` prefix, a rather frustrating situation because in the "normal" XML sense, that's exactly what you did!

As a final note: having to declare a namespace (prefix) using the `<ns>` element has the consequence that Schematron has no notion of a default namespace. This means that when you write an XPath expression in Schematron that uses an element or attribute that is in a namespace, you must use the namespace prefix. An example of how to work around this can be found in [Section 9.2, "Handling a default namespace"](#).

[\[12\] Section 1.3, "Using and finding the code examples"](#) describes how to obtain the example code.

[\[13\]](#) Although this is unlikely, you might encounter a different query language with a proprietary Schematron processor.

[\[14\]](#) There are several algorithms for traversing a tree structure, the most common being depth-first and breadth-first. This Wikipedia article contains

a good description of tree traversal:

https://en.wikipedia.org/wiki/Tree_traversal.

[15] A magic value is a hard-coded value (like 10 in [Example 5.26](#)) that gives you no information about what it represents.

Chapter 6. Advanced Schematron

[Chapter 5](#) dealt with the basics of Schematron: what you must know to start writing Schematron schemas. This chapter goes a step further and dives into some of the more advanced features of Schematron, including:

- **Diagnostics:** allow you maintain a set of reusable messages.
- **Schematron phases:** allow you to selectively enable/disable patterns.
- **Abstract rules:** allow you to maintain a set of reusable rules.
- **Abstract patterns:** allow you to create a pattern that can be reused.
- **Includes:** allow you to include a schema or fragment in another schema.

You can live without these features, but as your schemas get more complicated, it's good to know they're here. So even if you're just starting to use Schematron, I would advise you to browse this chapter. When the need arises, you'll know where to look.

6.1. Providing multiple messages: <diagnostics>

[Section 5.2.3, “Assert and Report processing”](#) describes how to define a message in an <assert> or <report> element. For instance:

```
<assert test="...">Some message...</assert>
```

Besides defining messages directly in <assert> and <report> elements, Schematron also provides the ability to centralize message definitions. This is done using the <diagnostics> element, which allows you to define reusable diagnostic messages. This capability is also helpful in other use cases, which I'll describe later.

To get a taste, [Example 6.1](#) is a simple example that rewrites [Example 5.19](#).

Example 6.1 – Using the <diagnostics> element to define a message. (schematron-book-code/examples/advanced/diagnostics-1.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="thing">
      <!-- 1 - Refer to a message in a <diagnostics> section with the
```

```

        diagnostics attribute. -->
        <assert test="(@type eq 'normal') or (@type eq 'special')"
          diagnostics="message-1"/>
      </rule>
</pattern>

<!-- 2 - A <diagnostics> section contains centralized definitions
of messages.-->
<diagnostics>
  <!-- 3 - A <diagnostic> element contains a single message. It is
    referenced by id. -->
  <diagnostic id="message-1">
    Invalid type on <value-of select="@name"/>: <value-of select="@type"/>.
  </diagnostic>
</diagnostics>

</schema>

```

1. The <assert> element no longer contains a message. Instead it references a diagnostic message using the identifier of the message (message-1) in the diagnostics attribute.
2. The <diagnostics> element is a container for diagnostic messages.
3. A <diagnostic> element defines a diagnostic message. It has a mandatory id attribute, containing an identifier for the message.

For this simple example, this capability doesn't add much value, but what if you want to issue the same message for other elements as well? For example, suppose the XML contains information not only about things, but also about alien artifacts. You can now use the already defined diagnostic to issue the same message when an artifact's type is wrong, as shown in [Example 6.2](#).

Example 6.2 – Referencing the same diagnostic message twice.

```

<pattern>
  <rule context="thing">
    <assert test="(@type eq 'normal') or (@type eq 'special')"
      diagnostics="message-1"/>
  </rule>
</pattern>
<pattern>
  <rule context="artifact">
    <assert test="(@type eq 'zork') or (@type eq 'martian')" diagnostics="message-
1"/>
  </rule>
</pattern>

```

Reusing messages makes your schemas more maintainable as they get larger or more complicated. Here are three main use cases:

- As seen in [Example 6.2](#), you can define *reusable messages*. The benefits of this are clear. Grammar corrections or reformulations need to be done only once. The same is true for the expressions in `<value-of>` elements, which can at times become quite complicated.
- You can define *multiple messages* for an assertion or report. More about this in the next section.
- You can *localize messages* and provide them in different languages. See [Section 6.1.2, “Localization of messages”](#) for more information.

6.1.1. Multiple messages

There are cases where you may need to attach more than one message to a Schematron assertion or report. For example, you might have a generic error message (**The type is wrong**) and a specialized explanation (**Types for ... must be ...**).

[Example 6.3](#) enhances [Example 6.1](#) to do this.

Example 6.3 – Defining multiple messages for an assertion or report. ([schematron-book-code/examples/advanced/diagnostics-2.sch](#))

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <pattern>
    <rule context="thing">
      <assert test="(@type eq 'normal') or (@type eq 'special')">
        diagnostics="message-1 thing-explanation"/>
      </rule>
    </pattern>
    <pattern>
      <rule context="artifact">
        <assert test="(@type eq 'zork') or (@type eq 'martian')">
          diagnostics="message-1 artifact-explanation"/>
        </rule>
      </pattern>

    <diagnostics>
      <diagnostic id="message-1">
        Invalid type on <value-of select="@name"/>: <value-of select="@type"/>.
      </diagnostic>
      <diagnostic id="thing-explanation">
        Allowed types for a thing: normal, special.
      </diagnostic>
      <diagnostic id="artifact-explanation">
        Allowed types for an artifact: zork, martian.
      </diagnostic>
    </diagnostics>
  </schema>
```

</schema>

The `diagnostics` attribute of each of the `<assert>` elements now contains two whitespace-separated identifiers. Both messages will be reported when the assertion fires.

How this shows up in your environment is implementation-defined. For instance, when you use [Example 6.3](#) to validate [Example 5.10](#), oXygen concatenates the messages and comes up with:

Invalid type on thing 4: vintage. Allowed types for a thing: normal, special.

Repeating yourself

Readers with a software engineering background might shudder looking at [Example 6.3](#). The horror! The list of possible types, for both the things and artifacts, is defined twice: once in the XPath expression of the assertions and once as text, in the explanatory messages. I did this to avoid over complicating the example. However, you can use XPath sequences to improve the code. [Example 6.4](#) does this, but I'll leave it to you as an exercise to work out the details.

Example 6.4 – An improved version of [Example 6.3](#). (`schematron-book-code/examples/advanced/diagnostics-2-improved.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <let name="thing-types" value="('normal', 'special')"/>
  <let name="artifact-types" value="('zork', 'martian')"/>

  <pattern>
    <rule context="thing">
      <assert test="@type = $thing-types"
        diagnostics="message-1 thing-explanation"/>
    </rule>
  </pattern>
  <pattern>
    <rule context="artifact">
      <assert test="@type = $artifact-types"
        diagnostics="message-1 artifact-explanation"/>
    </rule>
  </pattern>

  <diagnostics>
    <diagnostic id="message-1">
      Invalid type on <value-of select="@name"/>:
      <value-of select="@type"/>.
    </diagnostic>
    <diagnostic id="thing-explanation">
      Allowed types for a thing:
      <value-of select="string-join($thing-types, ', ')/>.
    </diagnostic>
    <diagnostic id="artifact-explanation">
      Allowed types for an artifact:
      <value-of select="string-join($artifact-types, ', ')/>.
    </diagnostic>
  </diagnostics>

</schema>
```

Besides using multiple references to diagnostic messages, as we've done so far, it's also possible to define multiple messages by combining a direct message in the assertion/report with one or more references to diagnostic messages (see [Example 6.5](#)).

Example 6.5 – Combining a direct assertion/report message with a diagnostic message.

```
<pattern>
  <rule context="thing">
    <assert test="(@type eq 'normal') or (@type eq 'special')">
      diagnostics="thing-explanation">
        Invalid type on <value-of select="@name"/>: <value-of select="@type"/>
      </assert>
    </rule>
  </pattern>

...

<diagnostic id="thing-explanation">
  Allowed types for a thing: normal, special.
</diagnostic>
```

As with the previous example, how this shows up in your environment is implementation-defined. For instance, when you use [Example 6.5](#) to validate [Example 5.10](#), SchXslt processes the message referenced in the `diagnostics` attribute first, followed by the message in the `<assert>` element, resulting in (note that SchXslt preserves spaces in the messages, including newline characters):

```
Allowed types for a thing: normal, special.

Invalid type on thing 4: vintage.
```

6.1.2. Localization of messages

Diagnostics are also a useful tool for localization; they allow you to provide messages in different languages. [Example 6.6](#) enhances [Example 6.1](#) so it can report problems in Dutch and English.

Example 6.6 – A Schematron schema with localized messages. (`schematron-book-code/examples/advanced/diagnostics-3.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <pattern>
    <rule context="thing">
```

```

        <!-- 1 - Refer to the diagnostics in all languages: -->
        <assert test="(@type eq 'normal') or (@type eq 'special')"
            diagnostics="type-message-en type-message-nl"/>
    </rule>
</pattern>

<diagnostics>
    <!-- 2 - Provide the diagnostics in all languages and set the language
        using the xml:lang attribute: -->
    <diagnostic id="type-message-en" xml:lang="en">
        Invalid type on <value-of select="@name"/>: <value-of select="@type"/>.
    </diagnostic>
    <diagnostic id="type-message-nl" xml:lang="nl">
        Ongeldig type voor <value-of select="@name"/>: <value-of select="@type"/>.
    </diagnostic>
</diagnostics>

</schema>

```

1. The <assert> element, in its diagnostics attribute, references the diagnostic messages for both languages (using a whitespace separated list of diagnostic message identifiers).
2. The diagnostic messages are defined for both languages. Each has a unique identifier and defines its language using the xml:lang attribute.

Depending on the language setting of your Schematron processor, the message will read either **Invalid type on ...** Or **Ongeldig type voor ...**

In practice (see the note below the next example), it's also possible to organize your diagnostic messages in multiple <diagnostics> elements and set the language on this level. This is useful if you have multiple messages and want to group them by language (see [Example 6.7](#)).

Example 6.7 – Organizing the diagnostic messages per language. ([schematron-book-code/examples/advanced/diagnostics-3b.sch](#))

```

<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

    <pattern>
        <rule context="thing">
            <assert test="@type = ('normal', 'special', 'deprecated')"
                diagnostics="type-message-en type-message-nl"/>
        </rule>
    </pattern>

    <diagnostics xml:lang="en">
        <diagnostic id="type-message-en" >
            Invalid type on <value-of select="@name"/>: <value-of select="@type"/>.
        </diagnostic>
        <!-- Other diagnostic messages in English... -->
    </diagnostics>

```

```
</diagnostics>

<diagnostics xml:lang="nl">
  <diagnostic id="type-message-nl" >
    Ongeldig type voor <value-of select="@name"/>: <value-of select="@type"/>.
  </diagnostic>
  <!-- Other diagnostic messages in Dutch... -->
</diagnostics>

</schema>
```

Multiplicity of the `<diagnostics>` element

The official Schematron schema (the one that comes with the standard) only allows for a single `<diagnostics>` element to appear in your Schematron schema. An `xml:lang` attribute on it is not allowed. This makes [Example 6.7](#) officially invalid.

However, both oXygen and SchXslt accept multiple `<diagnostics>` elements with an optional `xml:lang` attribute. I'm not a big fan of going beyond what a standard prescribes, but [Example 6.7](#) seems like such a natural way of doing these kinds of things that I consider this one of these small flaws in the standard. Whether to use this unofficial feature is, of course, up to you.

Finally, how does the Schematron processor know what language to use? This is implementation-defined: it depends on the Schematron processor whether and how you can set this.

For example, oXygen has four ways you can set the reporting language:

- Leave it the same as the oXygen application's language.
- Use the value of the `xml:lang` attribute on the `<schema>` root attribute.
- Set it in oXygen's application preferences.
- Ignore any language setting and output all referenced messages.

6.2. Selecting what patterns are active: `<phase>`

Schematron phases can be used to selectively enable/disable patterns in a schema. Maybe some patterns are CPU intensive and you don't always want to apply them. Or maybe some of them are applicable to final documents only but must be kept together with the patterns for documents that are works in progress. Another typical use case is reference checking: if your document contains identifiers that reference something else, you might want to turn off checking references until everything is complete. To support these kinds of situation, Schematron has phases.

Here is an example. Suppose you are writing an article about Schematron in some fictitious markup language. [Example 6.8](#) contains an example of a valid document in that language.

Example 6.8 – A valid example of a document about Schematron.
([schematron-book-code/data/simple-document.xml](#))

```
<document>

  <title>The wonderful world of Schematron</title>

  <para>Welcome to the wonderful world of Schematron
    <footnote-reference idref="more-schematron-info"/>. ...</para>
  <para>It is a useful tool for ...</para>
  <para>Here are some examples ...</para>

  <!-- Footnotes: -->
  <footnotes>
    <footnote id="more-schematron-info">
      More information to be found at ...
    </footnote>
  </footnotes>

</document>
```

Our fictitious XML markup language has the following rules:

- Titles must be no longer than 45 characters.
- There must be at least 3 paragraphs of text.
- Footnotes are referenced by identifiers.

[Example 6.9](#) contains a Schematron schema to check these rules.

Example 6.9 – A Schematron schema for checking the validity of
[Example 6.8](#). ([schematron-book-code/examples/advanced/phases-1a.sch](#))

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <pattern>
    <rule context="title">
      <assert test="string-length(.) le 45">The title is longer than 45
characters</assert>
    </rule>
  </pattern>

  <pattern>
    <rule context="/*">
      <assert test="count(//para) ge 3">There must be at least 3
paragraphs</assert>
    </rule>

  </pattern>
```

```

</pattern>

<pattern>
  <rule context="footnote-reference">
    <let name="footnote-id" value="@idref"/>
    <assert test="exists(//footnote[@id eq $footnote-id])">
      Footnote id <value-of select="$footnote-id"/> not found</assert>
    </rule>
  </pattern>
</schema>

```

The document in [Example 6.8](#) survives scrutiny by this schema with flying colors. But a full check is less useful if you write this document in phases. For example:

- First you create the document structure. Therefore, the only things you want to check are warnings for overly long titles.
- After that you write the text but not the footnotes, yet.
- Last, but not least, you add the footnotes.

Phases allow a Schematron schema to apply only the tests you need at each point in the process. [Example 6.10](#) shows how.

Example 6.10 – A Schematron schema for checking the validity of [Example 6.8](#) using phases. (schematron-book-code/examples/advanced/phases-1b.sch)

```

<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <!-- 1 - Define a phase that only checks the title length -->
  <phase id="titles-only">
    <active pattern="check-title-length"/>
  </phase>

  <!-- 2 - Define a phase that checks the title length and the number of paragraphs -->
  <phase id="titles-and-paragraphs">
    <active pattern="check-title-length"/>
    <active pattern="check-para-count"/>
  </phase>

  <!-- 3 - The actual patterns: -->
  <pattern id="check-title-length">
    <rule context="title">
      <assert test="string-length(.) le 45">The title is longer than 45
characters</assert>
    </rule>
  </pattern>

  <pattern id="check-para-count">

```

```

    <rule context="/*>
      <assert test="count(//para) ge 3">There must be at least 3
paragraphs</assert>
    </rule>
  </pattern>

  <pattern id="check-footnote-references">
    <rule context="footnote-reference">
      <let name="footnote-id" value="@idref"/>
      <assert test="exists(//footnote[@id eq $footnote-id])">
        Footnote id <value-of select="$footnote-id"/> not found</assert>
    </rule>
  </pattern>
</schema>

```

Each <pattern> element in [Example 6.10](#) has an id attribute.

1. The first <phase> element defines a phase with identifier titles-only. The child element <active> defines that only the pattern for checking the title length is active, referencing it by its identifier, check-title-length.
2. The second <phase> element defines another phase with the identifier titles-and-paragraphs. It has two <active> children that reference the patterns that must be active: the title length and the paragraph count checks.
3. There is no phase defined for all patterns in the schema. You could have done this but it's unnecessary. By default, a Schematron validation uses all available patterns unless you override this by specifying a phase.

Let's use this schema on a document in the making (see [Example 6.11](#)).

Example 6.11 – A document in the making about Schematron. (schematron-book-code/data/simple-document-invalid.xml)

```

<document>

  <title>The wonderful world of Schematron and much more information that you
    want to know but were afraid to ask</title>

  <para>Welcome to the wonderful world of Schematron
    <footnote-reference idref="more-schematron-info"/>. ...</para>
  ...

  <!-- Footnotes: -->
  <footnotes>
    <!-- To be done -->
  </footnotes>

</document>

```

The title of [Example 6.11](#) is too long, there's only one paragraph, and the footnote is missing. Let's validate this document using the phases introduced in [Example 6.10](#).

- At first, we're only interested in overly long titles, so we run the schema activating the `title-only` phase. This message is issued:

```
The title is longer than 45 characters
```

- We continue by activating the `titles-and-paragraphs` phase to check the number of paragraphs. The previous message is issued plus:

```
There must be at least 3 paragraphs
```

- Finally, we want to do a full check and run the schema without activating a particular phase. The two previous messages are issued plus:

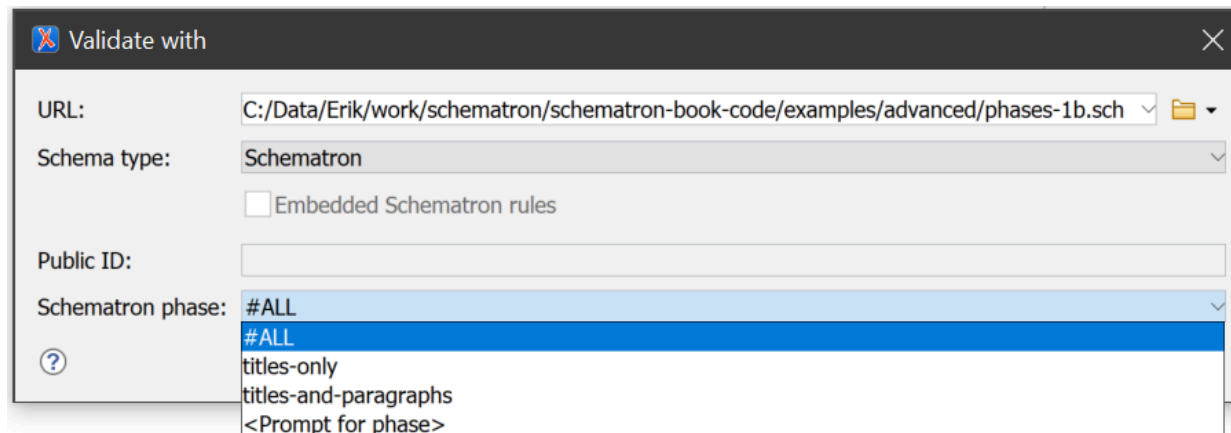
```
Footnote id more-schematron-info not found
```

Of course, for only three messages this is not a big deal, but in a real-life situation, not having to wade through lots of not-yet-relevant messages can be a real time (and annoyance) saver.

The remaining question is: how to activate a phase? This is, once again, implementation-defined: it depends on your Schematron processor and environment.

For instance, oXygen gives you the option of choosing a phase directly or getting a prompt to choose one (see [Figure 6.1](#)). SchXslt has a command-line option (`-p phase`).

Figure 6.1. Choosing a phase using oXygen



Finally, you can specify a default phase for Schematron validations by adding the `defaultPhase` attribute to the `<schema>` element. The processor will use this phase if no other phase has been specified. For example, to make the `titles-and-paragraphs` phase the default in [Example 6.10](#), change the root element as shown in [Example 6.12](#).

Example 6.12 – Specifying a default phase on the Schematron schema’s root element

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3"
  defaultPhase="titles-and-paragraphs">
  ...
</schema>
```

6.3. Reusing rules: abstract rules

“Do not repeat yourself” is a well-known adage from software engineering. We’ve already seen how Schematron diagnostics help you avoid repeating messages. Abstract rules are a mechanism to help you avoid repeating rules. A similar mechanism also exists for patterns (see [Section 6.4, “Reusing patterns: Abstract patterns”](#)).

Of the three mechanisms for reuse (diagnostics, abstract rules, and abstract patterns), abstract rules is probably used the least. You should need it only rarely. It’s much more common to use alternative mechanisms (see [Section 6.3.2, “Alternative to abstract rules”](#) for more).

Let's start our look at the abstract rule mechanism with a fresh example. Suppose you have some kind of loading manifest, containing a list of crates:

Example 6.13 – A loading list with crates ([schematron-book-code/data/crates.xml](#))

```
<crate-contents>
  <crate width="0.5" height="0.5" length="0.6" material="wood">books</crate>
  <crate width="0.5" height="0.8" length="1.6" material="wood">fishing rods</crate>
  <crate width="1.5" height="0.8" length="1.3" material="metal">dumbbells</crate>
</crate-contents>
```

Length, width, and height information is expressed in meters; crates may not be larger than one cubic meter. In addition, wooden crates are only allowed to contain books.

The Schematron schema in [Example 6.14](#) checks this.

Example 6.14 – A Schematron schema for checking the loading list rules ([schematron-book-code/examples/advanced/abstract-rules-1.sch](#))

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <pattern>

    <rule context="crate[@material eq 'wood']">
      <assert test="(@length * @height * @width) le 1">
        The crate with <value-of select="."/> is too big
      </assert>
      <assert test="." eq 'books'>
        A wooden crate may only contain books. This one contains
        <value-of select="."/>
      </assert>
    </rule>

    <rule context="crate[@material eq 'metal']">
      <assert test="(@length * @height * @width) le 1">
        The crate with <value-of select="."/> is too big
      </assert>
    </rule>

  </pattern>

</schema>
```

Validating [Example 6.13](#) against the schema in [Example 6.14](#) results in the following two messages:

A wooden crate may only contain books. This one contains fishing rods
The crate with dumbbells is too big

The schema in [Example 6.14](#) contains a repeated assertion for the volume check. The assertion that checks the maximum volume appears in two rules: once when the crate material is wood and again when the crate material is metal.

[Example 6.15](#) gets rid of this duplication using abstract rules. Note that in this explanation, I use the term *concrete* rule for the kinds of rules used so far in this book.

Example 6.15 – A Schematron schema for checking the loading list rules, using an abstract rule for the volume check (`schematron-book-code/examples/advanced/abstract-rules-2.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <pattern>

    <!-- 1 - Define the volume check rule as abstract -->
    <rule abstract="true" id="volume-check">
      <assert test="(@length * @height * @width) le 1">
        The crate with <value-of select="."/> is too big
      </assert>
    </rule>

    <rule context="crate[@material eq 'wood']">
      <!-- 2 - Use the abstract rule for the volume check for wooden crates -->
      <extends rule="volume-check"/>
      <assert test=". eq 'books'">
        A wooden crate may only contain books. This one contains
        <value-of select="."/>
      </assert>
    </rule>

    <rule context="crate[@material eq 'metal']">
      <!-- 3 - Use the abstract rule for the volume check for metal crates -->
      <extends rule="volume-check"/>
    </rule>

  </pattern>

</schema>
```

1. The rule for the volume check is now defined as an abstract rule. The following applies:

- An abstract rule must have an `abstract="true"` attribute.
- An abstract rule must have an identifier using the `id` attribute so you can reference it.

- An abstract rule must not have a `context` attribute. Its context is taken from the rule that uses it (extends it).
2. The concrete rule for wooden crates needs the volume check. Therefore, it references the abstract rule using `<extends rule="volume-check"/>`.
 3. The same goes for the metal crates.

A concrete rule can extend multiple abstract rules (contain multiple `<extends>` child elements). An abstract rule can itself extend other abstract rules (contain `<extends>` child elements).

Despite the fancy object-oriented jargon “extends,” there’s nothing object oriented about this capability. Abstract rules are implemented as macros. The `<extends>` element is simply replaced by all the child elements of the abstract rule. So, a variable in an abstract rule will be evaluated in the context of the rule that references it. This can be convenient, for example if you want to give a variable in an abstract rule a different value depending on where it is used. However, this can lead to naming conflicts if you are not careful.

6.3.1. Abstract rules in external documents

That abstract rules are actually macros is even more apparent in its other form: including rules from another document. To use this capability, replace the `rule` attribute on the `<extends>` element with an `href` attribute that points to the file that contains the abstract rule (see [Example 6.16](#)). The `<extends>` element has either a `rule` or an `href` attribute, not both.

Example 6.16 – A Schematron schema for checking the loading list rules, using a separate rule document to avoid rule repetition (schematron-book-code/examples/advanced/abstract-rules-3.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <pattern>

    <rule context="crate[@material eq 'wood']">
      <extends href="volume-check.sch"/>
      <assert test="." eq 'books'">
        A wooden crate may only contain books. This one contains
        <value-of select="."/>
      </assert>
    </rule>

    <rule context="crate[@material eq 'metal']">
      <extends href="volume-check.sch"/>
```

```
</rule>
</pattern>
</schema>
```

The referenced `volume-check.sch` document looks like [Example 6.17](#).

Example 6.17 – The volume check rule document, as referenced by [Example 6.16](#) (`schematron-book-code/examples/advanced/volume-check.sch`)

```
<rule xmlns="http://purl.oclc.org/dsdl/schematron">
  <assert test="(@length * @height * @width) le 1">
    The crate with <value-of select="."/> is too big
  </assert>
</rule>
```

Rules in an external document referenced by an `<extends href="...">` element must not be defined as abstract. However, the official Schematron specification does not state this clearly, and in practice it does not seem to matter. For example, SchXslt works the same way whether you define the external rule as abstract or not.

6.3.2. Alternative to abstract rules

You may have noticed that there is already a perfect mechanism for solving the repeated rules problem: instead of abstract rules you can in most, if not all, cases use separate patterns to avoid rule repetition.

In [Example 6.18](#), the first pattern fires for every crate and tests for the maximum volume. The second pattern fires only for wooden crates, testing for the contents.

The result is the same, but the Schematron schema in [Example 6.18](#) is shorter and, arguably, easier to understand. This may explain why the abstract rules feature is used less often than the other reuse features.

Example 6.18 – A Schematron schema for checking the loading list rules, using separate patterns to avoid rule repetition (`schematron-book-code/examples/advanced/abstract-rules-1b.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
```

```

    <rule context="crate">
      <assert test="(@length * @height * @width) le 1">
        The crate with <value-of select="."/> is too big
      </assert>
    </rule>
  </pattern>

  <pattern>
    <rule context="crate[@material eq 'wood']">
      <assert test="." eq 'books'">
        A wooden crate may only contain books. This one contains
        <value-of select="."/>
      </assert>
    </rule>
  </pattern>
</schema>

```

6.4. Reusing patterns: Abstract patterns

Abstract patterns are useful when your document contains structures that look alike and must abide by the same rules, but differ in element names or other details.

As an example (adapted from the one in the standard and shown in [Example 6.19](#)), let's take a document with two different, table-like, structures.

Example 6.19 – An XML document with two table-like structures (schematron-book-code/data/tables.xml)

```

<tables>

  <!-- HTML table: -->
  <table>
    <tr>
      <td>Yes!</td>
      <td>No!</td>
    </tr>
  </table>

  <!-- Calendar table-like structure: -->
  <year>
    <week>
      <day>Monday</day>
      <day>Tuesday</day>
      <!-- Etc. -->
    </week>
  </year>

</tables>

```

- The first structure is a straight HTML table contained in a <table> element. A table must have rows (<tr>), and a row must have cells (<td>).
- The second one is a calendar structure, contained in a <year> element. A year must have weeks (<week> elements), and a week must have days (<day> elements).

We could, with what we know by now, easily create a Schematron schema that checks this. However, given the similarities between the table and calendar structures, it would contain several rule/assertion repetitions.

[Example 6.20](#) avoids the repetition using abstract patterns.

Example 6.20 – A Schematron schema that uses abstract patterns to test [Example 6.19](#) (schematron-book-code/examples/advanced/abstract-patterns-1a.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <!-- 1 - Define the abstract pattern -->
  <pattern abstract="true" id="table-pattern">
    <rule context="$table">
      <assert test="$row">
        The element <value-of select="local-name()"/> is a table structure.
        Tables must contain the correct row elements.
      </assert>
    </rule>
    <rule context="$table/$row">
      <assert test="$entry">
        The element <value-of select="local-name()"/> is a table row.
        Rows must contain the correct cell elements.
      </assert>
    </rule>
  </pattern>

  <!-- 2 - Pattern for HTML tables: -->
  <pattern is-a="table-pattern" >
    <param name="table" value="table"/>
    <param name="row" value="tr"/>
    <param name="entry" value="td"/>
  </pattern>

  <!-- 3 - Pattern for a calendar table-like structure: -->
  <pattern is-a="table-pattern" >
    <param name="table" value="year"/>
    <param name="row" value="week"/>
    <param name="entry" value="day"/>
  </pattern>

</schema>
```

1. First, define an abstract pattern. This looks like a normal pattern, but the `abstract="true"` attribute defines it as abstract. The `id` attribute assigns it an identifier.

This pattern contains references to what look like undefined variables: `$table`, `$row`, and `$entry`. These are not variable references; they are pattern-parameters that must be passed in when the pattern is instantiated.

2. To instantiate an abstract pattern, use the `is-a` attribute on a `<pattern>` element, with the identifier of the abstract pattern as its value. Instead of `<rule>` elements you need to supply `<param>` elements to define the values of the pattern-parameters you declared in the abstract pattern. In this example, the parameters are the names of the table, row, and entry elements.
3. You can instantiate the abstract pattern as often as needed, using different element names as parameters.

What happens under the hood is that every abstract pattern instantiation is turned into a concrete pattern, substituting the pattern-parameters. So [Example 6.20](#) becomes [Example 6.21](#).

Example 6.21 – [Example 6.20](#) with all abstract patterns expanded ([schematron-book-code/examples/advanced/abstract-patterns-1b.sch](#))

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <pattern>
    <rule context="table">
      <assert test="tr">
        The element <value-of select="local-name()"/> is a table structure.
        Tables must contain the correct row elements.
      </assert>
    </rule>
    <rule context="table/tr">
      <assert test="td">
        The element <value-of select="local-name()"/> is a table row.
        Rows must contain the correct cell elements.
      </assert>
    </rule>
  </pattern>

  <pattern>
    <rule context="year">
      <assert test="week">
        The element <value-of select="local-name()"/> is a table structure.
```

```

        Tables must contain the correct row elements.
    </assert>
</rule>
<rule context="year/week">
    <assert test="day">
        The element <value-of select="local-name()"/> is a table row.
        Rows must contain the correct cell elements.
    </assert>
</rule>
</pattern>
</schema>

```

Abstract patterns are a useful construct to avoid repeated code. A few detailed notes:

- Although [Example 6.20](#) defines the abstract pattern before it is instantiated, you can place the abstract pattern definitions and instantiations in any order.
- As with abstract rules, abstract patterns are macros. Each pattern-parameter reference is simply replaced with the value of the `value` attribute on the corresponding `<param>` element.
- Related to that, a `<param>` element's `value` attribute is not an XPath expression. It's just a piece of text, used in place of the pattern-parameter.
- Pattern-parameters and variable references share the same syntax: the `$...` notation. This can lead to confusing and subtle bugs if, for example, you use the same name for both a parameter and a variable. For complex schemas I suggest you use a unique naming convention for pattern-parameters (for example, you could start all pattern-parameter names with `p-`).
- Partly because of this shared syntax, the Schematron processor has no way of knowing whether you have supplied all needed pattern-parameters. If you omit the `<param>` element for a pattern-parameter or mistype the parameter's name, the result is implementation-defined. You usually get a vague and uninformative error message.

6.5. Including documents: `<include>`

Like most programming and validation languages, Schematron schemas can include external documents. This is useful when you want to share definitions, such as patterns or rules, between different schemas. This

capability is similar to `<xsl:include>` in XSLT or `<xs:include>` in W3C XML Schema, but there is a serious restriction, which I discuss at the end of this section.

Let's return to [Example 6.6](#), which defines multilingual diagnostic messages inside a `<diagnostics>` element. Suppose you want to put this `<diagnostics>` element in a separate document to make it easier for a translator to work on it. To do this, you need to rewrite the schema as shown in [Example 6.22](#).

Example 6.22 – A Schematron schema that includes an external document (schematron-book-code/examples/advanced/diagnostics-included.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <pattern>
    <rule context="thing">
      <assert test="@type = ('normal', 'special', 'deprecated')">
        diagnostics="type-message-en type-message-nl"/>
      </rule>
    </pattern>

    <include href="separate-diagnostics.sch"/>

</schema>
```

The included file looks like this:

Example 6.23 – The document included by [Example 6.22](#) (schematron-book-code/examples/advanced/separate-diagnostics.sch)

```
<diagnostics xmlns="http://purl.oclc.org/dsdl/schematron">
  <diagnostic id="type-message-en" xml:lang="en">
    Invalid type on <value-of select="@name"/>: <value-of select="@type"/>.
  </diagnostic>
  <diagnostic id="type-message-nl" xml:lang="nl">
    Ongeldig type voor <value-of select="@name"/>: <value-of select="@type"/>.
  </diagnostic>
</diagnostics>
```

And that's all you need to do, no bells or whistles. The `<include>` element inserts the contents of the referenced document.

It sounds simple (and it is), but in practice this simplicity has a serious limitation: you cannot include schema parts if they don't share a common root element! For instance, patterns often belong together. Reviewing [Example 6.18](#), we see that both patterns are necessary. If you wanted to

include them (maybe because you needed them in some other schema also), it would take two external documents and therefore two `<include>` elements. This might be manageable for just two, but what if you have a lot...

In other words, the Schematron `<include>` element has no Schematron-specific semantics. It works just like a simple external entity or `XInclude`.

Chapter 7. Query language binding and using XSLT

Schematron supports query language binding (sometimes abbreviated to QLB). Query language binding allows you to specify the underlying programming language for, among other things, expressions in `<let>` and `<value-of>` elements. Some bindings, most notably XSLT bindings, allow you to add code in an additional programming language, thereby greatly expanding the scope of what you can do.

Theoretically, query language binding gives you a choice of an underlying language. In practice however, this is not the case. As far as I know, all publicly available Schematron processors support XSLT/XPath type bindings only. The Schematron specification mentions a few others, but these are generally not supported.

Having said that, it is, of course, possible that you're working in an environment that uses a Schematron processor that supports other query language bindings. If that's the case, you'll have to find out what works and what doesn't yourself.

Being able to use only XSLT/XPath type bindings is not as limiting as it sounds. Often, Schematron is used in environments that also use XSLT, so developers are already used to this language. XSLT is a powerful language for handling XML documents. Being able to use XSLT opens up a world of possibilities. Some XSLT processors, for instance Saxon EE, have very interesting extensions that support capabilities such as manipulating the file system or querying SQL databases.

So let's not mourn the limitations of the Schematron query language binding but, instead, rejoice at the possibilities of using XSLT. This chapter starts with an introduction to query language binding in general. However, if you're mostly interested in using XSLT feel free to skip ahead to [Section 7.2, "Using XSLT in Schematron"](#).

7.1. Introduction to query language binding

The Schematron specification defines query language binding as: “Named set, specified in a document called a Query Language Binding, of the languages and conventions used for assertion tests, rule-context expressions and so on, by a particular Schematron implementation.”

You need two things for a Schematron query language binding:

- A name (probably “identifier” would be a better term) that uniquely identifies the binding. This name is used as the value of the `queryBinding` attribute on the schema’s root element. A query language binding name is case-insensitive (so `xslt`, `XSLT`, and `XsLt` all mean the same thing).
- A query language binding document that defines what this binding actually means. The Schematron specification (section 6.4) describes what such a document should contain.

Your own query language binding

In theory you could define your own query language binding for some language of your choice and build a Schematron processor that supports it. To stay within the Schematron specifications, just pick a non-reserved query binding name, write a (usually short) document containing the necessary details and implement the language.

In practice, I expect this to be so rarely done that I won’t cover the details about how to do this in this book. If you’re that far into Schematron, you should have enough knowledge to take the information in the official standard and implement a new query language binding yourself.

The Schematron standard reserves a number of query binding names but provides definitions for only some of them. [Table 7.1](#) contains an overview of the reserved names and their status.

- The “Defined?” column indicates whether this binding is defined in the Schematron standard.

If the binding is undefined, I provide a “most obvious meaning,” based on its name. Be aware that this is just me (the author of this book) speaking. Since the binding is undefined you can make it mean anything you like, but be aware that at some point it may be defined differently from what you might like.

- The “Supported?” column indicates whether the publicly available Schematron processors, as of 2022, support this binding (see [Section 4.5, “Schematron validation processors”](#) for information about available processors).

Table 7.1 – Schematron reserved query language bindings

Name	Defined?	Supported?	Description
exslt	Yes	No	This binding augments the default <code>xslt</code> binding with EXSLT functions. EXSLT is an initiative to provide standardized extensions for XSLT. You can find it at http://exslt.org/. EXSLT can be considered outdated (the last change is from 2003), but most of its proposed extensions have found their way into recent XPath versions.
stx	Yes	No	STX (Streaming Transformations for XML) is an XML-based language for transforming XML documents without building a tree in memory, an approach known as <i>streaming</i>. The STX initiative never went beyond the working draft stage. Its last update was in April 2007. Its ideas found their way into XSLT 3.0.
xslt	Yes	Yes	The <code>xslt</code> binding is the default binding if you don’t add the <code>queryBinding</code> attribute to your schema. Expressions both for matching nodes and computing values must follow the XSLT 1.0 standard. This implies the use of XPath 1.0.

Name	Defined?	Supported?	Description
			In addition, it supports the <code><xsl:key></code> element, which can make lookups (for example, for identifiers) significantly faster. Section 7.2.1, “Using XSLT keys” contains an <code>xslt3</code> example that uses keys. This binding also supports the <code><xsl:copy-of></code> element inside <code><property></code> elements (see Section 8.3, “Properties”).
<code>xslt2</code>	Yes	Yes	This language binding is an extension of the <code>xslt</code> binding. It allows XSLT 2.0 expressions, implying XPath 2.0. This gives you a lot more options for your expressions and also more standard functions. Results of expressions are no longer limited to strings but can be any data type. An important additional feature is that it allows you to define XSLT functions in your schema and use these functions in expressions. Examples (although using the <code>xslt3</code> binding) can be found in Section 7.2.2, “Using XSLT functions”.
<code>xslt3</code>	Yes	Yes	This language binding is an extension of the <code>xslt</code> binding. It allows the use of XSLT 3.0 expressions, implying XPath 3.1. All examples in this book use the <code>xslt3</code> binding.

Name	Defined?	Supported?	Description
xpath	No	No	Most obvious meaning: XPath 1.0 expressions can be used. If so, it's almost the same as the <code>xslt</code> binding, but without the ability to use <code><xsl:key></code> and <code><xsl:copy-of></code> .
xpath2	Yes	No	The <code>xpath2</code> binding allows the use of XPath 2.0 expressions for matching nodes and computing values.
xpath3	Yes	No	The <code>xpath3</code> binding allows the use of XPath 3 expressions for matching nodes and computing values. This probably means 3.0, since there is a separate, undefined binding for 3.1 (<code>xpath31</code>).
xpath31	No	No	Most obvious meaning: Allow XPath 3.1 expressions.
xquery	No	No	Most obvious meaning: allow XPath 1.0 expressions and define additional functions using XQuery 1.0.
xquery3	No	No	Most obvious meaning: allow XPath 3.0 expressions and define additional functions using XQuery 3.0.

Name	Defined?	Supported?	Description
xquery31	No	No	Most obvious meaning: allow XPath 3.1 expressions and define additional functions using XQuery 3.1.

7.2. Using XSLT in Schematron

In practice, only the `xslt` type bindings are supported by the current, publicly available Schematron processors. Therefore, let's concentrate on that and more specifically on the most recent one (in 2022): `xslt3`.

7.2.1. Using XSLT keys

Referencing in XML documents is often done using identifiers. For instance, the following example contains orders that reference items by identifier:

Example 7.1 – Example of an XML document that contains references using identifiers. (`schematron-book-code/data/orders-with-ids.xml`)

```
<orders>
  <item id="bolts" price="5.49">A box with 20 bolts</item>
  <item id="nuts" price="3.78">A box with 20 nuts</item>
  <!-- ... many, many more items... -->
  <order>
    <ordered-item id-ref="bolts" quantity="5"/>
    <ordered-item id-ref="nuts" quantity="10"/>
  </order>
  <!-- ... many, many more orders... -->
</orders>
```

The value of each `id-ref` attribute on an `<ordered-item>` element must contain the identifier of an `<item>` element in the same document. [Example 7.2](#) checks this.

Example 7.2 – Schematron schema that checks the identifier references for [Example 7.1](#). (`schematron-book-code/examples/query-binding/check-orders-1.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="ordered-item">
      <let name="item-id" value="@id-ref"/>
      <assert test="exists(/*/item[@id eq $item-id])">
        The referenced item <value-of select="$item-id"/> does not exist
      </assert>
    </rule>
  </pattern>
</schema>
```

The `<let>` element stores the identifier to check in the variable `$item-id`. The `<assert>` uses it to check whether an `<item>` element with the same identifier exists. Straightforward and correct.

But what if the document is very large and contains thousands and thousands of `<item>` elements? Every `<ordered-item>` element causes the schema processor to search all the `<item>` elements, from top to bottom, again and again. That's not very efficient and can take a long time.

A solution is to create a key. This is an in-memory data structure that allows fast lookup of elements using a key index value. XSLT has an instruction for this, `<xsl:key>`, which we can use with an `xslt` binding. [Example 7.3](#) does the same thing as [Example 7.2](#), but much more efficiently.

Example 7.3 – Schematron schema that checks the identifier references for [Example 7.1](#) using a key (`schematron-book-code/examples/query-binding/check-orders-2.sch`)

```
<!-- 1 - Define the XSLT namespace: -->
<schema xmlns="http://purl.oclc.org/dsdl/schematron"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform" queryBinding="xslt3">

  <!-- 2 - Define a key using the XSLT key instruction: -->
  <xsl:key name="item-ids" match="*/item" use="@id"/>

  <pattern>
    <rule context="ordered-item">
      <!-- 3 - Reference the key using the key() function: -->
      <assert test="exists(key('item-ids', @id-ref))">
        The referenced item <value-of select="@id-ref"/> does not exist
      </assert>
    </rule>
  </pattern>
</schema>
```

1. To use instructions from XSLT, you need to declare the XSLT namespace. Hence the `xmlns:xsl="http://www.w3.org/1999/XSL/Transform"` namespace declaration on the root element. Every element that starts with `xsl:` is now considered an XSLT instruction.
2. The XSLT `<xsl:key>` instruction defines a key. It has three components:
 - The name of the key, in this case `item-ids`.
 - The nodes the key is about, in this case the `/*/item` elements.
 - The value of the key, in this case the identifier of the item, contained in its `id` attribute.

What happens under the hood is that the Schematron processor creates a data structure that allows fast lookup of `<item>` elements using the value of their `id` attribute.

3. The `match` attribute of the `<assert>` element uses the XSLT `key()` function to look up values in the data structure. This function takes two or three parameters:
 - The key name, in this case `item-ids`, written as a quoted string (`'item-ids'`).
 - The value to look up, in this case the `id-ref` attribute of the `<ordered-item>` element.
 - The third (optional and unused here) parameter allows you to limit the returned nodes to a specific part of the document (a *subtree*). You do this by specifying the root node of the part you're interested in. Default value is the document node `/`.

The `key()` function performs a fast and efficient lookup and returns the `<item>` element(s) associated with the given identifier. If the identifier is not found the `key()` function returns an empty sequence.

For documents like [Example 7.1](#), keys can speed things up considerably. There are many more nifty things you can do with keys. If you're curious, read the sections about `<xsl:key>` and the `key()` function in the XSLT books listed in [Section E.5, "XSLT"](#).

A warning before we end this topic: keys don't come for free. Building a key takes time, and you have to weigh this against the time raw lookups take (as done in [Example 7.2](#)). In general, don't use keys on small documents. The tipping point is fuzzy. If this is important to you, experiment and measure!

Another example of using XSLT keys in Schematron can be found in [Section 9.4, “Checking multiple identifier references”](#).

Using W3C XML Schema to check identifier references

In W3C XML Schema it's possible to check for correct identifier references as well. An example of a schema that does this for [Example 7.1](#) can be found in `schematron-book-code/examples/query-binding/orders.xsd`.

There are however a few noticeable differences:

- You have no control over the error message when using a schema. On my system the error message looks like this:

```
Key 'key-id-ref' with value '...' not found for identity constraint
of element 'orders'.
```

Not terribly puzzling for the initiated, but it takes some background knowledge (what's an “identity constraint”?) to fully understand it. Schematron, on the other hand, gives you full control over the message.

- As tests get more complicated, XML Schema may no longer be able to handle them. For instance, sometimes identifier references are not singular but a whitespace separated list of values, like `id-refs="nuts bolts"`. When you have only one type of identifier, you can handle this case using the `xs:ID` data type for identifiers and `xs:IDREFS` for references. However, if you have multiple types of identifiers (that is, identifiers that refer to different kinds of things in the XML document), this no longer works.

7.2.2. Using XSLT functions

Modularizing code using functions is a common thing to do when programming. However, Schematron itself lacks the ability to define functions. For this, it relies on the query language binding feature.

For example, suppose you have a reference document that tells you the expected price for items of different types. In [Example 7.4](#), there is a `<data>`

element for each type that includes the name of the type (`type` attribute) and the expected price (`price` attribute).

There is also a default price, defined in the `default-price` attribute on the root element. The default is used when you encounter a thing that has a type that is not in the reference document.

Example 7.4 – A list with type codes and prices (`schematron-book-code/examples/query-binding/type-codes-and-prices.xml`)

```
<type-codes-and-prices default-price="10.0">
  <data type="A125" price="17.25"/>
  <data type="X96" price="89.34"/>
</type-codes-and-prices>
```

You can use this reference document to check whether things listed in documents like [Example 7.5](#) contain the expected price for their type.

Example 7.5 – Data containing type codes and prices (`schematron-book-code/data/things-with-type-codes-and-prices.xml`)

```
<things>
  <thing name="thing 1" type="A125" price="17.25"/>
  <thing name="thing 2" type="A125" price="17.26"/>
  <thing name="thing 3" type="X96" price="89.34"/>
  <thing name="thing 4" type="Y78" price="10.01"/>
</things>
```

To check the price of a thing in [Example 7.5](#), you need to compare it with the expected price for a thing of that type in [Example 7.4](#). If the type is not listed in the reference document, compare the price with the default price.

You could express this directly in Schematron as a complicated and rather long XPath expression, but it's much nicer and more maintainable to define a function for this using XSLT, which is what [Example 7.6](#) does.

Example 7.6 – Schematron schema that checks the prices in [Example 7.5](#) against [Example 7.4](#) (`schematron-book-code/examples/query-binding/check-type-codes-and-prices.sch`)

```
<!-- 1 - Define the XSLT namespace on the root element: -->
<schema xmlns="http://purl.oclc.org/dsdl/schematron"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform" queryBinding="xslt3">
```

```

<!-- 2 - Define a namespace for the functions as an <ns> element: -->
<ns uri="#functions" prefix="f"/>

<!-- 3 - Define your function using XSLT: -->
<xsl:function name="f:get-price" as="xs:double">
  <xsl:param name="type" as="xs:string"/>
  <xsl:variable name="prices-document" as="document-node()"
    select="doc('type-codes-and-prices.xml')"/>
  <xsl:variable name="data-element-for-type" as="element(data)?"
    select="$prices-document//data[@type eq $type]"/>
  <xsl:choose>
    <xsl:when test="exists($data-element-for-type)">
      <xsl:sequence select="xs:double($data-element-for-type/@price)"/>
    </xsl:when>
    <xsl:otherwise>
      <xsl:sequence select="
        xs:double($prices-document/type-codes-and-prices/@default-price)
        "/>
    </xsl:otherwise>
  </xsl:choose>
</xsl:function>

<pattern>
  <rule context="thing">
    <!-- 4 - Use the defined function to get the price: -->
    <let name="expected-price" value="f:get-price(@type)"/>
    <assert test="$expected-price eq xs:double(@price)">
      The price for <value-of select="@name"/> should be
      <value-of select="$expected-price"/>
    </assert>
  </rule>
</pattern>

</schema>

```

1. This example embeds XSLT code in the schema. Therefore, you have to define the XSLT namespace on the root element:

```
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
```

2. XPath function names must also be in a namespace. This example defines the namespace `#functions` with the prefix `f`. You use the prefix when defining and using the `f:get-price` function. The namespace and prefix values were chosen at random; you can use anything you like (for more on namespaces, see [Section 5.5, “Declaring namespaces”](#)).
3. Define your function(s) using the XSLT programming language. In this example, the function is called `f:get-price`, and it takes the value of the `type` attribute as input and if it locates a `<data>` element that has that

type, it returns the value of the `price` attribute. If no `<data>` element matches the input type, it returns the default (`default-price`).

Since this is not a book about XSLT, I'm not going to explain the function itself. As with all programming languages, [Example 7.5](#) is not the only way to express the same functionality.

4. This rule uses the `f:get-price()` function to get the expected price from [Example 7.4](#) and then uses it in the `<assert>` element's test expression.

7.2.3. Using other XSLT features

In addition to embedding XSLT functions in Schematron, can you use other XSLT features, too? For example:

- Functions sometimes have to instantiate XSLT templates, for instance when parsing some complicated piece of XML. Can you embed `<xsl:template>` elements?
- XSLT allows more flexibility in defining global variables than Schematron. For example, you can compute a variable value using XSLT code as a child of the `<xsl:variable>` element. Can you embed `<xsl:variable>` elements?
- It can be more convenient to write XSLT in an IDE and store it in an external file. Can you reference external XSLT code using the `<xsl:include>` or `<xsl:import>` elements?

The answer to all these questions is both no and yes:

- *No.* The official Schematron standard describes the XSLT3 query language binding in annex J. It mentions only a minimal set of allowed XSLT elements, that does not include `<xsl:template>`, `<xsl:variable>`, or `<xsl:include>`. So officially you can't.
- *Yes.* All of the current publicly available Schematron processors support these features.

This leaves you with a choice, depending on your attitude on conformance. If you want to be absolutely compatible with future Schematron processors that are possibly more strict, you should stay within the current language definition and not use any unmentioned XSLT features. But for all practical

purposes, don't bother. It's perfectly safe to use any XSLT construct. And besides that, it's possible that the rules for this will be expanded in a future Schematron version.

Chapter 8. Additional features

Whatever system you devise for categorizing things, there will always be miscellaneous stuff, such as left-over bits and pieces (one of the reasons you always end up with a junk drawer). Schematron is no exception. After describing the core of Schematron in [Chapter 5](#) through [Chapter 7](#), there are still some language bits and pieces left. While these are less often used, they are nonetheless there for a reason.

8.1. Messages with markup: `<emph>`, `<span>`, and `<dir>`

Schematron allows limited markup in the messages defined in `<assert>`, `<report>`, and `<diagnostic>` elements. [Example 8.1](#) contains an example.

Example 8.1 – Example of using markup in an assert message

```
<assert test="...">
  Do <emph>not</emph> use the <value-of select="local-name()"/> element here!
</assert>
```

Three markup elements are allowed:

- The `<emph>` element specifies that the enclosed text should be rendered with emphasis. It has no attributes.
- The `<span>` element is a general-purpose, text-spanning element (like in HTML). It has a `class` attribute to signify what should happen with its contents.

The standard does not specify what a `<span>` element or its `class` signify. CSS styling is an obvious possibility, but other ways to control appearance are also possible.

- The `<dir>` element specifies the writing direction of the text. By default this is left-to-right, but if there's a piece of text in a language that reads from right-to-left (for example Arabic), you can surround it with `<dir value="rtl">`.

Using markup in messages is not common, and how this markup is handled by a processor is implementation defined. Both SchXslt and oXygen copy these markup elements through to the SVRL, but do not display or interpret the markup when displaying error messages. Since the markup is copied to the SVRL, it could be used by an application that processes the SVRL and renders the result as a custom report.

There are several other places where markup is allowed (for example in properties, see [Section 8.3, “Properties”](#)). See the Schematron reference in [Appendix C](#) for details.

8.2. Flags

The `<rule>`, `<assert>`, and `<report>` elements have an optional `flag` attribute. The value of this attribute is the name of a boolean variable. The value of this variable is initially `false`. When a rule, assertion, or report on which this flag is defined fires, the value becomes `true`.

[Example 8.2](#) shows how you could use this mechanism to identify some assertions as more serious than others.

Example 8.2 – Example of using flags to identify important assertions

```
<?xml version="1.0" encoding="UTF-8"?>
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="...">
      <assert test="..." flag="serious-error">This is bad</assert>
      <assert test="...">This is not so bad</assert>
    </rule>
  </pattern>
  <pattern>
    <rule context="...">
      <assert test="..." flag="serious-error">This is also bad</assert>
    </rule>
  </pattern>
</schema>
```

In the validation result you can check whether the `serious-error` flag was set to `true` and, if so, trigger an appropriate follow-up (display a big red error message, sound a horn, sue the person who produced the document, etc.).

How to detect that a flag-variable is set to `true` is implementation-defined. Flags do show up in the SVRL, so they're probably most useful in a scenario where you post-process the SVRL after the validation has taken place. For example, when the first assertion of [Example 8.2](#) fires, the following shows up in the SVRL (see [Example 8.3](#)).

Example 8.3 – SVRL fragment when an assertion with a flag fires

```
<failed-assert location="..." test="..." flag="serious-error">
  <text>This is bad</text>
</failed-assert>
```

8.3. Properties

Schematron properties allow you to add additional arbitrary information to assertions and reports. As with markup, what happens with this information is implementation-defined and probably useful only in a scenario where the SVRL validation results are post-processed.

[Example 8.4](#) shows a simple (invalid) example. It is a document with invoice information (derived from our first example, [Example 1.1](#)).

Example 8.4 – Invalid example XML document with invoice information (`schematron-book-code/data/invoices-2.xml`)

```
<invoices date="2020-11-11" total="10117.82">
  <invoice date="2020-10-04" total="10000.50" id="12345"/>
  <invoice date="2020-10-16" total="100.83" id="56789"/>
  <invoice date="2020-10-18" total="16.5" id="56789"/>
</invoices>
```

This example is invalid because the `total` attribute on the root element is incorrect (off by 1 cent, which is probably a rounding error) and the identifier 56789 is used twice. The schema in [Example 8.5](#) dutifully reports these problems.

Example 8.5 – Schematron schema to validate [Example 8.4](#) (`schematron-book-code/examples/additional-features/properties-1.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
```

```

<pattern>
  <rule context="/invoices">
    <assert test="xs:double(@total) eq sum(invoice/@total)">
      Total <value-of select="@total"/> not correct
    </assert>
  </rule>
</pattern>

<pattern>
  <rule context="invoice">
    <let name="id" value="@id"></let>
    <assert test="count(..invoice[@id eq $id]) eq 1">
      Id <value-of select="@id"/> not unique
    </assert>
  </rule>
</pattern>

</schema>

```

Now suppose you want to post-process the SVRL validation result to produce a report with hints for the developers of your invoicing XML application, but you don't want these hints to be visible to the normal users.

Therefore, you shouldn't put them into the assertion text. Instead, one way to do this is to add the developer information in Schematron properties. This will place that information into the SVRL output.

[Example 8.6](#) adds developer information using Schematron properties.

Example 8.6 – [Example 8.5](#) with developer hints added in properties (schematron-book-code/examples/additional-features/properties-2.sch)

```

<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <properties>
    <property id="programmer-alert">Programmers, watch out!</property>
    <property id="check-rounding-error">
      Please check for rounding errors!
    </property>
    <property id="check-subsystem">Did subsystem XYZ fail again?</property>
  </properties>

  <pattern>
    <rule context="/invoices">
      <assert test="xs:double(@total) eq sum(invoice/@total)"
        properties="programmer-alert check-rounding-error">
        Total <value-of select="@total"/> not correct
      </assert>
    </rule>
  </pattern>

```

```

</pattern>

<pattern>
  <rule context="invoice">
    <let name="id" value="@id"></let>
    <assert test="count(..invoice[@id eq $id]) eq 1"
      properties="programmer-alert check-subsystem">
      Id <value-of select="@id"/> not unique
    </assert>
  </rule>
</pattern>

</schema>

```

Both <assert> elements in [Example 8.6](#) have a properties attribute that references multiple <property> elements by identifier. This attribute allows you to specify multiple identifiers as a whitespace-separated list.

The direct validation outcomes of [Example 8.6](#) are still the same as those of [Example 8.5](#). However, the properties are visible in the resulting SVRL, shown in [Example 8.7](#).

Example 8.7 – The SVRL generated when validating the document in [Example 8.4](#) with the schema from [Example 8.6](#).

```

<schematron-output xmlns="http://purl.oclc.org/dsdl/svrl">
  <active-pattern documents=".../invoices-2.xml"/>
  <fired-rule context="/invoices"/>
  <failed-assert location="/Q{}invoices[1]"
    test="xs:double(@total) eq sum(invoice/@total)">
    <property-reference property="programmer-alert">
      <text>Programmers, watch out!</text>
    </property-reference>
    <property-reference property="check-rounding-error">
      <text>Please check for rounding errors!</text>
    </property-reference>
    <text>Total 10117.82 not correct</text>
  </failed-assert>
  <active-pattern documents=".../invoices-2.xml"/>
  <fired-rule context="invoice"/>
  <failed-assert location="/Q{}invoices[1]/Q{}invoice[2]"
    test="count(..invoice[@id eq $id]) eq 1">
    <property-reference property="programmer-alert">
      <text>Programmers, watch out!</text>
    </property-reference>
    <property-reference property="check-subsystem">
      <text>Did subsystem XYZ fail again?</text>
    </property-reference>
    <text>Id 56789 not unique</text>
  </failed-assert>

```

```

<fired-rule context="invoice"/>
<failed-assert location="/Q{}invoices[1]/Q{}invoice[3]"
  test="count(..invoice[@id eq $id]) eq 1">
  <property-reference property="programmer-alert">
    <text>Programmers, watch out!</text>
  </property-reference>
  <property-reference property="check-subsystem">
    <text>Did subsystem XYZ fail again?</text>
  </property-reference>
  <text>Id 56789 not unique</text>
</failed-assert>
</schematron-output>

```

Properties can go further than this:

- `<property>` elements have an optional `scheme` attribute that, according to the specification, “should be an IRI or other public identifier which specifies the notation used for the metadata value.” For instance, if your property contains an ISO 3166-1 country code like NL, UK, or DE, you can add this information to your property as follows:

```

<property id="country" scheme="ISO 3166-1">NL</property>

```

- Properties also have an optional `role` attribute which can contain even more information about the property. Its value is neither defined nor restricted, so you can come up with anything you like. For instance, the standard contains the following example:

```

<<property id="australianDollar" scheme="ISO 4217"
  role="currency">AUD</property>

```

- The `<property>` element can contain additional attributes (in a different namespace). Properties can contain arbitrary XML (in any namespace). This example property contains a W3C XML Schema fragment:

```

<property id="non-negative-decimal"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <xsd:simpleType type="xs:decimal">
    <xsd:maxInclusive value="0" />
    <xsd:fractionDigits value='2' />
  </xsd:simpleType>
</property>

```

- When an XSLT query binding is used, you can use the `<xsl:copy-of>` element to add information to the property. For instance, this example

copies the full context node (the node that triggered the Schematron rule) to the property:

```
<property id="context" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:copy-of select="."/>
</property>
```

The official Schematron schema (the one that comes with the standard) only allows for a single `<properties>` element to appear in your Schematron schema. However, in most cases, multiple `<properties>` elements are accepted by Schematron processors.

8.4. Adding structured comments: `<title>` and `<p>`

You usually document a schema using XML comments (`<!-- ... -->`). Adding well-worded comments in strategic places makes code more comprehensible for somebody else (or yourself in a few month times). The following example documents [Example 8.5](#):

Example 8.8 – The schema from [Example 8.5](#) with documentation in XML comments added (`schematron-book-code/examples/additional-features/properties-1b.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <!-- This schema validates an invoice list. -->

  <pattern>
    <!-- Checking whether the total attribute is ok: -->
    <rule context="/invoices">
      <assert test="xs:double(@total) eq sum(invoice/@total)">
        Total <value-of select="@total"/> not correct
      </assert>
    </rule>
  </pattern>

  <pattern>
    <!-- Checking whether invoice ids are unique: -->
    <rule context="invoice">
      <let name="id" value="@id"></let>
      <assert test="count(..invoice[@id eq $id]) eq 1">
        Id <value-of select="@id"/> not unique
      </assert>
    </rule>
  </pattern>
</schema>
```

Schematron also has native means for documenting schemas: it allows `<title>` and `<p>` elements in places where you might want to add documentation. For example, you can add these elements as direct children of the root `<schema>` element or inside a `<pattern>` element. [Example 8.9](#) rewrites [Example 8.8](#) using this feature.

Example 8.9 – The schema from [Example 8.5](#) with documentation using Schematron’s `<title>` and `<p>` element (schematron-book-code/examples/additional-features/properties-1c.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <title>Invoice list validation</title>
  <p>This schema validates an invoice list.</p>

  <pattern>
    <title>Total attribute</title>
    <p>This pattern checks whether the total attribute is correct.</p>
    <rule context="/invoices">
      <assert test="xs:double(@total) eq sum(invoice/@total)">
        Total <value-of select="@total"/> not correct
      </assert>
    </rule>
  </pattern>

  <pattern>
    <title>Identifier uniqueness</title>
    <p>This pattern checks whether invoice identifiers are unique.</p>
    <rule context="invoice">
      <let name="id" value="@id"></let>
      <assert test="count(..//invoice[@id eq $id]) eq 1">
        Id <value-of select="@id"/> not unique
      </assert>
    </rule>
  </pattern>
</schema>
```

Why and when would you want to use `<title>` and `<p>` instead of the more usual XML comments? Both methods enable you to explain your code. However, an advantage of using these elements, and the reason Schematron’s designers added them, is that it becomes much easier to extract documentation from the schema.

Schemas are code. There are situations, especially when the code size increases, where you may want to extract information from your code into a separate documentation structure. Maybe you want to produce a PDF that lists everything your schema checks. Or you have a support website with

information about the schemas. Having the base information in structured markup instead of XML comments makes it much easier to create either of these outputs and more.

Having said that, I'm not claiming that it is easy to create documentation this way. You have to write code that extracts the information, build the result page(s), add structure, and more. As far as I know there is currently nothing available that does this. However, if you ever decide to do this, parsing `<title>` and `<p>` elements will be much easier than parsing XML comments.

Here are some details about using structured comments:

- The `<title>` element is allowed only as (single) child of the `<schema>` and `<pattern>` elements.
- The `<p>` element is allowed as (multiple) children of the following elements:
 - `<schema>` (after an optional `<title>` element)
 - `<pattern>` (after an optional `<title>` element)
 - `<phase>`
 - `<rule>`
- There are limited ways to markup the contents of the `<title>` and `<p>` elements (see also [Section 8.1, “Messages with markup: `<emph>`, `<span>`, and `<dir>`”](#)):
 - The `<title>` element allows in its contents the `<dir>` element (writing direction) only.
 - The `<p>` element allows all Schematron's markup elements.
- `<title>` and `<p>` elements in the schema are not copied to the resulting SVRL.

8.5. Validating documents referenced by XInclude

XML documents frequently include other documents. We have already seen this with the Schematron `<include>` element. Including external documents is

so common that there's a standard for it, called XInclude. Schematron can access documents that are referenced by XInclude as it processes the including document. [Example 8.10](#) uses XInclude to include two sub-documents:

Example 8.10 – Master document that includes two other documents using XInclude (schematron-book-code/data/subdocuments-master.xml)

```
<document xmlns:xi="http://www.w3.org/2001/XInclude">
  <xi:include href="subdocuments-1.xml"/>
  <xi:include href="subdocuments-2.xml"/>
</document>
```

The included documents are shown in [Example 8.11](#) and [Example 8.12](#).

Example 8.11 – The first included document by [Example 8.10](#) (schematron-book-code/data/subdocuments-1.xml)

```
<document>
  <p class="emphasize">This is the first included sub-document.</p>
</document>
```

Example 8.12 – The second included document by [Example 8.10](#) (schematron-book-code/data/subdocuments-2.xml)

```
<document>
  <p class="brightred">This is the second included sub-document.</p>
</document>
```

Suppose you want to check whether <p> elements in the included documents use the brightred class and issue a message if one does. The schema in [Example 8.13](#) does this.

Example 8.13 – Schematron schema to validate the included documents of [Example 8.10](#) (schematron-book-code/examples/additional-features/subdocuments-1.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <ns prefix="xi" uri="http://www.w3.org/2001/XInclude"/>
  <pattern documents="/*/xi:include/@href">
    <rule context="p">
      <assert test="@class ne 'brightred'">
        The class <value-of select="@class"/> is forbidden
      </assert>
    </rule>
  </pattern>
</schema>
```

```

        (<value-of select="base-uri(.)"/>)!
    </assert>
</rule>
</pattern>

</schema>

```

- The `<ns>` element defines the XInclude namespace using the usual `xi` prefix.
- The Schematron processor encounters the `<pattern>` element with the `documents` attribute on it. It evaluates the attribute's contents, `/*/xi:include/@href`, as an XPath expression.

Notice that this is an XPath expression that accesses the reference to the external file in the document you are processing. If you want to specify a file system path directly, you need to put it between quotes, for instance `documents="'subdocument.xml'"`.

- This results in a sequence of two values: `subdocuments-1.xml` and `subdocuments-2.xml`. Both values are interpreted as a reference to another document.
- The rules in the pattern are resolved against both of the referenced documents but not against the original master document.
- The line `<value-of select="base-uri(.)"/>` in the message displays the URI of the document. Displaying the URI is optional, but if you don't it can be difficult to determine which document the message refers to.

Some details:

- A document referenced in a `documents` attribute is evaluated only for rules found inside the pattern that references it. Rules in other patterns are not applied to the referenced document.
- Evaluation of a document referenced using a `documents` attribute is done in the context of that document's root node. Therefore, it is safest to use absolute XPath expressions, as in [Example 8.13](#).
- When a pattern's `documents` attribute results in relative references (as in [Example 8.13](#)), these are resolved against the document that is validated.

- The resulting value of the `documents` attribute must be a URI. This means that if it resolves to an absolute path (and not a relative path as in [Example 8.13](#)), it must begin with `file:/`, for instance `file:/C:/docs/subdoc.xml`.

Finally, while useful, this mechanism has one, rather limiting, drawback: it cannot handle nested includes. For instance, if `subdocument-2.xml` ([Example 8.12](#)) included some other document, that sub-document would not be validated. Since nesting of includes is common in many XML applications, this limitation can be significant.

8.6. Specifying a role: the `role` attribute

The `role` attribute on an assertion, report, or property allows you to say something about its role. But what is a role? The Schematron standard defines it as “a name describing the function of the assertion or context node in the pattern” and provides neither (preferred) values nor examples. Therefore, what a role is, its values, what it’s used for, and how it is used, is up to you. Frequently, `role` is used to signify the severity of assertions and reports. For instance, [Example 8.14](#) uses `role` to indicate that this assertion is a fatal error.

Example 8.14 – Example of using a `role` attribute on an assertion

```
<assert test="..." role="fatal">This is very bad</assert>
```

The oXygen IDE assigns a meaning to certain values of the `role` attribute and changes the representation of messages depending on the value. To see this in action, run the schema in [Example 8.15](#) in oXygen against any XML document. [Figure 8.1](#) shows the result.

Example 8.15 – Schema to show the effect of different `role` attribute values in oXygen (`schematron-book-code/examples/additional-features/roles-1.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="/*">
      <assert test="false()" role="info">This is informational</assert>
      <assert test="false()" role="warning">This is a warning</assert>
      <assert test="false()" role="error">This is an error</assert>
    </rule>
  </pattern>
</schema>
```

```

    <assert test="false()" role="fatal">This is a fatal error</assert>
  </rule>
</pattern>
</schema>

```

Figure 8.1. The effect of specifying different roles in oXygen in the Results window

Results		
	Info	Description - 4 items
test.xml, schema "flags-1.sch" (4 items)		
	-	[ISO Schematron] This is informational
	-	W [ISO Schematron] This is a warning
	-	E [ISO Schematron] This is an error
	-	F [ISO Schematron] This is a fatal error

However, handling of the `role` attribute is implementation specific. oXygen's useful interpretation of the `role` attribute is not part of the standard, so other IDEs or schema validators can process roles differently or not process them at all.

Some details:

- The `role` attribute can appear on `<assert>` and `<report>` elements.
- It can also appear on the parent `<rule>` element, specifying the default role for all the assertions and reports in this rule. Usually a `role` attribute on a child `<assert>` or `<report>` element overrides this default, but this is not defined in the standard as such. Whether it works that way, however logical it may seem, is implementation defined.
- The `role` attribute can also appear on `<property>` elements (see [Section 8.3, "Properties"](#)).
- The `role` attribute is present in the resulting SVRL, allowing you to act on it when post-processing validation results.

8.7. Specify a different location: the `subject` attribute

As we've learned, the Schematron `<rule>` element specifies the context: the location of the content we're validating. Child assertions and reports

normally apply to this location. However, sometimes the context specified in the `<rule>` element is not the location you want to apply the rule to. This can happen if you try to choose a context that is efficient to locate.

For example consider [Example 8.16](#), which is a simple table.

Example 8.16 – Simple table document (`schematron-book-code/data/simple-table.xml`)

```
<table>
  <row>
    <entry>First cell</entry>
    <entry>Second cell</entry>
    <entry>Third cell</entry>
    <entry>Fourth cell</entry>
  </row>
</table>
```

Assume that tables are not allowed to have more than two columns. It is easy to check this with the schema in [Example 8.17](#).

Example 8.17 – Schematron schema to check the number of columns for [Example 8.16](#). (`schematron-book-code/examples/additional-features/subject-1a.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="row">
      <assert test="count(entry) le 2">Row with superfluous column(s)</assert>
    </rule>
  </pattern>
</schema>
```

If you run this, the validation error identifies the `<row>` element, as specified by the `<rule>` element's `context` attribute. The oXygen IDE for instance shows this with a red squiggly line on this element.

But what if you want to specify the error on the first superfluous column (`<entry>` element)? This might be useful when the table gets more complex, or maybe users find it easier to fix this error when the exact column is specified. [Example 8.18](#) takes care of this and correctly identifies the third and fourth `<entry>` elements of [Example 8.16](#) as the error locations.

Example 8.18 – Schematron schema to check the number of columns for [Example 8.16](#) (`schematron-book-code/examples/additional-features/subject-1b.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <pattern>
    <rule context="entry">
      <assert test="count(preceding-sibling::entry) lt 2">
        Superfluous column
      </assert>
    </rule>
  </pattern>

</schema>
```

`preceding-sibling::?`

If you're new to XPath, the `preceding-sibling::` construct is probably unfamiliar to you. It's an example of what is called an "XPath axis." An XPath axis tells the XPath processor where to look to match the XPath expression that follows the axis.

For example, the XPath expression `preceding-sibling::entry` in [Example 8.18](#) returns all `<entry>` elements that precede the context item and have the same parent. Since the assertion that contains this XPath expression is inside a rule that matches every `<entry>` element in the document, the context item is an `<entry>` element, and the XPath expression returns every `<entry>` element that comes before the context item and has the same parent element (in this case, `<row>`). It then counts the number of elements returned and issues a message if there are two or more.

XPath defines several axes. See https://www.w3schools.com/xml/xpath_axes.asp for more information. Another example of using axis can be found in [Section 9.6, "Validating doubled elements in mixed content"](#).

Although XPath axes are useful and often the best way to accomplish a particular goal, in this set of examples, [Example 8.18](#) is less clear and efficient than [Example 8.17](#):

- There usually will be far fewer `<row>` than `<entry>` elements in a document, so using the `<row>` element as context is better for performance reasons.
- [Example 8.18](#) is more complex and, therefore, less easy to interpret than [Example 8.17](#). Simply counting the number of child `<entry>` elements is easy to grasp, while mucking about with preceding siblings adds complexity and might need a comment to explain it.

For situations like this the `subject` attribute comes to the rescue. Its content is an XPath expression that, evaluated against the context of the rule, leads to the subject of the error. [Example 8.19](#) rewrites [Example 8.18](#) using this feature.

Example 8.19 – Schematron schema to check the number of columns for [Example 8.16](#), using the `subject` attribute to shift the error location to the first superfluous column. (schematron-book-code/examples/additional-features/subject-1c.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="row" >
      <assert test="count(entry) le 2" subject="entry[3]">
        Start of superfluous column(s)
      </assert>
    </rule>
  </pattern>
</schema>
```

The XPath expression in the `subject` attribute (`entry[3]`) is evaluated using the context of the rule (the `<row>` element). This flags the third `<entry>` element as the error location.

Some details:

- The `subject` attribute can appear on `<assert>` and `<report>` elements.
- It can also appear on the parent `<rule>` element, specifying the default subject for all assertions and reports of this rule. Usually a `subject` attribute on a child `<assert>` or `<report>` element overrides this default, but this is not defined in the standard. Whether it works that way, however logical it may seem, is implementation defined.
- In the resulting SVRL, the location of an assertion or report shows up in the `<failed-assert>` or `<successful-report>` element's `location` attribute. This allows you to identify the intended subject location when post-processing the SVRL.

8.8. The `<name>` element

The `<name/>` element inserts the name of the context item (usually the name of an element or attribute) into a message. It can be used in locations where `<value-of/>` is allowed.

When using XPath-enabled query bindings (the only ones currently available), the element is superfluous. You can use `<value-of select="name()"/>` instead. However, in the rare case that you are using a non-XPath-enabled query binding, `<name/>` can be useful.

8.9. The `icon`, `see`, and `fpi` attributes

Last in this chapter about additional features are three attributes: `icon`, `see`, and `fpi`. These attributes can be used in several places, including the root `<schema>` element and the `<pattern>`, `<rule>`, `<assert>`, and `<report>` elements. Usually, if you use one of these attributes on an element, its value overrides any value set for that attribute on an ancestor element. [\[16\]](#)

The meaning of these attributes is defined in the standard, but usage is implementation-defined. They show up in the resulting SVRL, so they can be used when post-processing the results.

`icon`

The `icon` attribute contains a link to a graphic file associated with an assertion or report, which, for example, you could display for a failed assertion.

`see`

The `see` attribute contains a link to additional information about assertions or reports. For example, you could point to a web page with additional information about an error. [\[17\]](#)

`fpi`

The `fpi` attribute is defined as “a formal public identifier for the schema, phase, or other element.” This attribute and its rather cryptic definition are holdovers from the SGML days and unlikely to be relevant.

[\[16\]](#) As logical as that may seem, this behavior is implementation defined.

[\[17\]](#) The oXygen IDE uses the value of the `see` attribute to provide a link to additional information in the Results window.

Chapter 9. Schematron examples and recipes

This chapter contains some more advanced examples and recipes for applying Schematron. All code and data is in the book's example code GitHub repository. See [Section 1.3, “Using and finding the code examples”](#) for information on finding and downloading examples.

Introducing advanced XPath techniques and operators

Because of the more advanced nature of this chapter, I use several XPath techniques and operators that are not covered in the technology primer in [Appendix A](#). I explain anything not covered there in a note like this.

9.1. Validating a Schematron schema

Schematron schemas are XML documents and, as such, can be validated themselves. Doing this adds value, especially when you're writing a schema using a schema-aware editor. A schema-aware editor can flag mistakes and help you by showing what elements and attributes are valid at the current cursor position.

The official schemas for validating Schematron are part of the Schematron standard and listed in its PDF. To get them in machine-usable format, you have to copy the code from the PDF (page by page) and paste it into an appropriate document—a cumbersome procedure. To save everyone work and money (the PDF of the standard is not free), you can obtain these schemas (for Schematron and the reporting language SVRL) in two ways:

- Clone or download the GitHub repository <https://github.com/Schematron/schema>.
- Download them from the example code GitHub repository for this book, subdirectory `schematron-book-code/schemas`.

Whichever source you choose, you will get the following files:

- `schematron.rnc` and `svrl.rnc` are the RELAX NG compact schemas from the standard. These are straight copies, except the versions included in

the GitHub repository for this book include a provenance comment added at the beginning of each file.

- `schematron.sch` and `svrl.sch` are the original Schematron schemas from in the standard. These are almost straight copies, except the versions included in the GitHub repository for this book include a provenance comment added at the beginning of each file and a bug fix in the SVRL schema.[\[18\]](#)

The Schematron schemas provide an additional layer of validation, on top of the RELAX NG schemas. It may sound like hocus pocus to use Schematron schemas to validate Schematron schemas, like Baron Munchhausen rescuing himself from the swamp by pulling his own hair. But there's no magic involved: nothing prevents you from using a Schematron schema to validate another Schematron schema.

- The GitHub repository for this book also includes the files `schematron-converted.xsd` and `svrl-converted.xsd`, which are the W3C XML Schemas, converted (by me, using the oXygen schema converter) from their RELAX NG Compact versions. These conversions are approximations (RELAX NG has some different features than W3C XML Schema) and needed a little patching to make them valid. Nonetheless, I am providing them because W3C XML Schema is more widely supported than RELAX NG.

Note

There are some subtle differences between the RELAX NG and the W3C XML Schemas, but, unless you need to be really strict, there is nothing to be concerned about. For instance, the RELAX NG schema requires at least a single `<pattern>` element, but the W3C variant allows zero-or-more. Be aware that there might be other differences that I didn't spot.

How to make these validations happen?

- Some XML-aware IDEs and editors, for instance oXygen, automatically detect that you're editing a Schematron schema and

provide validation without further ado. You don't even need the schemas provided in the example code of this book.

- The XML community has developed common ways to include a reference to a schema in an XML file. If your environment supports this functionality, it should trigger validation when you work with a file that contains such references. Here are some examples:
 - Referencing a RELAX NG compact schema:

Example 9.1 – A reference to the RELAX NG compact schema for Schematron (schematron-book-code/templates/empty-schema-relaxng-compact-validation.sch)

```
<?xml-model
  type="application/relax-ng-compact-syntax"
  href="../schemas/schematron.rnc"
?>
<schema xmlns="http://purl.oclc.org/dsdl/schematron"
  queryBinding="xslt3">
  <pattern/>
</schema>
```

- Referencing a W3C XML Schema:

Example 9.2 – A reference to the W3C XML Schema for Schematron (schematron-book-code/templates/empty-schema-w3c-schema-validation.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron"
  queryBinding="xslt3"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://purl.oclc.org/dsdl/schematron
    ../schemas/schematron-converted.xsd">
  <pattern/>
</schema>
```

- Referencing a Schematron schema:

Example 9.3 – A reference to the Schematron schema for Schematron (schematron-book-code/templates/empty-schema-schematron-validation.sch)

```
<?xml-model
  type="application/xml"
```

```

    schematypens="http://purl.oclc.org/dsdl/schematron"
    href="../schemas/schematron.sch"
?>
<schema xmlns="http://purl.oclc.org/dsdl/schematron"
queryBinding="xslt3">
  <pattern/>
</schema>

```

Test to find out what works in your environment. You will need to change the path to the schemas, ../schematron/... in the examples, to the real (relative or absolute) path in your environment.

In some environments you can have multiple schema references. If supported, you can have references to RELAX NG plus Schematron or W3C XML Schema plus Schematron, which gives you the benefit of checking the validity of your document against both the XML schema and the Schematron rules.

- Other environments may need a specific configuration to set up validation. You're on your own here, but it can be worth the effort to figure out what works in your environment.

9.2. Handling a default namespace

Even among XML specialists it's not unusual to run into a frustrating situation where you have written your code (in Schematron or another XML language) to the best of your abilities, checked it multiple times, and it still doesn't do anything. Nothing. Zilch. When this happens, the odds are that somewhere a namespace plays a role (if you don't know what I'm talking about, the XML namespace mechanism is explained in [Appendix B](#)).

Namespaces can be tricky, and their declarations are hard to spot. Because of the default namespace mechanism (`xmlns=""`), elements that are in a namespace don't always need a prefix. And when this default namespace is declared somewhere way up in a long and complicated document, it's easy to miss. So when your code seems perfect but your XPath expressions refuse to work, look again and check the data for namespace usage.

Schematron, because of the way namespaces must be defined, doesn't have the concept of a default namespace (see [Section 5.5, "Declaring namespaces"](#)). However, documents, the data to be checked, can use a

default namespace. If you're new to namespaces it can be rather mysterious how to handle this. Fortunately, the solution is simple.

Suppose you're writing a Schematron schema that checks a piece of text written in DocBook. DocBook is a standard for texts like books, articles, and manuals (this book, for instance, is written entirely in DocBook).

[Example 9.4](#) contains a short piece of text expressed in DocBook.

Example 9.4 – Some text expressed in DocBook (schematron-book-code/data/docbook-text.xml)

```
<article xmlns="http://docbook.org/ns/docbook" version="5.0">
  <title>An example document</title>

  <sect1>
    <title>The first section</title>

    <para>A paragraph of text.</para>
    <para>A <emphasis>second</emphasis> paragraph of text!</para>
  </sect1>

</article>
```

One of the things you might want to validate for this example is that sections on the first level (<sect1> elements) must have at least three paragraphs of text (<para> elements). This makes [Example 9.4](#) invalid. A first attempt at writing a Schematron schema for this might look like [Example 9.5](#).

Example 9.5 – A first, non-working, attempt at a Schematron schema validating [Example 9.4](#) (schematron-book-code/examples/examples-recipes/check-docbook-1.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="sect1">
      <assert test="count(para) ge 3">
        This section must contain at least three paragraphs of text
      </assert>
    </rule>
  </pattern>
</schema>
```

When you run this schema against the data in [Example 9.4](#), nothing will happen. Your computer will proclaim the data as valid, which it clearly

isn't. Something is wrong, and, yes, it has to do with namespaces:

- Because of the default namespace declaration `xmlns="http://docbook.org/ns/docbook"` on the root element of [Example 9.4](#), all of its elements are in the DocBook namespace.
- However, the two XPath expressions in [Example 9.5](#), `sect1` and `count(para) ge 3`, do not reference a namespace: they expect the elements to be in the null namespace.
- To an XML processor, a `<sect1>` or `<para>` element in the null namespace is considered different from elements with the same names in the DocBook namespace.
- Therefore, although the element names are the same, nothing happens. The namespaces in the document and the XPath expressions in the Schematron schema differ.

This situation is confusing because the element names in [Example 9.4](#) are written exactly the same as the names used in the Schematron schema of [Example 9.5](#). It's the default namespace declaration on the root element of [Example 9.4](#) that causes the problem.

How can we solve this? Schematron has no default namespace mechanism for its XPath expressions, so that road is closed. The only remaining option is to use Schematron's namespace mechanism, the `<ns>` element:

- Define the DocBook namespace using the `<ns>` element and assign some prefix to it:

```
<ns prefix="db" uri="http://docbook.org/ns/docbook"/>
```

You are free to choose the value of the prefix attribute, but the value of the `uri` attribute must match the DocBook namespace name (its URI).

- Use this prefix in the XPath expressions that refer to DocBook elements: `db:sect1` and `count(db:para) ge 3`.

[Example 9.6](#) shows the resulting Schematron schema,

Example 9.6 – A working Schematron schema validating

[Example 9.4](#) (schematron-book-code/examples/examples-recipes/check-docbook-

2.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <ns prefix="db" uri="http://docbook.org/ns/docbook"/>
  <pattern>
    <rule context="db:sect1">
      <assert test="count(db:para) ge 3">
        This section must contain at least three paragraphs of text
      </assert>
    </rule>
  </pattern>
</schema>
```

The XPath expressions in [Example 9.6](#) reference the DocBook namespace using the namespace prefix mechanism. [Example 9.4](#) references the DocBook namespace using the default namespace mechanism. However, the mechanism for referencing a namespace doesn't matter: as long as the referenced namespace name is the same, the elements are considered the same.

9.3. Using Schematron for schema validation

When you validate using schema languages such as DTD, W3C XML Schema, or RELAX NG the messages do not always make sense to non-technical users. The message will say something is wrong, but it may not say exactly what is wrong, and it can be unclear what you need to do to fix the problem. One of Schematron's major strengths is that you control the messages. Because of this, some people use Schematron for validations that are usually done with other languages.

However, Schematron cannot easily check everything that other validation languages check. For instance, checks on the order of elements (<a> elements must always come before elements) are hard to implement efficiently. Other checks, like the presence of attributes/elements and checks on their data types, can be done but need a lot of repetitive and boring code.

The repetitive nature of this kind of code led to the idea of generating a Schematron schema from another schema language. For W3C XML Schema and RELAX NG, which are expressed in XML, you

can use XSLT. It's not easy, because schemas can be complex, but it is possible.

Writing such a generator is beyond the scope of this book, so I'll leave this to your imagination, but in this section, I provide you with some guidance on how to implement some of the more classic schema checks using Schematron.

[Example 9.7](#) contains a sample XML instance to illustrate these checks.

Example 9.7 – The XML for validation using Schematron (`schematron-book-code/data/xml-for-validation.xml`)

```
<data timestamp="2022-03-01T12:12:23">
  <thing thingid="12345">Something</thing>
  <thing thingid="12346">Another thing</thing>
  <object>Object 1</object>
  <object remark="just an object">Object 2</object>
</data>
```

Here is a list of checks we will use to validate this example (the Schematron examples in this section refer to the numbers in this list):

1. The root element must be `<data>`.
2. The root has a required attribute called `timestamp`.
3. The value of the `timestamp` attribute must be a valid date/time.
4. There must be between one and three child `<thing>` elements.
5. A `<thing>` element has a required `thingid` attribute.
6. The value of the `thingid` attribute must be a valid integer.
7. Each `<thing>` element is followed by zero or more `<object>` elements.
8. The `<object>` element has an optional `remark` attribute.

Validation number 1, [Example 9.8](#), checks the root element.

Example 9.8 – Validating the root element (`schematron-book-code/examples/examples-recipes/validation-1.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <!-- Validation check 1: -->
  <pattern>
    <rule context="/">
      <assert test="exists(data)">
```

```

        The root element must be data
    </assert>
</rule>
</pattern>

</schema>

```

Validations 2 and 3, [Example 9.9](#), check the `timestamp` attribute.

Example 9.9 – Validating the attribute on the root element ([schematron-book-code/examples/examples-recipes/validation-2-3.sch](#))

```

<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <!-- Validation check 2 and 3: -->
  <pattern>
    <rule context="/data">
      <assert test="exists(@timestamp)">
        The root element must have a timestamp attribute
      </assert>
      <assert test="empty(@timestamp) or @timestamp castable as xs:dateTime">
        The timestamp attribute must hold a valid date/time
      </assert>
      <assert test="empty(@* except @timestamp)">
        Only the timestamp attribute is allowed on the root element
      </assert>
    </rule>
  </pattern>

</schema>

```

- The first `<assert>` in [Example 9.9](#) checks whether an attribute called `timestamp` is present.
- The second `<assert>` checks whether the value of `timestamp` is a valid date/time.

The XPath `castable as` operator

The XPath `castable as` operator returns `true` when a piece of data can be cast to a certain type. If `@timestamp castable as xs:dateTime` in [Example 9.9](#) returns `true`, this means that the value of the `timestamp` attribute is a valid date/time and can be transformed to the right data type without raising an error using `xs:dateTime(@timestamp)`.

You can find more about working with dates and times in [Section A.6.6, “Working with dates, times and durations”](#).

Why is `empty(@timestamp)` part of the second assertion? It is there to avoid a double error message if there is no `timestamp` attribute at all. So this test first makes sure the attribute exists and only then performs the check on its data type.

- The third `<assert>` in [Example 9.9](#) checks whether there are any other attributes on the root element, which is not allowed.

The XPath `except` operator

The XPath `except` operator takes two sequences and returns all members of the first sequence that do not occur in the second sequence. So in this case it will return all attributes except the attribute that is allowed. If this result is empty, everything is fine, but if not, the assertion raises an error.

What if there were more attributes allowed, for instance an `id` attribute? In that case you would write:

```
empty(@* except (@timestamp, @id))
```

Validations 4 and 7, [Example 9.10](#), check the `<thing>` and `<object>` elements.

Example 9.10 – Validating the root’s child elements (`schematron-book-code/examples/examples-recipes/validation-4-7.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <!-- Validation check 4 and 7: -->
  <pattern>
    <rule context="/data">
      <assert test="count(thing) ge 1 and count(thing) le 3">
        There must be between 1 and 3 thing elements
      </assert>
      <assert test="empty(* except (thing, object))">
        Only thing and object elements are allowed as children of the root
      </assert>
    </rule>
  </pattern>

</schema>
```

- The first assertion in [Example 9.10](#) counts the number of `<thing>` elements and makes sure the total is between 1 and 3.
- You don’t need to check the `<option>` element, because it is optional and there is no upper limit on how many times it can appear.

- The second assertion checks whether there are any elements underneath the root other than the allowed `<thing>` and `<object>` elements. This assertion uses the same technique as was used in [Example 9.9](#) to check for extra attributes on the root element.

Validations 5 and 6, [Example 9.11](#), check the `thingid` attribute. This check is similar to the check for validations 2 and 3 in [Example 9.9](#).

Example 9.11 – Validating the `thingid` attribute on the `<thing>` element (schematron-book-code/examples/examples-recipes/validation-5-6.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <!-- Validation check 5 and 6: -->
  <pattern>
    <rule context="/data/thing">
      <assert test="exists(@thingid)">
        A thing must have a thingid attribute
      </assert>
      <assert test="empty(@thingid) or @thingid castable as xs:integer">
        A thingid must be a valid integer
      </assert>
      <assert test="empty(@* except @thingid)">
        Only the thingid attribute is allowed on the thing element
      </assert>
    </rule>
  </pattern>

</schema>
```

Finally, validation 8, [Example 9.12](#), checks the attributes on the `<object>` element. Because the `remark` attribute is declared optional, and there are no type constraints for it, we only have to check whether any other attributes are present. [\[19\]](#)

Example 9.12 – Validating the attributes on the `<object>` element (schematron-book-code/examples/examples-recipes/validation-8.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <!-- Validation check 8: -->
  <pattern>
    <rule context="/data/object">
      <assert test="empty(@* except @remark)">
        Only the remark attribute is allowed on the object element
      </assert>
    </rule>
  </pattern>

</schema>
```

</schema>

I hope you agree with me that this is not the most exciting code. Writing all this by hand is boring and error-prone. Bottom line: you can partially implement classic schema checks in Schematron, but I would only do this when controlling the error messages is critically important.

9.4. Checking multiple identifier references

Referencing in XML documents is often done using identifiers. [Section 7.2.1, “Using XSLT keys”](#) contains an example of how to use XSLT indexes to check these kinds of references.

However, sometimes you will find references to multiple identifiers in a single attribute. For instance, `idrefs="id1 id2 id3"` references three identifiers. Of course, you can check that all of these references exist using Schematron.

This example works with an excerpt from Shakespeare’s play *Macbeth*, adapted from the full text on <http://gutenberg.org/>, and turns it into XML using a fictitious play markup language.^[20]

A role in the play (there are 39 of them...) is defined by a `<Role>` element (there are also role groups combining roles, defined with `<RoleGroup>` elements). Each role gets an identifier using the `role-id` attribute. For instance, [Example 9.13](#) defines the role of Macbeth.

Example 9.13 – Example of defining a role in the XML rendering of Macbeth.

```
<Role role-id="MACBETH">
  <Name>Macbeth</Name>
  <PreferredSex>male</PreferredSex>
  <Description>General in the King's army</Description>
</Role>
```

Clauses link to a role using the `role-ref` attribute (entrances and exits also use the `role-ref` attribute). [Example 9.14](#) shows a clause that contains a piece of text spoken by Macbeth, who is identified by the `role-ref` attribute.

Example 9.14 – Example of referencing roles in the XML rendering of Macbeth.

```
<Clause role-ref="MACBETH">
  <Text>I go, and it is done; the bell invites me.<Br/>
    Hear it not, Duncan, for it is a knell<Br/>
    That summons thee to heaven or to hell.
</Text>
</Clause>
<Exits>
  <Exit role-ref="MACBETH"/>
</Exits>
```

The task is to validate whether every `role-ref` attribute references a role in the play.

Checking multiple identifier references using other schema languages

A W3C XML or RELAX NG schema can handle a single identifier attribute (in this case `role-id`), as having type `xs:ID` and multiple attributes as having type `xs:IDREFS`. In this case, these schemas can flag any reference errors.

However, [Example 9.14](#) contains two types of identifiers: one for roles (`role-id` attributes) and one for scenes (`scene-id` attributes). If you define both of these attributes as being of type `xs:ID`, a W3C XML or RELAX NG processor can't handle them separately. When an attribute that was supposed to refer to roles instead refers to a scene, this problem will not be detected.

[Example 9.15](#) is a Schematron schema that checks whether all the roles referenced in `role-ref` attributes actually exist. For this example, I'm using the XSLT key mechanism, which is explained in [Section 7.2.1, "Using XSLT keys"](#)).

Example 9.15 – Checking the role references in Macbeth, assuming `role-ref` attributes contain a single identifier (`schematron-book-code/examples/recipes/check-role-refs-1.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <xsl:key name="role-ids" match="*/Roles/*" use="@role-id"/>

  <pattern>
    <rule context="@role-ref">
      <assert test="exists(key('role-ids', .))">
        The referenced role id <value-of select="."/> does not exist
      </assert>
    </rule>
  </pattern>
```

```

    </assert>
  </rule>
</pattern>
</schema>

```

When you validate the entire play against this schema it reports 72 validation problems. These problems can be divided into two types:

- Some `role-ref` attributes contain multiple references. For instance on line 320:

```
<Entry role-ref="DUNCAN MALCOLM DONALBAIN LENNOX ATTENDANTS"/>
```

Multiple players enter the stage at this point, which is expressed in a single `role-ref` attribute.

- Some `role-ref` attributes contain the special value `#all` (for instance on line 305), which signifies that this applies to all players who are on stage at that moment in the play.

Let's solve these problems one by one.

To handle a `role-ref` attribute with multiple values, we have to break it into its constituents and check each separately. [Example 9.16](#) does this.

Example 9.16 – Checking the role references in Macbeth, handling multiple references in the `role-ref` attributes (`schematron-book-code/examples/examples-recipes/check-role-refs-2.sch`)

```

<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <xsl:key name="role-ids" match="/*/Roles/*" use="@role-id"/>

  <pattern>
    <rule context="@role-ref">
      <assert test="
        every $ref in tokenize(., '\s+')[.]
          satisfies exists(key('role-ids', $ref))
      ">
        Role reference(s) in "<value-of select="."/>" not found
      </assert>
    </rule>
  </pattern>
</schema>

```

The value of the `role-ref` attribute is turned into its members. Each member is checked (using the key lookup we've already seen in [Example 9.15](#)). When one of these lookups fails, the assertion fails and a message is issued. If you run [Example 9.16](#) against the play, only the `role-ref="#all"` attributes are still reported as invalid.

The XPath `tokenize()` function

The expression `tokenize(., '\s+')[.]` in [Example 9.16](#) separates the value of the `role-ref` attribute apart into its constituents:

- The `tokenize()` function takes a string as its first argument and splits it into a sequence of substrings.
- The second argument of `tokenize()` tells the function where to split the string. This argument must be a regular expression (or regexp).^[21] The regexp `\s+` matches a string of one or more whitespace characters (spaces, tabs, newlines, etc.).
- The `[.]` after the `tokenize()` function is a predicate (like in [Section A.5, "Predicates in tree navigation"](#)). It examines the sequence of strings coming out of the `tokenize()` function for empty strings, which happens when the attribute value starts or ends with whitespace. The effective boolean value of an empty string is `false`, so the predicate `[.]` kicks them out of the sequence.

The result of all this is that the value of the `role-ref` attribute is turned into a sequence of substrings, each of which holds a separate identifier.

The XPath `every ... satisfies ... expression`

The expression `every $ref in ... satisfies ...` in [Example 9.16](#) checks all members of a sequence against some boolean expression. If the boolean expression returns `true` for every member of the sequence, the expression in its entirety returns `true`, otherwise it returns `false`.

The last thing we need to add to our Schematron schema is a check for the special value `#all`. [Example 9.17](#) is what probably comes to mind first.

Example 9.17 – Checking the role references in Macbeth, handling multiple references and the special value `#all` in the `role-ref` attributes (schematron-book-code/examples/examples-recipes/check-role-refs-3.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <xsl:key name="role-ids" match="*/Roles/*" use="@role-id"/>
```

```

<pattern>
  <rule context="@role-ref">
    <assert test="
      every $ref in tokenize(., '\s+')[.]
        satisfies exists(key('role-ids', $ref))
    or
      . eq '#all'
    ">
    Role reference(s) in "<value-of select="."/>" not found
  </assert>
</rule>
</pattern>
</schema>

```

This example works, but there is a different, possibly less obvious, way to solve this using a different technique: another `<rule>` element. Remember that only one rule in a pattern is executed. So if you catch instances of `role-ref="#all"` before checking for identifiers, the identifier check never happens. [Example 9.18](#) uses this technique.

Example 9.18 – An alternative for [Example 9.17](#): catch any `role-ref="#all"` attributes before checking the identifier references
([schematron-book-code/examples/examples-recipes/check-role-refs-4.sch](#))

```

<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <xsl:key name="role-ids" match="*/Roles/*" use="@role-id"/>

  <pattern>
    <rule context="@role-ref[. eq '#all']">
      <assert test="true()"/>
    </rule>
    <rule context="@role-ref">
      <assert test="
        every $ref in tokenize(., '\s+')[.]
          satisfies exists(key('role-ids', $ref))
      ">
      Role reference(s) in "<value-of select="."/>" not found
    </assert>
  </rule>
</pattern>
</schema>

```

The rule element `<rule context="@role-ref[. eq '#all']/>` catches any `role-ref="#all"` attributes. When it fires, its only `<assert>` test gets executed. This assertion never fails, because its test expression is the constant `true()`. It may look like a trick to have a constant, never-failing assertion, but you

must include it because `<rule>` elements cannot be empty. When this rule fires, the second rule is never checked. While this method may look more complex, it is actually a straightforward use of Schematron that may be a bit more efficient than the previous example.

9.5. Validating processing instructions and comments

Schema languages, like DTD, W3C XML Schema, and RELAX NG, are unable to check processing instructions (`<? ... ?>`) or XML comments (`<!-- ... -->`). They can check neither their presence nor their contents, but Schematron can.

The anatomy of a processing instruction

Consider [Example 9.19](#), which is a fictitious processing instruction.

Example 9.19 – An example processing instruction

```
<?xmlvalidate type="something" check="full"?>
```

This processing instruction has two parts:

- **Name:** `xmlvalidate`
- **Value:** `type="something" check="full"`

The value of a processing instruction often leads to misunderstandings. In most cases, like [Example 9.19](#), the value looks like one or more attributes. However, that is not the case. What comes after the processing instruction's name is just a string. XML parsers do not attempt to turn this into separate attributes.

If you do want to process the value as though it is a list of attributes, you have two choices:

- If you're using Saxon EE combined with an `xslt2` or `xslt3` query language binding, use the Saxon-specific function `saxon:get-pseudo-attribute()`.[\[22\]](#)
- You can write a parser yourself using regular expressions. This goes beyond the scope of this book.

To see how you can validate processing instructions and comments, suppose you have data coming from an external supplier that you need to validate. In addition to the normal element and attribute validations, you also want to validate the following things:

- The data must have the processing instruction `<?process type="full"?>` up front.
- The first child of the root element must be a comment that contains at least the name of the data supplier, `ACMEDATA`.

[Example 9.20](#) is a valid example of such a document.

Example 9.20 – Some XML data with a processing instruction (`schematron-book-code/data/data-with-pi.xml`)

```
<?process type="full"?>
<external-data>
  <!-- Produced by ACMEDATA -->
  ...
</external-data>
```

As [Appendix A](#) explains, XPath treats XML as a tree-shaped data structure. This data structure contains the elements, attributes, and text as well as any processing instructions and comments. When navigating XPath expressions, you can use `processing-instruction()` and `comment()` to match processing instructions and comments, respectively. You can match a processing instruction by name by adding the name between the parentheses. For example, you could match a processing instruction named “process” using `processing-instruction(process)`.

Why do you need parentheses at the end of `comment()` and `processing-instruction()`?

You might wonder why the terms `processing-instruction()` and `comment()` are suffixed by parentheses, almost as if they’re functions.

The reason for this is that if you omit these parentheses they’re indistinguishable from `<processing-instruction>` and `<comment>` elements. XPath wouldn’t know whether you wanted to address an element or a processing instruction/comment. The same reasoning applies to `text()`, which matches any text and `node()`, which matches any node.

[Example 9.21](#) checks the validity of documents like [Example 9.20](#).

Example 9.21 – A Schematron schema that checks the validity of data like [Example 9.20](#) (`schematron-book-code/examples/examples-recipes/check-pi-comments-1.sch`)

```

<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <!-- 1 - Test for a processing-instruction (pi) with the name 'process': -->
  <pattern>
    <rule context="/">
      <assert test="processing-instruction(process)">
        There must be a leading process pi present
      </assert>
    </rule>
  </pattern>

  <!-- 2 - Test the contents of this processing-instruction: -->
  <pattern>
    <rule context="/processing-instruction(process)">
      <assert test="contains(., 'type=&quot;full&quot;')">
        The process pi must contain type="full"
      </assert>
    </rule>
  </pattern>

  <!-- 3 - Check for a comment mentioning ACMEDATA: -->
  <pattern>
    <rule context="/*">
      <assert test="comment()[contains(., 'ACMEDATA')]">
        There must be a comment mentioning ACMEDATA underneath the root
      </assert>
    </rule>
  </pattern>

</schema>

```

The schema contains three patterns:

1. The first pattern checks for the presence of the processing instruction. It takes the document node as context (context="/"). Then it asserts whether this has a child processing instruction called `process` (test="processing-instruction(process)").
2. The second pattern checks for the contents of the processing instruction (context="/processing-instruction(process)"). If found, it becomes the context item, and the assertion checks whether its value matches the expected string.

Character entity usage

Notice the two instances of the string `"` in the test expression. This string is a character entity. Character entities are used to represent characters that, if inserted directly, would be interpreted as operators or delimiters. In this case, `"` represents a double quote (`"`).

The character entity is necessary here because we want to check for a string that contains double quotes, but the test expression is inside an XML attribute delimited by double quotes. If the test was `contains(., 'type="full"')`, the XML interpreter would not be able to tell where the attribute value ended. Therefore, we have to write the quotes as character entities.

Here are some common character entities defined in XML:

- `>`; for the greater-than character `>`
- `<`; for the less-than character `<`
- `&`; for the ampersand `&`
- `'`; for the single quote `'`

3. The third pattern tests for a comment underneath the root element (`/`*) that contains the word `ACMEDATA`.

[Example 9.21](#) can still be improved. For instance, when the processing instruction is moved to the end of the document (after the closing `</external-data>`), it's still considered valid. The same happens when the comment containing `ACMEDATA` is not the first child of the root element. Adjusting this requires some advanced XPath usage that would distract too much from the purpose of this book. [\[23\]](#)

9.6. Validating doubled elements in mixed content

XML is often used for expressing textual data, like web pages, book contents, etc. This kind of data is called *mixed content*. For example:

```
<p>Hello <i>Schematron</i> lovers!</p>
```

The `<p>` element has children of type text and of type element (the `<i>` element).

One of the things that is very hard to express efficiently in validation languages like DTD, W3C XML Schema, or RELAX NG is *doubled markup*, that is, a situation where an element is nested inside an element of the same name. Here, the `<i>` element is doubled; one `<i>` is nested in another:

```
<p>Hello <i><i>Schematron</i></i> lovers!</p>
```

Here is an example of a doubled `<b>` element, hidden inside a child `<i>` element:

```
<p>Hello <b><i><b>Schematron</b></i></b> lovers!</p>
```

Expressing constraints like “a `<b>` element cannot be a child or a descendant of a `<b>` element” leads to a combinatorial explosion of schema rules in classic validation languages. This in turn leads to enormous and unmaintainable schemas, especially when the number of elements that can be used for marking up text goes up. As a result, doubled markup is usually not checked.

However, Schematron can handle this case using a simple assertion. Suppose you want to check [Example 9.22](#) for doubled `<b>`, `<i>`, and `<u>` elements?

Example 9.22 – Data with a doubled `<b>` element (`schematron-book-code/data/doubled-elements.xml`)

```
<body>
  <p>Hello <i>you</i>, are you seeing <b><i><b>double</b></i></b>?</p>
</body>
```

The Schematron schema in [Example 9.23](#) checks for these doubled elements.

Example 9.23 – A schema that checks [Example 9.22](#) for doubled elements (`schematron-book-code/examples/examples-recipes/check-doubled-elements-1.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="i">
      <assert test="not(ancestor::i)">
        You cannot have doubled i elements
      </assert>
    </rule>
    <rule context="u">
      <assert test="not(ancestor::u)">
        You cannot have doubled u elements
      </assert>
    </rule>
    <rule context="b">
      <assert test="not(ancestor::b)">
        You cannot have doubled b elements
      </assert>
    </rule>
  </pattern>
```

</schema>

The three rules trigger on the elements we want to check. The assertions check whether an element (an ancestor) with the same name occurs between the element we triggered on and the root of the document.

Axis steps

Axis steps are expressions that, starting from a context node, select a related set of nodes. They are written as a name followed by a double colon (e.g., `ancestor::`). Here are some frequently used examples:^[24]

- The `ancestor::` axis step selects all ancestors of the context node, up to the document node (its parent, grandparent, and so on).
 - The `preceding-sibling::` axis step selects all nodes that have the same parent but come before the context node in the document.
 - The `following-sibling::` axis step does the same but for the nodes that come after the context node.
-

It's possible to reduce [Example 9.23](#) to just a single rule by using the XPath “or” operator (`|`).^[25] If you write `i|u|b`, XPath interprets this as either `<i>` or `<u>` or `<b>`. Using this construct, [Example 9.23](#) becomes:

Example 9.24 – A much shorter version of [Example 9.23](#) (`schematron-book-code/examples/examples-recipes/check-doubled-elements-2.sch`)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">

  <pattern>
    <!-- 1 - Trigger on all elements we want to check: -->
    <rule context="i|u|b">
      <!-- 2 - Store the name element that triggered this rule in a
            variable: -->
      <let name="current-element-name" value="local-name()"/>
      <!-- 3 - Use this variable to check whether there are any ancestors
            with the same name: -->
      <assert
        test="not(ancestor::*[local-name() eq $current-element-name])">
        You cannot have doubled <value-of select="$current-element-name"/>
        elements
      </assert>
    </rule>
  </pattern>

</schema>
```

1. Use the single pipe `|` operator to combine the elements to be checked. Notice that this construct makes it easy to expand the list.
 2. Store the name of the element that triggered the rule in the variable `current-element-name`.
 3. Look for any ancestor element (`ancestor::*`) that has the same name.
-

[18] The schema does not define a `queryBinding` attribute, so its query language defaults to `xslt`, which limits XPath expressions to version 1.0. Unfortunately, this makes the schema invalid, because one of the XPath expressions (`<sch:value-of select="$attribute/name()"/>` on line 133) uses an XPath 2.0 construct. Therefore, the attribute `queryBinding="xslt2"` was added.

[19] `schematron-book-code/examples/examples-recipes/validation-combined.sch` contains a full Schematron schema that combines all of these checks.

[20] The complete XML document is 3,373 lines long, so I show only fragments here. You can find the full marked-up text in the GitHub repository for this book (`schematron-book-code/data/macbeth.xml`).

[21] A regular expression is a sequence of characters that are used to search for patterns in text. A detailed discussion of regular expressions is beyond the scope of this book, but you can find a good, detailed introduction on Wikipedia at: https://en.wikipedia.org/wiki/Regular_expression.

[22] See <https://www.saxonica.com/documentation11/index.html#!functions/saxon/get-pseudo-attribute>.

[23] For the curious, see `schematron-book-code/examples/examples-recipes/check-pi-comments-2.sch`, which is an improved and annotated example.

[24] The XPath standard defines 13 axis steps. You can find an overview at <https://www.w3.org/TR/xpath-31/#axes>. See https://www.w3schools.com/xml/xpath_axes.asp for another useful list and explanation.

[\[25\]](#) Officially, this is called the “union” operator, but for the purpose of this example it’s easier to think of it as “or.”

Appendix A. XPath technology primer

To apply Schematron, you need a basic knowledge of the underlying XPath technology. This appendix describes the basics of this. It's useful information for beginners, but it might also be of value for seasoned programmers in other languages who need to know their way around XPath.

XPath is a truly wonderful invention in the XML world. It's a standard for things like **selecting data in XML documents** and for expressions on simple data, like `1 + 2` or `"XP" || "ath"`. But these properties in themselves, as nifty as they are, don't make XPath wonderful. It's the fact that XPath is used everywhere that makes it stand out. All important XML-related languages, including XSLT, XQuery, and Schematron, use XPath as a foundation. And this means that if you understand XPath, you're already halfway to understanding any XML language!

Schematron relies heavily on XPath, which means that if you want to write Schematron schemas, you will need to understand XPath. This appendix assumes only that you've seen XML and worked with it at least a little.

XPath is a huge subject and could easily fill a book. What I will try to do here is provide you with enough background information to understand the examples in this book. Even this takes a lot of pages! But if you master XPath to this level, writing simple to medium-complex Schematron schemas is definitely within reach. For those that want or need to know more, see [Section E.2, "XPath"](#).

Simplifications...

To prevent newcomers from dropping out, this appendix simplifies XPath quite a bit. I chose not to describe every nook, cranny, and detail. For instance, I cover only simple tree navigation, and I left out axis specifiers (`preceding-sibling::` etc.), maps, and arrays. I also make no distinction between XPath selection and (XSLT) match expression.

Yes, all of this omitted information is important if you want to master XPath. But to reach the level you need to use Schematron, you don't need to understand the full range of XPath capabilities.

A.1. XML as a tree

Trees as used here are concepts from computer science. To work with XML and XPath, you need to start thinking of an XML document as a tree of data.

For those that are new to this I can almost hear you think: “Tree? What tree? I just see text with a lot of angled brackets, not a tree....” Yes, superficially that seems to be the case. But all these angled brackets are just a way to describe a tree-shaped structure. How does this work and why is it important? Let’s dive in.

A.1.1. Basic trees: **documents**, **elements**, and **text**

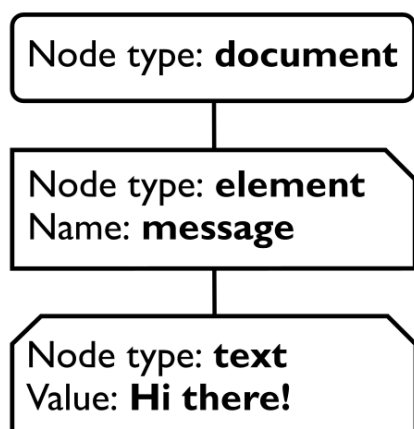
An XPath tree representation is an abstract way of thinking about how XML documents are structured. This abstraction is the basis for the XPath way of navigating XML documents. Let’s start our orchard gardening with a very simple example document:

Example A.1 – A very simple XML document

```
<message>Hi there!</message>
```

An abstract tree representation for this document, in the XPath sense, looks like [Figure A.1](#):

Figure A.1. The XPath tree representation for [Example A.1](#)



Let’s take a closer look at the tree in [Figure A.1](#):

- XPath trees are made up of **nodes**: the various shaped boxes in [Figure A.1](#). Our simple XML document in [Example A.1](#) contains three nodes.
- Nodes always have a **type**. The tree in [Figure A.1](#) contains nodes of three types: document, element, and text.
- A node can have other **properties**, depending on its type. For instance, the node of type element has a property called name with a **value** message.
- A tree for an XML document always starts with a node of type document.

- Every element in an XML document, like `<message>` in [Example A.1](#), becomes a node of type `element`.

An element always has a property called `name` that holds the name of the element, in this case: `message`.

- Text inside an element becomes a node of type `text`.

Text nodes have a property called `value` that holds the actual literal `text`, in this case: `Hi there!`

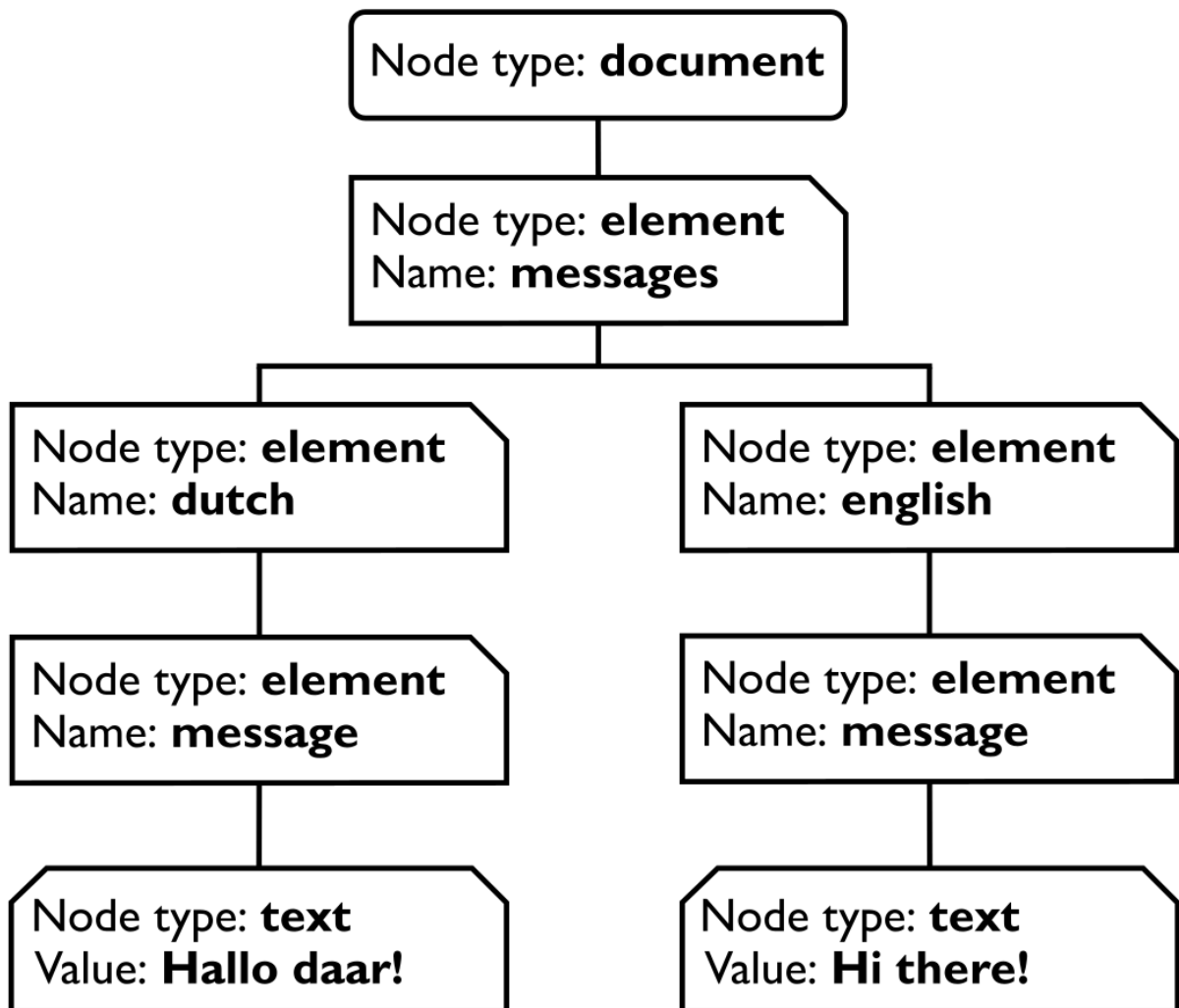
Let's do this again for a somewhat less simple document:

Example A.2 – A less simple XML document

```
<message$>
  <dutch>
    <message>Hallo daar!</message>
  </dutch>
  <english>
    <message>Hi there!</message>
  </english>
</message$>
```

A tree for this document looks like [Example A.2](#):

Figure A.2. The XPath tree representation for [Example A.2](#)



- Again there is the starting node of type document.
- After that comes the element node for the `<messages>` root element.
- The root element has two element *children*: `<dutch>` and `<english>`. As you can see, the tree splits here and contains a node for each of them.
- The `<dutch>` and `<english>` elements each have a *child* `<message>` element.
- And each `<message>` element has a child node of type text that contains the contents.

As was done above, it's customary to talk about relations in such a tree in family terms:

- The `<messages>` element has two *child* nodes: the `<dutch>` and `<english>` elements.
- The *parent* of the `<english>` and `<dutch>` elements is the `<messages>` element.
- The `<dutch>` and `<english>` elements are *siblings*.

If you've never seen representations like this before, it often raises questions:

- **It doesn't look like a tree!** It's drawn upside-down! Wouldn't "root system" be a better name?

Regarding data structures, anything that branches out like [Figure A.1](#) and [Figure A.2](#) is called a tree, regardless of how it is drawn. That we draw it with the starting node at the top is just a convention.

- **Where are the closing tags?** Looking at, for instance, [Figure A.1](#): where is the closing `</message>` in this tree? Shouldn't there be a node of type `element` with name `/message` as child of the `text` node?

The answer is no. Even though an element in XML-as-text is made up of an opening and closing tag, that does not mean you need to represent both in the tree. The name of a closing tag is always the same as its opening tag, so it would be superfluous to have these in the tree.

Another way of looking at it is that the closing tags are just there for the textual representation of the XML. It's the tree representation that leads, not the XML-as-text representation.

- **Why is there a document node?** Any well-formed piece of XML has a single root element only. So for the always-single root element, a parent seems unnecessary.

Yes, there always is a single root element. But XML has two constructs that are allowed outside the root element: **comments** (`<!-- ... -->`) and **processing instructions** (`<?... ..?>`). It is valid to have these as siblings of (so before or after) the root element. Without a document node, you couldn't represent these "outliers" in the tree representation.

Representing comments and processing instructions in the tree

XML documents can also contain comments (`<!-- ... -->`) and processing instructions (`<?... ..?>`):

- Comments are meant to **add information for humans**, like `<!-- Start of chapter: -->` OR `<!-- Incomplete! Fix this later! -->`
- Processing instructions add additional **instructions for the systems that process the XML**. For instance, they're often used to **add references to schemas**. For an example see [Section 9.1, "Validating a Schematron schema"](#).

Comments and processing instructions are also represented in the tree, using nodes of type `comment` and `processing-instruction`. These node types rarely play a role in Schematron schemas. An example

of how to work with them can be found in [Section 9.5, “Validating processing instructions and comments”](#).

If you intend to do something with them, a warning: XML documents often start with the so-called **XML header**, for instance:

```
<?xml version="1.0" encoding="UTF-8"?>
```

Such a header is not considered a processing instruction (although it certainly mimics one). Therefore you will not find it in the tree representation of your document.

A.1.2. Attributes in the tree

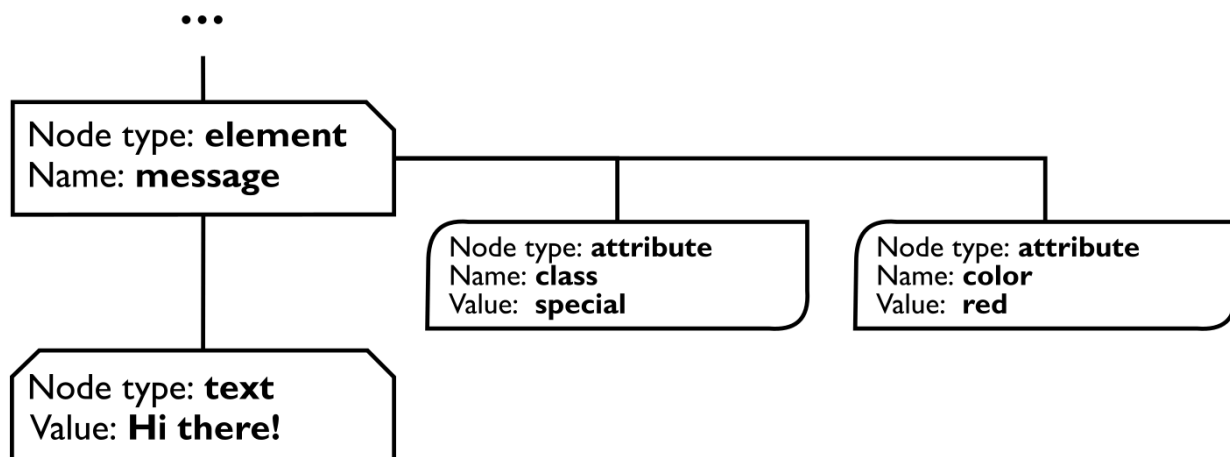
Our examples so far haven’t contained any attributes. How are these represented? The following element, appearing somewhere in a document, contains some:

Example A.3 – An element with some **attributes (appearing somewhere in a document)**

```
... <message class="special" color="red">Hi there!</message>
...
```

An abstract tree representation for this element looks like [Figure A.3](#):

Figure A.3. The XPath tree representation for [Example A.3](#)



Attributes are represented by nodes also. They appear as an offshoot of the element node that they belong to.

- Their node type is (unsurprisingly): attribute.
- They have a **property name** which holds the name of the attribute.

- They also have a **property called value that holds the contents of the attribute.**

Some questions that this might raise include:

- Why do attributes have a property that holds their value and not a node of type `text`, like elements have?

Very good question, but maybe it should be stated the other way around: why have elements a separate node of type `text`, instead of a `value` property? That would make the model simpler, wouldn't it?

The reason for text in elements being in separate nodes is that you can't represent mixed contents (like `<p>hi <i>there</i>!</p>`) without using separate nodes.^[26] However, the value of an attribute is just a string, a piece of text. So for attribute values the model can be simple and a `value` property suffices.

- Does the order of the attributes matter?

The simple answer here is: no. The XML specification says that the order of attributes is insignificant. So `<p class="large" id="x123"/>` is the same as, and must be processed the same as, `<p id="x123" class="large"/>`.

Of course, attributes in a document, and therefore the nodes in the tree representation, are in an unavoidable sequence. But code, whether it is an XSLT stylesheet or a Schematron schema, should never rely on the order of attributes.

A.1.3. Representing mixed content

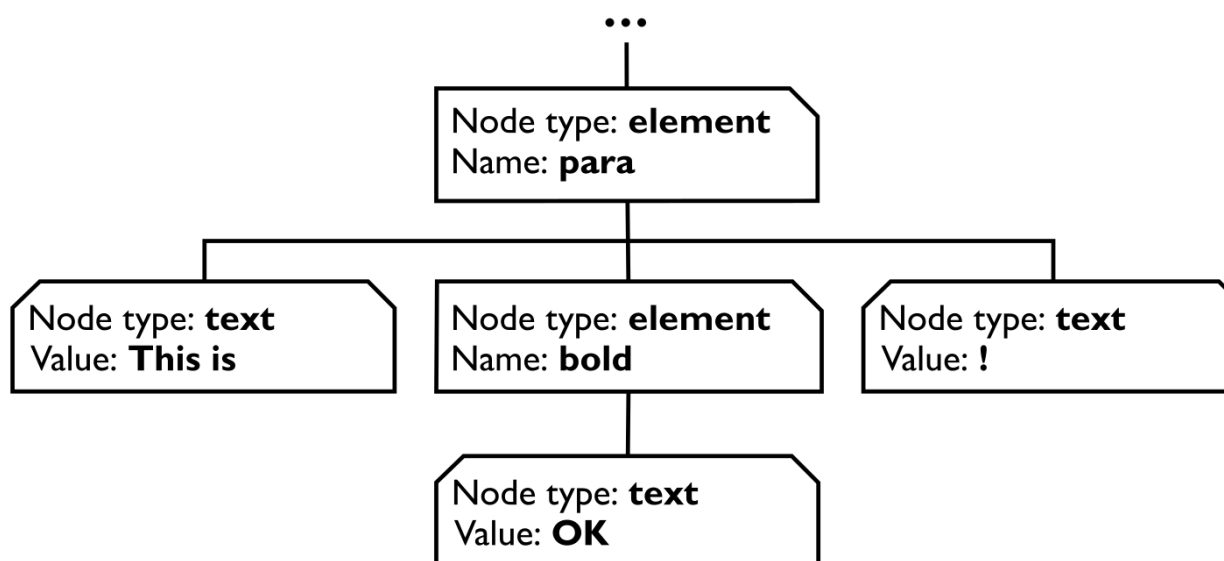
Mixed contents is XML where text and elements are mingled, for instance:

Example A.4 – An element containing mixed contents (appearing somewhere in a document)

```
...
<para>This is <bold>OK</bold>!</para>
...
```

You most often encounter this in situations where human readable text needs to be represented in XML, for instance on an (X)HTML page or in book content. With the node types we've encountered so far it's not difficult to turn [Example A.4](#) into a tree:

Figure A.4. The XPath tree representation for [Example A.4](#)



It's simply a matter of following the content of the element from left to right:

- A piece of text becomes a text node.
- A nested element, like `<bold>` in [Example A.4](#), becomes a sibling node of type `element`.
- If this nested element contains something (text, other elements), the tree gets an extra level to represent this.

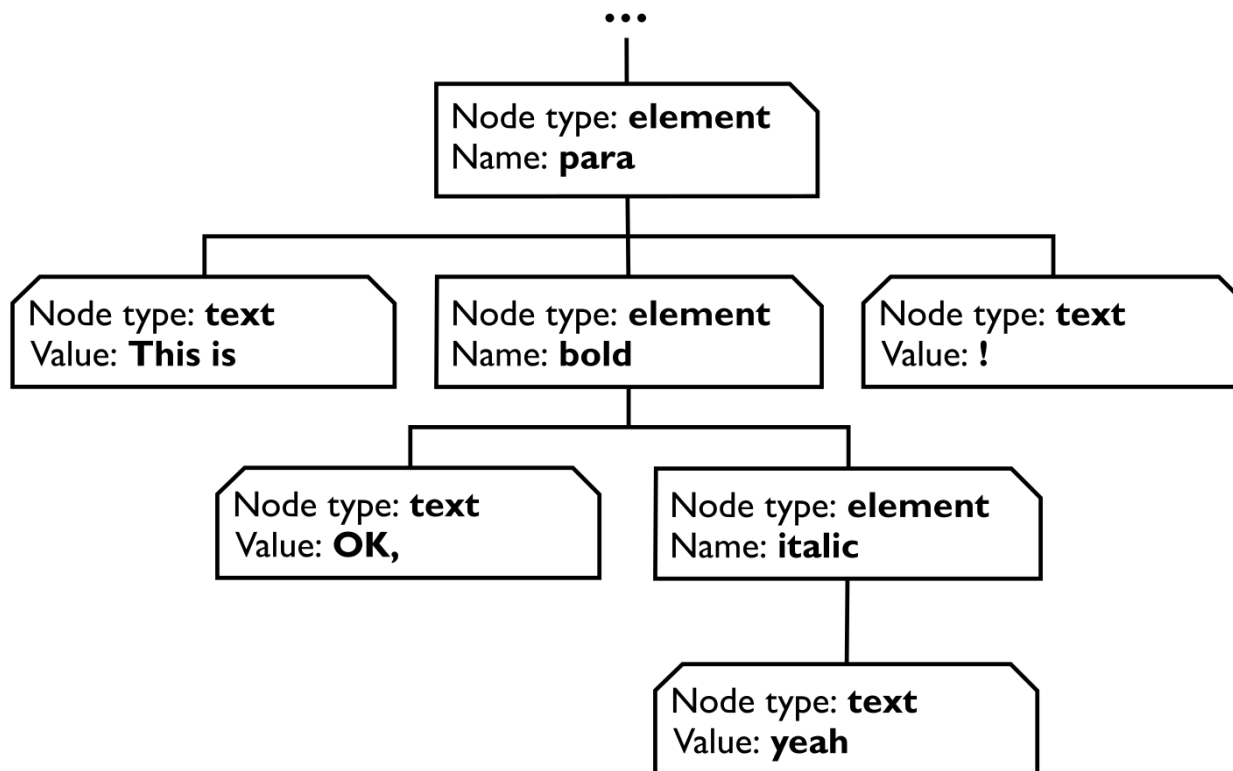
And so on, and so forth. If there's another nested element, we simply dive deeper:

Example A.5 – An element containing more complex mixed contents (appearing somewhere in a document)

```
...  
<para>This is <bold>OK, <italic>yeah</italic></bold>!</para>  
...
```

The tree representation for [Example A.5](#) is:

Figure A.5. The XPath tree representation for [Example A.5](#)



A.2. Basic tree navigation

I hope that by now you have an idea of how we can view XML documents as abstract tree-shaped structures. But what we haven't handled yet is why... Why do you need to know this and how are these structures useful?

Consider [Example A.6](#), which I will return to in the following sections. Suppose you have a document with human-readable messages in various languages (a variation on [Example A.2](#)). The document also specifies the color the messages should be displayed in (the `color` attribute) and that English is the default language (the `default` attribute).

Example A.6 – An XML document with human-readable messages (`schematron-book-code/data/basic-messages.xml`)

```
<messages>
  <dutch color="red">
    <message>Hallo daar!</message>
  </dutch>
  <english color="yellow" default="true">
    <message>Hi there!</message>
  </english>
  <french color="purple">
```

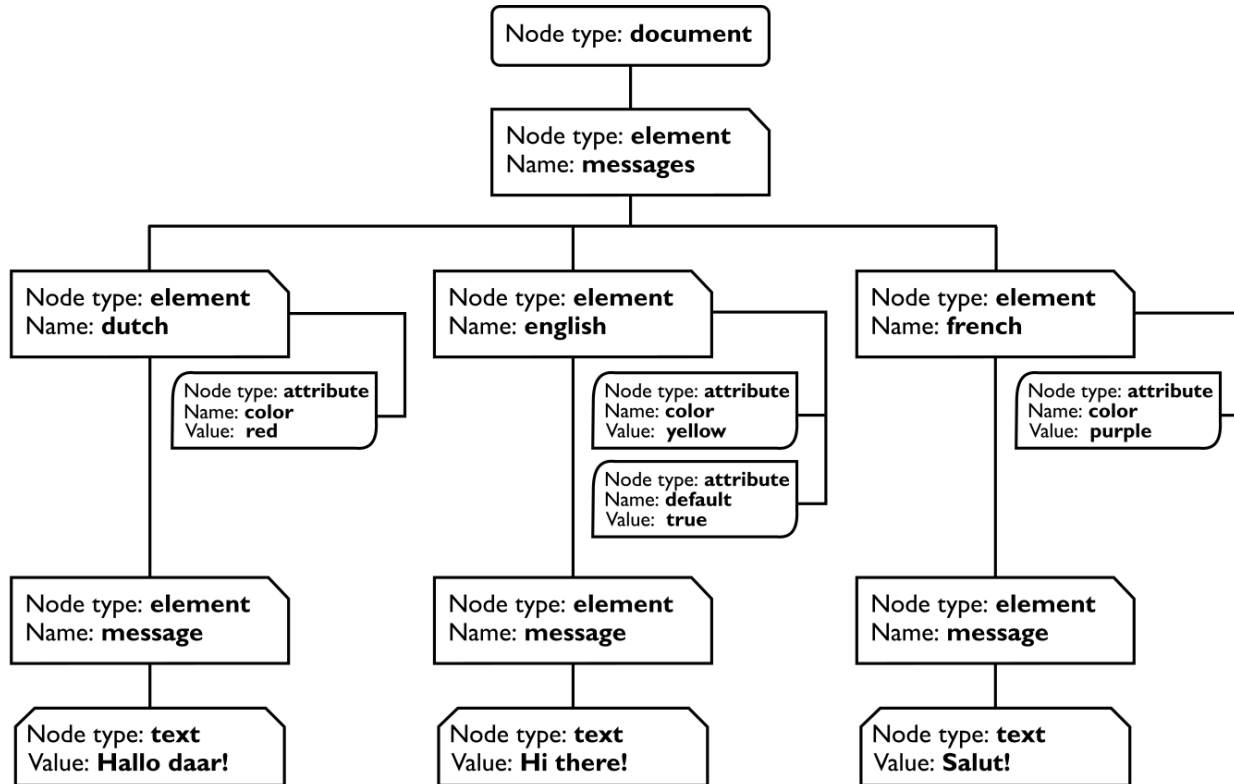
```

    <message>Salut!</message>
  </french>
</messages>

```

The tree for this document looks like [Figure A.6](#):

Figure A.6. The XPath tree representation for [Example A.6](#)



Given this document, we want to find the French message. How do you do that using XPath?

Basic XPath tree navigation uses the **element names as stepping stones** and the **slash character / as separator**. Using this, the XPath navigation expression to select the French message becomes:

Example A.7 – XPath expression to select the French message in [Example A.6](#))

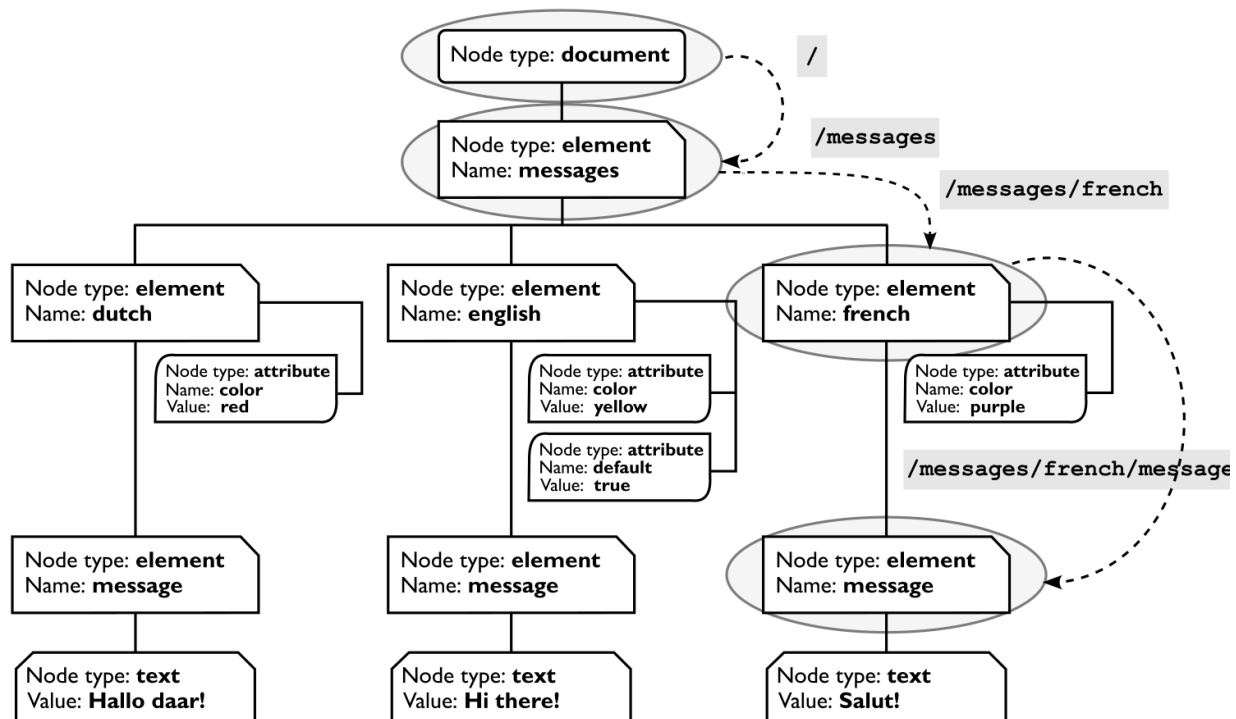
```
/messages/french/message
```

- The first character is a /, which means we start our journey through this document at its top-level document node.
- It then uses the name of the first (root) element in the tree: `messages`.

- Separated by another / comes the language element we want to navigate to: french.
- And finally, separated again by a /, the name of the element that holds the actual message: message.

If we plot this in the document's tree, it looks like [Figure A.7](#):

Figure A.7. The XPath expression `/messages/french/message` shown as a document tree



Tree navigation and file systems...

You might notice a striking similarity with navigating/addressing files and directories in a file system, especially when you're used to Unix- or Linux-based systems with paths like `/home/erik/work/...`. I suppose some inspiration for the XPath design came from this convention.

File/directory paths on Windows look a bit different, using backslashes (`\`) and drive letters (for instance `C:\users\erik\work\...`), but the idea is the same.

A.2.1. Using navigation expressions

To further illustrate this, let's see how we can actually use these expressions. Assume for instance **you want to show the French message in an HTML page**. In [XSLT](#), a language that relies heavily on XPath, you could write something like [Example A.8](#).

Example A.8 – Using an XPath tree navigation expression in XSLT

```
<p>  
  <xsl:value-of select="/messages/french/message"/>  
</p>
```

This would result in the HTML code `<p>Salut!</p>`.

Schematron also uses XPath navigation expressions. For instance, to check that the length of the French greeting is not too long you could write a test like that shown in [Example A.9](#) (`lt` means less than).

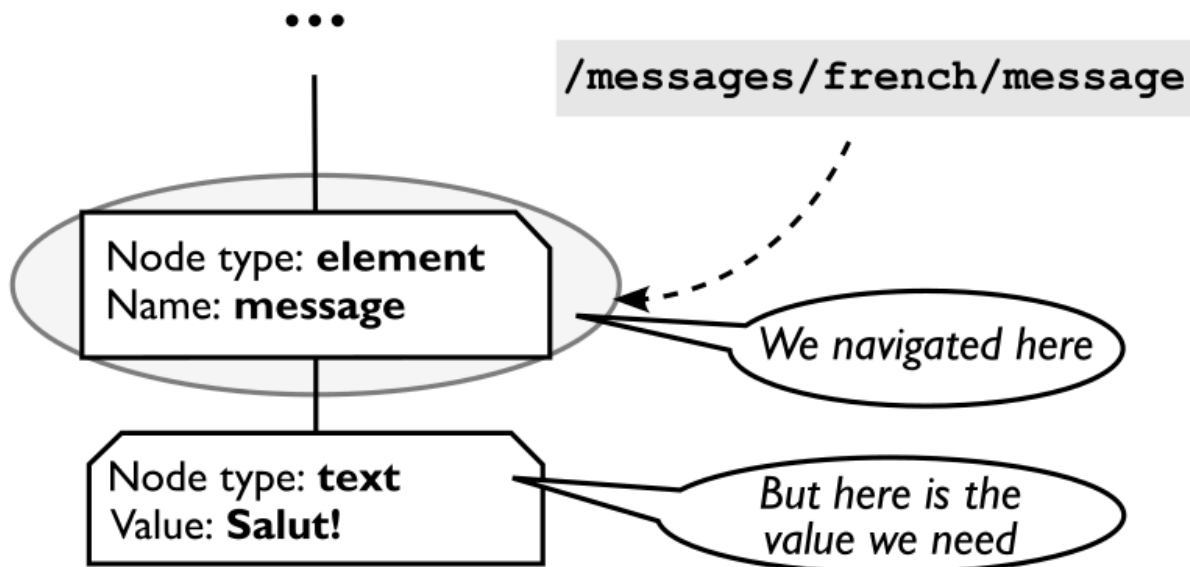
Example A.9 – Using an XPath tree navigation expression in Schematron

```
<assert test="string-length(/messages/french/message) lt 15">  
  French message too long  
</assert>
```

The value of elements

Eh, wait a minute, there seems to be something not quite right here! Our XPath expression `/messages/french/message` navigates to an element node. The code in [Example A.8](#) requests the value of this element node. But a node of type `element` does not have a `value` property. What you would think of as its “value” is in its children:

Figure A.8. The value of a node of type `element`?



The answer to this riddle: if you request the value of an element node, XPath looks down for descendant text nodes and uses their values. Underneath `/message/french/message` is a single text node `Salut!`, so the value of `/message/french/message` is also `Salut!`.

A (useful) gotcha here is that XPath does not look only at the direct children, **it looks all the way down in the tree. It searches for all descendant text nodes** and concatenates them. This is helpful when you're working with mixed text. For instance, the value of `<para>Hello <b>there</b>!</para>` is, as you would expect, the concatenation of all text nodes: `Hello there!` (the `<b>` element has disappeared). But it can also easily lead to mistakes like accidentally asking for the value of `/messages`, which is the, not very helpful, `Hallo daar!Hi there!Salut!`.

Since XPath looks at all descendant text nodes when it computes the value of an element node, this means that, in our example, the value of `/messages/french` is the same as the value of `/messages/french/message`. However, it is good practice to navigate as close as possible to the node you're interested in. This prevents problems if the document changes (for example, consider what would happen if you used `/messages/french` and the `<french>` element gets another child element with text).

A.2.2. Basic navigation to **attributes**

Now what if you want to retrieve an attribute value from our tree? For instance, how do you retrieve the value of the `color` attribute for the French language.

An attribute in XPath is addressed using the **@ sign**, followed by the name of the attribute. So to get the color of the French message you could write:

Example A.10 – An XPath tree navigation expression to select the `color` attribute of the French message

`/messages/french/@color`

A.2.3. Multiple selections: sequences

An XPath navigation expression can return more than a single node. In fact, an expression may return multiple nodes or no nodes at all. For instance, consider [Example A.11](#), which is a variation of [Example A.6](#), and you apply this XPath expression: `/messages/message`:

Example A.11 – An XML document with human-readable messages (variation of [Example A.6](#))

```
<messages>
  <message language="nl">Hallo daar!</message>
  <message language="en">Hi there!</message>
  <message language="fr">Salut!</message>
</messages>
```

Since XPath cannot guess which message you want, it addresses all three `<message>` elements in the document. If you use the original XPath expression, `/messages/french/message`, it addresses none, since `<messages>` has no child element named `<french>`.

That the answer to an expression can yield multiple (or no) results is actually an important XPath feature, called **sequences**. Every XPath expression always results in a sequence of answers, with none, one, or multiple results.

This is important because you can do lots of wonderful things with sequences. For instance: The XPath `count()` function returns the number of members in a sequence. So to check whether there are exactly three messages in [Example A.11](#) with Schematron, you could write (eq means equal to):

Example A.12 – Checking how many members a sequence has in Schematron

```
<assert test="count(/messages/message) eq 3">  
  There must be exactly three messages!  
</assert>
```

Sequences and expressions on simple data

Expressions on simple data (see [Section A.6, “Expressions on simple data”](#)), such as numbers and strings, also work with sequences. For basic operations, you may not notice this. For instance, `1 + 2` has the (single) answer `3`, and concatenating two strings, like in `'XP' || 'ath'` yields the single answer `'XPath'`.

However, working with sequences is a useful tool to have in your toolbox. For instance, in [Example A.12](#) suppose you want to check whether the value of a `language` attribute is one of Dutch, English, or French. You could express this in XPath as:

Example A.13 – An XPath expression that uses a sequence of strings in a comparison

```
@language = ('en', 'nl', 'fr')
```

The code `('nl', 'en', 'fr')` constructs a sequence of strings. The expression in [Example A.13](#) asks whether the value of a `language` attribute is a member of this sequence. For all `language` attributes in [Example A.12](#) this will return `true`.

A.3. Tree navigation and the context item

In the previous section we navigated the tree by starting at the top-level document node. Such a navigation expression always begins with the `/` character.

XPath also allows you to navigate the tree starting anywhere. Such a starting point within the tree is called the **context item**. Navigation expressions that start at the context item **do not begin with the `/` character**. This is a useful feature because it allows you to navigate the tree without having to know exactly where you are. Expressions become reusable.

The context item can be referenced or addressed **using the dot operator .** (see also [Section A.4.1, “The context item single dot . operator”](#)).

What determines the location of the context item in the tree? The answer to this lies not in XPath but in the language it’s embedded in. As explained at the beginning of this appendix, **XPath never stands on its own. It’s always used as part of some other programming language.** For instance, both **XSLT** and **Schematron** use XPath as the language for expressions and tree navigation. Each language has a different mechanism for determining the context item.

XSLT and the context item

XSLT’s main mechanism for setting the context item is called template matching. Here is a basic example:

Example A.14 – Example of setting and using the context item in XSLT

```
<xsl:template match="message">
  <p>
    <xsl:value-of select="."/>
  </p>
</xsl:template>
```

This piece of XSLT matches any `<message>` element in the document (because of the `match="message"` attribute). When your XSLT program encounters a `<message>` element, the code inside the `<xsl:template>` is executed. **This will produce a `<p>` element containing the value of the context item.** The context item itself is referenced with the dot character (`select="."`).

For instance, the result when the XSLT program encounters the French message is: `<p>Salut!</p>`

To keep [Example A.14](#) simple, I intentionally left-out the details of what the rest of the XSLT program looks like. XSLT template matching needs a careful program structure that is beyond the scope of this book.

In Schematron, the context item is set using the `context` attribute of **the `<rule>` element**. [Example A.15](#) is a basic example that checks whether all messages are less than 10 characters long (lt means less than).

Example A.15 – Basic usage of the context item in Schematron ([schematron-book-code/examples/technology-primer/check-messages-1.sch](#))

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>

    <rule context="message">
      <assert test="string-length(.) lt 10">
        The message <value-of select="."/> is too long!
      </assert>
    </rule>
```

```
</pattern>
</schema>
```

- The `<rule context="message">` triggers on every `<message>` element in the document. So when validating [Example A.6](#) it triggers three times.
- Every time it triggers, the `context="message"` attribute makes the `<message>` element it triggered on the context item.
- It then uses the context item dot operator to check the length of the message: `string-length(.) lt 10` (we'll get to strings and string-functions later on).
- It uses the context dot operator again to insert the value of the too long message into the result: `<value-of select=".">`.
- As a result, the Dutch message will be reported as offender: **The message Hallo daar! is too long!.**

The way Schematron works with context items shows their power. If the mechanism wasn't available, we would have to write a lot more code to make things happen and would be dependent on the structure of our input document. For example, consider [Example A.16](#).

Example A.16 – [Example A.15](#) implemented without using the context item

```
<?xml version="1.0" encoding="UTF-8"?>
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>

    <rule context="/messages/dutch/message">
      <assert test="string-length(/messages/dutch/message) lt 10">
        The message <value-of select="/messages/dutch/message"/> is too long!
      </assert>
    </rule>

    ... rule repeated for all languages in the input document ...

  </pattern>
</schema>
```

Think about what would happen to [Example A.16](#) as more languages are added over time: every time a new language is added, you need to add a new rule. This would quickly become an unmaintainable software nightmare!

A.4. Some special operators

Now that I've introduced the concepts of XPath navigation expressions, sequences and the context item, it's time to introduce a few additional common XPath operators.

A.4.1. The context item single dot . operator

Both [Example A.14](#) and [Example A.15](#) already used the single dot . operator. This operator always refers to the context item. That's all there is to it.

Referring to children of the context item

Starting at the context item you can of course refer to its children. For instance, assume the context item is the element `<message color="red">`, and you want to get the value of the `color` attribute. How can you do that?

One way to approach this is: "This context item element can be referred with a dot. We want to get to one of its children, an attribute. So I have to write a dot, a slash and then an @ followed by the name of the attribute": `./@color`. This is completely correct and it works.

However, XPath allows you to shorten expressions that start at the context item. Writing the starting `./` is optional. So in our example, `./@color` can be shortened to `@color`. This is also correct and it still works.

Not writing a leading `./` is very common. In the vast majority of cases it is omitted.

A.4.2. The parent double dot .. operator

The double dot .. operator refers to the parent node. For example, assume the context item is the `type` attribute of the following XML element:

Example A.17 – An XML element to illustrate the parent double dot .. operator on

```
<thing type="X23" code="123-456"/>
```

For this example, let's create a rule that says that a `type` element must start with the letter `c`. In the Schematron error message for this we want to display the value of the `code` attribute. The parent double dot .. operator allows us to do this, as shown in [Example A.18](#).

Example A.18 – Using the double dot .. operator to get the value of another attribute

```
<rule context="@type">
  <assert test="starts-with(., 'C')">
    Invalid type for code <value-of select="../@code"/>: <value-of select="."/>
  </assert>
</rule>
```

- The `<rule context="@type">` element sets the context item to the `type` attribute.

- The `<assert test="starts-with(., 'C')>` tests whether this starts with a capital c (in [Example A.17](#) it does not).
- The `<value-of select="../@code">` starts its tree traversal at the context item (the type attribute). So we could have written `../@code`, but as explained in [Section A.4.1, “The context item single dot . operator”](#), the leading `./` is superfluous.
- Then it goes up to its parent, the `<thing>` element, using the double dot `..` operator.
- From there it goes down again to the `code` attribute.

The result of applying [Example A.18](#) to [Example A.17](#) is the error message:

Invalid type for code 123-456: X23

A.4.3. The * and @* wildcard operators

The wildcard `*` operator refers to any child element. Since a wildcard can return more than one result, it is a typical application of XPath sequences. Let’s try this out on [Example A.6](#), which is repeated below in [Example A.19](#).

Example A.19 – [Example A.6](#) repeated. (`schematron-book-code/data/basic-messages.xml`)

```
<messages>
  <dutch color="red">
    <message>Hallo daar!</message>
  </dutch>
  <english color="yellow" default="true">
    <message>Hi there!</message>
  </english>
  <french color="purple">
    <message>Salut!</message>
  </french>
</messages>
```

- The XPath expression `/messages/*` results in a sequence of three results: the elements `<dutch>`, `<english>`, and `<french>`.
- To access the underlying `<message>` elements, you could write `/messages/*/message`. This gives you three `<message>` elements, one for each language.

Addressing the root element as /*

XML prescribes that a document has exactly one root element (in [Example A.19](#) this is `<messages>`). Being lazy, this means we could write the previous two examples as `/*/*` and `/*/message`.

This may feel like a misuse of the wildcard operator. There's only one root element and its name is known, so why not write it in full. Well, since programmers tend to be stingy with keystrokes, you'll often see XPath expressions start with `/*` where the root element is meant.

Another reason for using `/*` to address the root element is that some XML dialects have more than one root element with (roughly) the same type of contents. For instance, DocBook (a standard for writing texts) uses both `<book>` and `<article>` as root elements. Using `/*` let's you address either one.

As you might have guessed there's also a wildcard operator for attributes: `@*`. For example, the expression `/messages/english/@*` returns two attributes: `color="yellow"` and `default="true"`.

Wildcard operators are often used in conjunction with predicates. You can find more information about this in [Section A.5, “Predicates in tree navigation”](#).

A.4.4. The “search” `//` operator

Although it's not its official name, the `//` operator acts like a search. For instance, `//english` will return the `<english>` element, no matter where it is in the document. So even when [Example A.19](#) is embedded somewhere in a much larger XML structure, `//english` will return the right element.

That is to say... it works fine on [Example A.19](#), but for other documents it could just as well return none or multiple `<english>` elements. Be careful what you wish for and know your document structure before using it.

Some important details:

- Starting an XPath expression with `//` always starts searching at the very beginning, at the document node.
- To start your search from the context item, you must start your expression with `./`

For instance, if the `<english>` element is the context item, writing `./message` will return the underlying `<message>` element, even if, maybe in a future version of the document, it's no longer a direct child.

- You can also use the `//` operator in the middle of an XPath expression. For instance, to search for any `<english>` element that is not the root element, you could write `/messages//english` (or `/*//english`).
- It is also possible to search for attributes. As you might have guessed writing `//@` followed by the attribute's name does the trick.

A.5. Predicates in tree navigation

The last concept I want to introduce is *predicates*. Predicates are filters, applied during the tree traversal. To illustrate this, let's go back to [Example A.6](#), repeated here in [Example A.20](#).

Example A.20 – [Example A.6](#) repeated. (schematron-book-code/data/basic-messages.xml)

```
<messages>
  <dutch color="red">
    <message>Hallo daar!</message>
  </dutch>
  <english color="yellow" default="true">
    <message>Hi there!</message>
  </english>
  <french color="purple">
    <message>Salut!</message>
  </french>
</messages>
```

Assume we need the message in the default language. We don't know which one that is, but we do know that this language's element has the attribute `default="true"`. [Example A.21](#) is an XPath tree navigation expression that gives us the default language:

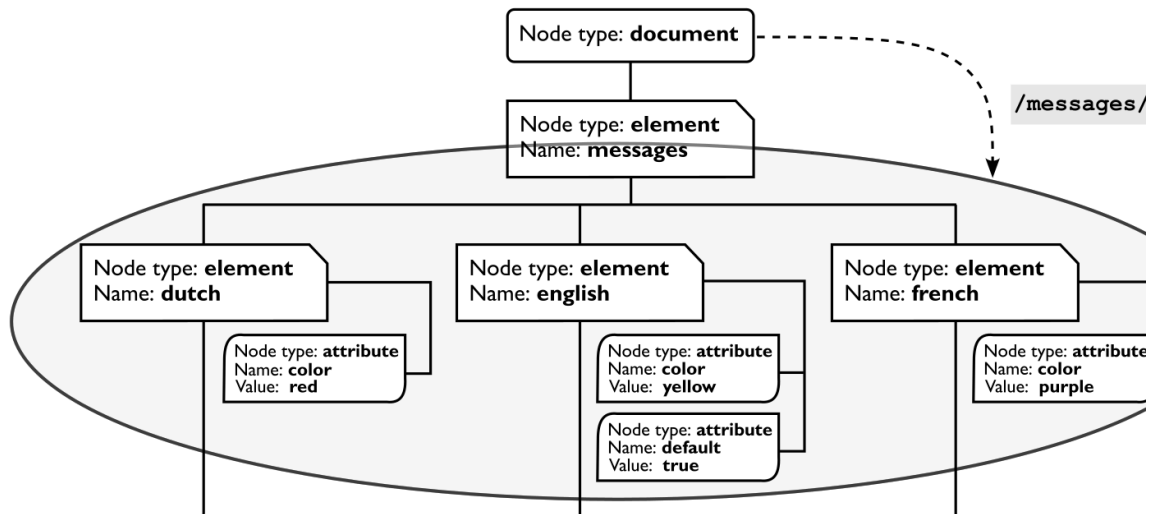
Example A.21 – Getting the default language in [Example A.20](#) using a predicate

```
/message/*[@default eq 'true']/message
```

The part between the square brackets, `[@default eq 'true']` is a predicate. The operator `eq` means equal to, so this predicate tests for an attribute named `default` with a value equal to `true`. Let's see what exactly is happening here:

- With what we already know about tree navigation expressions, the first part of [Example A.21](#), `/messages/*`, holds no surprises. It returns a sequence of three elements: `<dutch>`, `<english>` and `<french>`.

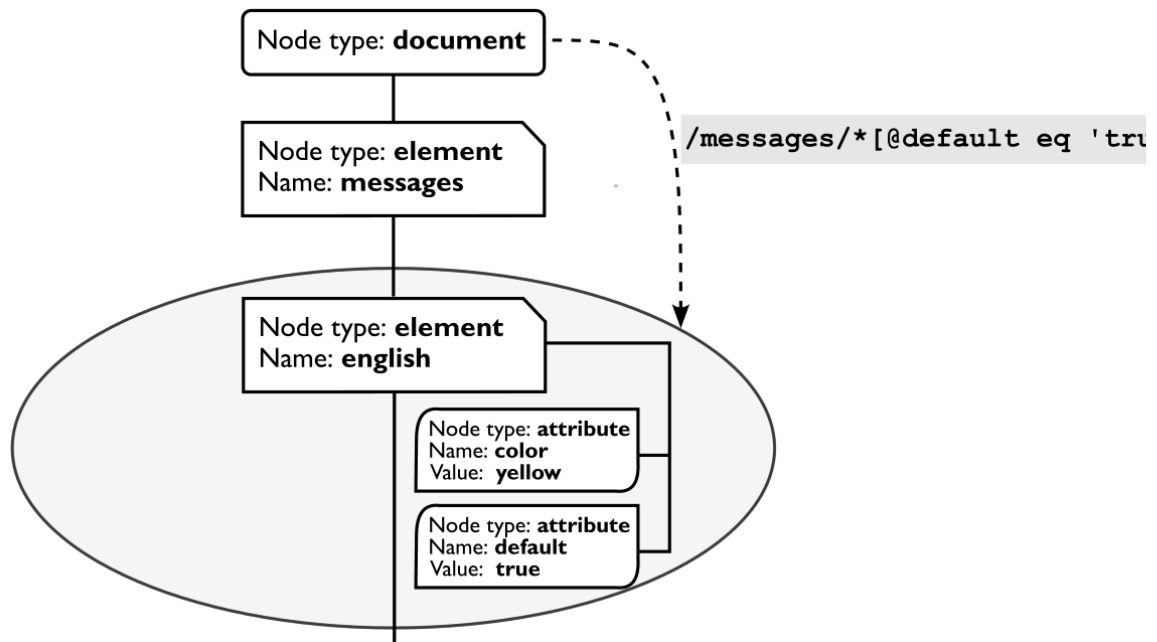
Figure A.9. `/*/messages` applied to the (partial) tree representation of [Example A.20](#)



- Then the predicate `[@default eq 'true']` kicks in. This expression is held against all members of the sequence:
 - For `<dutch>` and `<french>` the result of this expression is `false`: these elements have no attribute `default="true"`. Because of this, these elements are dismissed.
 - However, for the `<english>` element the predicate is `true`: this element does have an attribute `default="true"`. So this element is kept.

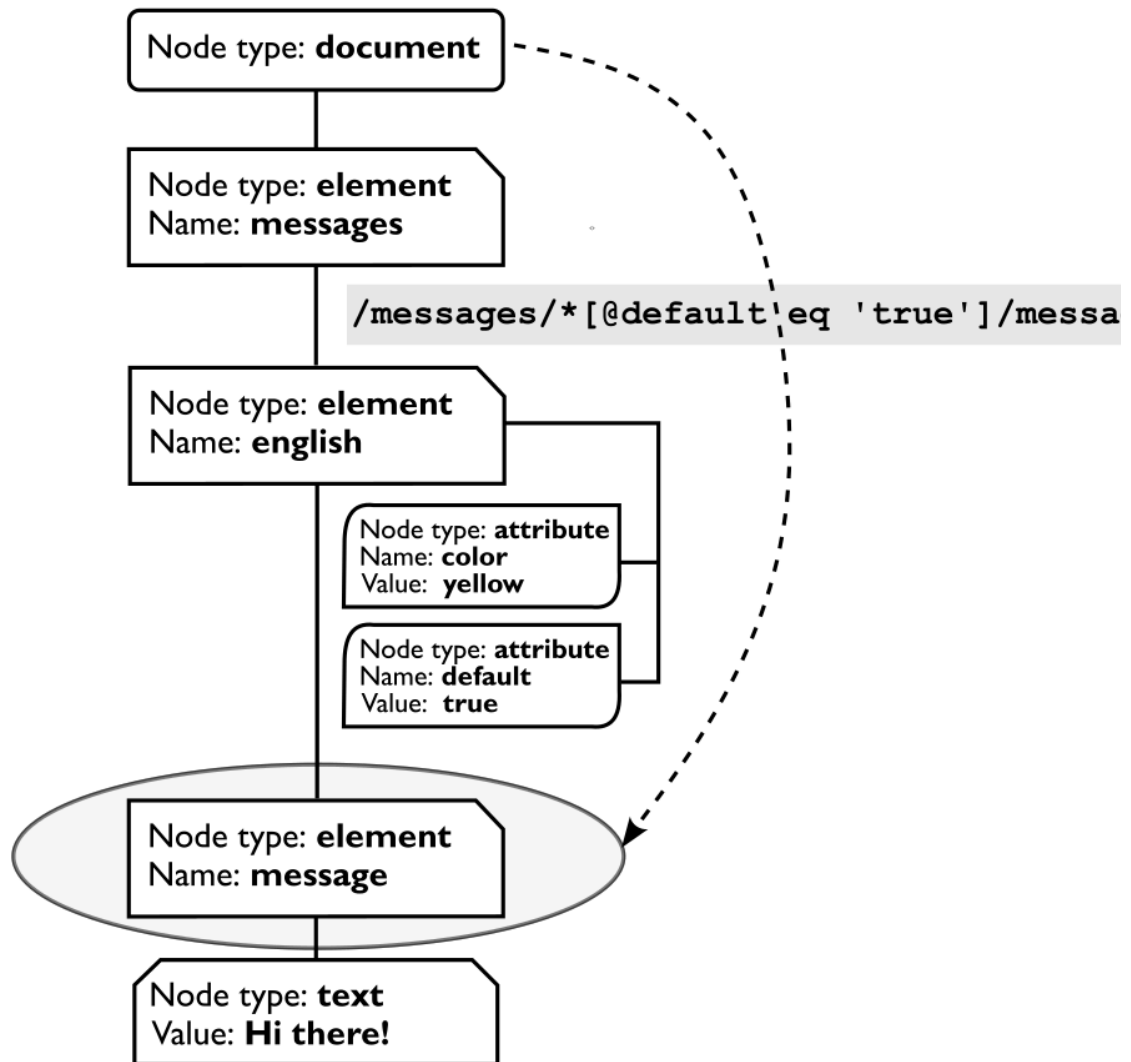
As a result, the sequence returned by `/messages/*` is reduced, only the `<english>` element remains. In other words: the predicate acted as a filter on what came before.

Figure A.10. `/*/messages[@default eq 'true']` applied to the (partial) tree representation of [Example A.20](#)



- The final part of the original XPath expression in [Example A.21](#), `/message` now only has to deal with this single remaining `<english>` element and returns the correct `<message>` element: `<message>Hi there!</message>`.

Figure A.11. `/*/messages[@default eq 'true']/message` applied to the tree representation of [Example A.20](#)



A useful mnemonic when thinking about predicates is the words “for which,” as in: Give me the elements `/messages/*` *for which* the expression `@default eq 'true'` holds.

Predicates and the context item

In [Section A.3, “Tree navigation and the context item”](#) I talked about the context item and how it allows us to write relative XPath navigation expressions (ones that do not start with a `/`).

Observant readers might already have noticed that this is also what we do in the predicate of [Example A.21](#): `@default` is a relative XPath navigation expression: it selects the `default` attribute of the current context item.

What happens here is that, when a predicate is evaluated, the context item is always the node the predicate is applied on. So for our example, the context item will first be the `<dutch>` element, then `<english>` and finally `<french>`.

A frequently used application of predicates is to get the n^{th} element of a sequence. Assume you want the first message in [Example A.20](#). This can be arranged as follows:

Example A.22 – Getting the first language in [Example A.20](#) using a predicate

```
/message/*[1]/message
```

The `[1]` predicate filters out all but the first element in the `/messages/*` sequence, leaving only the `<dutch>` element.

For more elaborate predicate filtering based on the position of nodes in a sequence you can use the functions `position()` and `last()`:

- The `position()` function returns the position of the node in the sequence. For the first node it returns 1, for the second 2, and so on.

So, for instance, to get a sequence with the first two messages, you can write (`le` means less than or equal to):

Example A.23 – Getting the first two messages in [Example A.20](#) using a predicate

```
/message/*[position() le 2]/message
```

Positional syntactic sugar

Strictly speaking, [Example A.22](#), which gets the first message, should be written as:

Example A.24 – Getting the first language in [Example A.20](#) using a predicate with the `position()` function

```
/message/*[position() eq 1]/message
```

The simpler predicate notation `[1]` is what is called “syntactic sugar”: an easier way of stating often occurring things. XPath silently rewrites this to `[position() eq 1]`. Using the shorter form shown in [Example A.22](#) is so common that most people probably don’t even realize its sugar level...

- The `last()` function returns the number of nodes in the sequence (which is also the position of the last one, therefore `last()`). So to get the last message

of [Example A.20](#):

Example A.25 – Getting the last message in [Example A.20](#) using a predicate with the `last()` function

```
/message/*[last()]/message
```

Or (waiving the syntactic sugar):

Example A.26 – Getting the last message in [Example A.20](#) using a predicate with the `position()` and `last()` functions

```
/message/*[position() eq last()]/message
```

Predicates are enormously useful in navigating XML trees and you'll see and use them a lot. For instance, the Schematron fragment in [Example A.27](#) uses the expression from [Example A.21](#) to get to the message in the default language. After this, it checks whether this message is too long (`lt` means less than).

Example A.27 – A Schematron rule that uses a predicate

```
<rule context="/messages/*[@default eq 'true']/message">
  <assert test="string-length(.) lt 25">
    The default message is too long!
  </assert>
</rule>
```

To wrap up, a few possible questions predicates might raise:

- What if a predicate filters out all nodes?

In that case the result of the predicate is an empty sequence and the result of the full XPath expression will also be empty. For instance, the following XPath expression tries to find a brown message, and since there's no such thing in [Example A.20](#), the result of the expression will be empty: `/message/*[@color eq 'brown']/message`

- Can you use predicates on predicates?

Absolutely. For instance, suppose you need a single red message but aren't sure there is only one in the XML. In that case you could decide that the first one will do. The following XPath expression first selects all red messages and then selects the first red message by applying another predicate:

```
/message/*[@color eq 'red'][1]/message
```

- How complex can predicates become?

Very. All tricks in the XPath book are allowed. For instance, it's entirely possible, and not uncommon, to use expressions with predicates inside a predicate. Because this is a primer, I won't go into any details, but the sky and your imagination are the limit.

A.6. Expressions on simple data

The previous sections of this technology primer explained how you can use XPath to navigate through an XML document. XPath tree navigation expressions are what most people think of as “doing XPath.”

However, XPath can also handle more mundane programming jobs like working with text, numerical computations, or making comparisons. So if you want to know how much $2 + 3$ is, the sine of 3.14, or the first three characters of a piece of text, XPath can compute it for you.

Once you get a little beyond the basics, you need at least some knowledge about this. Therefore, I end this technology primer with a (lengthy) discussion of working with data types like strings (text) and numbers. I also talk about performing comparisons (Boolean expressions).

At the end there's a section about working with dates, times, and durations (see [Section A.6.6, “Working with dates, times and durations”](#)). This is a bit more exotic, but Schematron schemas often deal with comparisons such as “is this date n days before that date,” etc. If this isn't applicable to your schemas you can skip this, but in case you need it, it's there.

A.6.1. The XPath function library

XPath has an extensive library of functions. Functions can compute all kinds of things for you, from the length of a piece of text, `string-length(...)`, to rounding a number to its nearest integer, `round(...)`. Although the coming sections will cover some of them, it's impossible to discuss them all here. There are simply too many.

Here are two web pages I recommend for more information:

- https://www.w3schools.com/xml/xsl_functions.asp provides an overview with simple examples. Unfortunately, this page is for a previous version of XPath (2.0), and it doesn't contain the extensions introduced in newer

versions (as of this writing, we're now at 3.1). Although incomplete, this page is a useful resource for the most commonly used functions.

- <https://www.w3.org/TR/xpath-functions-31/> covers XPath 3.1 and lists all of the functions. However, this is an official standards page, which contains a lot of information and uses formal language that might not make sense to a beginner. Please don't let this keep you from using this resource; it is a valuable source of information.

Both pages order the information by the kind of data the functions are working on. For example, you can find text (string) functions here:

https://www.w3schools.com/xml/xsl_functions.asp#string and here:

<https://www.w3.org/TR/xpath-functions-31/#string-functions>. If you're looking for something, go to the right section and look at the function names. Most likely that will be enough to help you find what you need.

What's the `fn:` doing in front of the function names?

Both information sources list the function names with an `fn:` prefix, for instance `fn:string-length`. What does this mean and why isn't this used in the examples throughout this book?

Let me reassure you first: you don't have to write the `fn:` prefix, it's entirely optional.

But then, what is it doing there? XPath functions must be in an XML namespace (namespaces are explained in [Appendix B](#)). For the standard XPath functions this namespace is

<http://www.w3.org/2005/xpath-functions>, usually denoted by the prefix `fn:`. But this is also the default namespace for functions, so using a namespace prefix on a standard function is not necessary.

A.6.2. XPath data types

To handle data correctly, computer languages attach a *data type* to a piece of data. Computers need to know whether something is, for instance, a number, string, boolean, or date and, if it's a number, whether it's an integer or decimal number. The collection of data types a particular computer language works with is called its *type system*.

XPath has a rather elaborate type system:

- It works with XML, so it contains types like documents, elements, and attributes. These types are called *nodes*. Working with nodes is what the previous sections about navigating the tree were all about.
- It can handle “simple” data like numbers, strings, and booleans. The XPath type system calls these *atomic types*. The basics of these will be covered in the coming sections.

- It also has two more advanced data types, maps and arrays, which I do not cover here.
-

Data types galore

XPath has an astonishing number of atomic data types (over 40): numbers (in several flavors), strings, booleans, dates, times, durations, and more. In addition, some of these types have subtypes.

For instance, integers have subtypes for various number ranges. Floating point data types (numbers with decimals) come in several precisions. Strings have subtypes for things like names and URIs. Even durations (stretches of time) have two variants.

Fortunately, in practice, it is usually not that complicated. You only need a few. In most cases you don't even have to be aware that things like data types play a role.

[Table A.1](#) describes the most common data types:

Table A.1 – XPath common data types

Name	Description	Example
xs:string	A string of characters, aka a piece of text.	'Schematron rocks!'
xs:integer	An integer number.	16 -267,801

Name	Description	Example
xs:double	A floating point number (a number with decimals). Use this for numbers on a continuous scale (for example, weights, distances, or temperatures). It's possible to use exponential notation, using the E character: 1E3 is 1,000 and -2E-4 is -0.0002. Due to the way an xs:double is stored in computer memory, small rounding errors can occur. If this is not acceptable use the less efficient but more precise xs:decimal (see below).	16.688999 -1.1E3
xs:decimal	A floating point number (a number with decimals) with accurate precision. Due to how computers work with floating point numbers like xs:double, rounding sometimes causes minor decimal deviations. This will not happen for the xs:decimal data type. Therefore, use this for numbers that represent discrete quantities, like sums of money.	16.23 1.00001
xs:date xs:time xs:dateTime	Dates, times, and the combination of these. See Section A.6.6, “Working with dates, times and durations”.	2022-01-01 12:01:34.897 2022-01-17T09:59:36
xs:dayTimeDuration	A duration or period (stretch of time), expressed in days, hours, minutes,	P3DT6M

Name	Description	Example
	seconds, and fractions of a second. See Section A.6.6, “Working with dates, times and durations” .	PT1M1.1S

What’s this `xs:` in front of the type names?

XPath borrows its type system from the XML Schema language. The XML Schema language defines an XML namespace `http://www.w3.org/2001/XMLSchema`. This namespace is, in most cases, referenced with the prefix `xs:`. Therefore, XPath (atomic) type names are usually written using an `xs:` prefix.

In practice it’s simple: If you need to explicitly refer to an XPath data type (see for instance [Section A.6.2.1, “Explicit data typing and data type conversions”](#)), use the `xs:` prefix, like in [Table A.1](#).

For Schematron that’s it. However, some XML programming languages, like XSLT, require that you define the XML Schema namespace with the right prefix explicitly. So if you get error messages about an undeclared `xs:` prefix, add the attribute `xmlns:xs="http://www.w3.org/2001/XMLSchema"` to the root element. For instance, for XSLT:

```
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
  xmlns:xs="http://www.w3.org/2001/XMLSchema" ...>
```

XML programming languages based on XPath differ in how they work with data types. For example, XSLT and XQuery allow you to specify data types explicitly when you declare a variable or parameter. Schematron, on the other hand, doesn’t. When you create a variable using the `<let>` element, its data type is automatically inferred from its value, which is usually OK but not always. The next section explains the why and how of more precise data typing.

A.6.2.1. Explicit data typing and data type conversions

There are situations where you need to be explicit about the data type you’re handling. For instance, assume an attribute has a numerical value:

Example A.28 – An attribute `count` with a numerical value

```
<things count="3">
```

Now assume we want to make sure there are exactly three things. A Schematron schema fragment that at first sight looks like it performs this check is:

Example A.29 – Comparing a numerical value in Schematron (incorrect)

```
<rule context="things">
  <assert test="@count eq 3">There must be 3 things!</assert>
</rule>
```

When you run this, an error message will pop-up saying: Cannot compare xs:untypedAtomic to xs:integer. XPath does not know what to make of the attribute's value and gives up. The solution is to tell XPath explicitly what type the attribute is supposed to be. This can easily be done: use the type's name as a function, as shown in [Example A.30](#).

Example A.30 – Comparing a numerical value in Schematron **([Example A.29](#) corrected)**

```
<rule context="things">
  <assert test="xs:integer(@count) eq 3">There must be 3 things!</assert>
</rule>
```

There's an additional benefit here: by telling XPath it must consider a value to be an integer, values that we humans think of as 3, like "03", "000003", or " 3", are all considered 3.

The only data type XPath can compare directly in situations like this is string (xs:string). So you could rewrite [Example A.30](#) as [Example A.31](#).

Example A.31 – Comparing an attribute's value as a string

```
<rule context="things">
  <assert test="@count eq '3'">There must be 3 things!</assert>
</rule>
```

This will work for [Example A.28](#), but it dismisses values written like "03" or " 3". Therefore, if you use a value other than a string for a comparison or calculation, explicitly type the value using the data type's name as a function (as in [Example A.30](#)).

For Schematron there is one other situation where you might want to use this “data type name as function” trick. That's when you need to be sure that a variable has the correct data type. Schematron does not have explicit variable typing. The only way to explicitly set a variable's data type is to initialize it with a correctly typed value in the first place.

For instance, assume you're doing something financial and notice that all over your schema you use the important tax threshold amount 1234.56. A good design change would be to put this value in a variable and reference the variable instead. Then, if the tax threshold ever changes, the only thing you need to do is change

the variable's value. You won't have to make the change all over the code, which would, of course, lead to you forgetting a crucial one.

Without explicit data typing, the declaration for this variable in Schematron looks like [Example A.32](#).

Example A.32 – Declaring an amount as a variable in Schematron

```
<let name="tax-threshold" value="1234.56"/>
```

This will work most of the time, but it has one problem: XPath will, by default, assume the data type of the variable is `xs:double`. And, as mentioned earlier (see the entry for `xs:decimal` in [Table A.1](#)), working with `xs:double` values for money can lead to minor decimal deviations, causing weird failing comparisons or seemingly unexplainable calculation results. Therefore, the data type of the `tax-threshold` variable should be `xs:decimal`, not `xs:double`. Fortunately, rewriting [Example A.32](#) to do this is simple ([Example A.33](#)).

Example A.33 – Declaring an amount as a variable with explicit data typing in Schematron

```
<let name="tax-threshold" value="xs:decimal(1234.56)"/>
```

To declare variables with data types `xs:date`, `xs:time`, `xs:dateTime`, or `xs:duration` you must use this construction. See [Section A.6.6, “Working with dates, times and durations”](#) for more information and examples.

A.6.3. Numerical expressions

XPath has numerical expressions, just like other computer languages. They usually work as expected: `2+3` yields 5 and `12*5.1+7.3` yields 68.5. However, here are a few things to know:

- The subtraction operator is the hyphen (-), as in most computer languages. But since a hyphen is a valid character in an XPath name (for instance, a variable, element, or attribute name), XPath doesn't always know what to do.

For example, `30-2` works and yields 28. However, if you have a variable `$a` with value 30, `$a-2` will not work. XPath assumes you mean a variable named `$a-2`, which, even if such a variable exists, is probably not what you intended. To solve this, surround the hyphen with spaces and write `$a - 2` instead. This makes your code unambiguous.

- The usual division operator in most computer languages is the slash character /. However, in XPath the slash character is already used as an operator in XPath navigation expressions (like `/messages/dutch/@color`). Therefore, XPath uses the word `div` for division. For instance, `4 div 2` is 2 and `10 div 4` is 2.5.

XPath also has an integer division operator, written as `idiv`. This will throw away all decimals and always return an integer result. So `10 idiv 4` is 2 and `10 idiv 6` is 1.

There are also functions that work on numerical data. For instance `round()` rounds a number to its nearest integer. Please refer to [Section A.6.1, “The XPath function library”](#) to learn how to find these functions.

A.6.4. String expressions

Strings are pieces of text, or sequences of characters if you prefer. As with most computer languages, a string constant is written between quotes, for example, `'XPath'` or `'John Doe'`. You can use both single or double quotes, so `"XPath"` and `"John Doe"` are also correct.

However, XPath expressions, including string constants, are very often written as attribute values, and attribute values must be surrounded by quotes. This requires you to use quotes within quotes. For instance, setting a Schematron variable to a constant string value looks like [Example A.34](#).

Example A.34 – A string constant as an XPath expression, using quotes within quotes.

```
<let name="firstname" value="'John'"/>
```

The double quotes in `value="'John'"` delineate the attribute value, the single quotes the string constant within.

Quoting the other way around

It's not very common, but you can use single quotes to delineate attribute values. With this, [Example A.34](#) becomes:

Example A.35 – Writing [Example A.34](#) with the quotes the other way around.

```
<let name='firstname' value='"John'"/>
```

To concatenate strings, use the “double pipe” operator `||`. For instance, assume you have two name variables, `$firstname` and `$lastname`, and you want to turn these into a full name in Schematron. [Example A.36](#) shows how to do this.

Example A.36 – Concatenating strings using the `||` operator.

```
<let name="fullname" value="$firstname || ' ' || $lastname"/>
```

Concatenating strings for older versions of XPath

The `||` operator was introduced in version 3.0 of XPath. If you’re using an older version, [Example A.36](#) will not fly. Use the `concat()` function instead ([Example A.37](#)).

Example A.37 – Concatenating strings using the `concat()` function.

```
<let name="fullname" value="concat($firstname, ' ', $lastname)"/>
```

There are a lot of standard functions for working with strings (see [Section A.6.1](#), “[The XPath function library](#)”). Here are a few noteworthy examples:

- The `substring()` function returns part of a string: `substring('abcdefg', 3, 2)` returns `'cd'`. The second argument, 3, is the starting position and the third argument, 2, the length of the required substring. If you omit the third argument you get the remainder of the string, so `substring('abcdefg', 3)` returns `'cdefg'`.

Note

Programmers, be aware that, unlike languages such as C and Java, indexing in XPath always starts at 1, not at 0 (everywhere, not only here). So to get the first character of a string write `substring(..., 1, 1)`, NOT `substring(..., 0, 1)`.

- The `string-length()` function returns a string’s length. So `string-length('abc')` returns 3.
- the `upper-case()` and `lower-case()` functions convert the case of a string. For example, `upper-case('XPath')` returns `'XPATH'` and `lower-case('XPath')` returns `'xpath'`.
- The `normalize-space()` function, which is remarkably useful, takes a string and “normalizes” the whitespace characters (spaces, tabs, newlines, etc.). It removes all leading and trailing whitespace and replaces all sequences of whitespace characters inside the string with a single space. For example, `normalize-space(' XPath 3.0 ')` returns `'XPath 3.0'`.

One reason this function is so useful is that it can make your code resilient against inadvertent but common typos. If you accidentally type `<code>XYZ6</code>` instead of `<code>XYZ6</code>`, a comparison of this code value to 'XYZ6' will fail. Doing a `normalize-space()` on the code value makes comparisons more robust: `normalize-space(code) eq 'XYZ6'` will work whether or not there are additional leading or trailing spaces.

`normalize-space()` is also often handy to correct problems caused by the pretty-printing feature of XML editors. The pretty-print feature can break text into multiple lines, as shown in [Example A.38](#).

Example A.38 – A long value broken into multiple lines by the editor's pretty-printing feature.

```
<para>Something typed on a single line but broken  
  into multiple lines by the pretty-printing feature of  
  the editor.</para>
```

Performing a `normalize-space()` on the value of the `<para>` element in [Example A.38](#) removes all of this additional whitespace.

Of course, be careful applying `normalize-space()`. Sometimes whitespace is important!

Regular expressions

XPath also contains functions for working with regular expressions, `regexps` for short. Regexp are strings that perform almost magical pattern matching on other strings. It's a complex technology, too complicated to explain here, but I do want to mention it in case you know what I'm talking about and need the features. For the curious there's an excellent Wikipedia page: https://en.wikipedia.org/wiki/Regular_expression.

Let's keep it to a simple example. The regular expression `'[^0-9]'` matches all characters in a string that are not numbers. Replacing all non-numbers with hyphens, can be done with the XPath `replace()` function: `replace('88A|B99', '[^0-9]', '-')` yields `'88---99'`.

A.6.5. Boolean expressions and comparisons

Boolean expressions are all about making decisions in your code. As an example, let's look back at the `<assert>` element in [Example A.15](#), repeated in [Example A.39](#).

Example A.39 – A Schematron assert using a boolean expression

```
<assert test="string-length(.) lt 10">  
  The message <value-of select="."/> is too long!  
</assert>
```

The expression in the test attribute of the <assert> element, `string-length(.) lt 10`, tests whether the string length of the context item is less than 10 characters. The result is a boolean expression: either `true` or `false`.

Boolean expressions are commonly used to compare two values, as shown in [Example A.39](#). The value comparison operators in XPath are shown in [Table A.2](#).

Table A.2 – XPath value comparison operators

Operator	Meaning	Example
eq	Equal	<code>\$code eq 'XQZ56'</code>
ne	Not equal	<code>\$code ne 'XQZ56'</code>
lt	Less than	<code>string-length(\$code) lt 12</code>
le	Less than or equal to	<code>\$number-of-references le 100</code>
gt	Greater than	<code>\$author-count gt 0</code>
ge	Greater than or equal to	<code>count(//message) ge 3</code>

Where are > and the <?

If you have used other languages, you might expect comparison operators to be written using symbols: `=` for equal and `>` for greater than, etc. And if you try this in XPath expressions, it will work.... What's happening here?

XPath has a second set of comparison operators for so-called “general comparisons.” These are `=`, `!=`, `>`, `>=`, `<` and `<=`. General comparisons are considerably more powerful than the value expressions in [Table A.2](#). They can, for instance, compare sequences (see [Section A.2.3, “Multiple selections: sequences”](#)), while value comparisons can only compare single values.

That sounds like an advantage, but it has downsides, including the following:

- Comparing values using general comparisons is more expensive (takes more CPU cycles) than using the simpler value expressions.
- Writing (and reading) them can be awkward, because `<` is a reserved character in XML and must be written as `<`. For example, `count(//messages) <= 3`. Just typing/reading `le` instead of `<=` is much easier.

Strictly speaking, the > character does not need to be written as > . However, this is often done also.

- The exact rules for general comparisons are complicated and can lead to situations where they don't provide the answer you expect. There are even circumstances where `a = b` is `true` and `a != b` is also `true`.... Therefore, to negate the outcome of `a = b`, always use `not(a = b)` and `not a != b`.

So, as general guidance, use the value comparisons from [Table A.2](#) when comparing values, and use general comparisons only when sequences come into play.

If you want to know more about value versus general comparisons, I recommend reading the sections about comparisons in chapter 8 of the book *XSLT 2.0 and XPath 2.0: Programmer's Reference* by Michael Kay (Wiley, 2008). A more concise and technical description can be found in the [XPath 3.1 standard](#).

Here are some additional things to know about comparisons and boolean expressions:

- You can combine boolean expressions with the operators `and` and `or`. [Example A.40](#) tests whether some number (stored in the variable `$count`) is between 3 and 10:

Example A.40 – Testing whether some number is in between 3 and 10

```
$count ge 3 and $count le 10
```

- To negate the value of a boolean expression (`true` becomes `false` and `false` becomes `true`), use the `not()` function. For instance, to negate the expression of [Example A.40](#):

Example A.41 – Testing whether some number is not in between 3 and 10

```
not($count ge 3 and $count le 10)
```

- Boolean constants must also be written as functions. So it's not `true` but `true()` and not `false` but `false()`. For instance, to set a variable that determines whether some debug tests are enabled:

Example A.42 – Setting a variable to a fixed boolean value

```
<let name="enable-debug-tests" value="true()"/>
```

A.6.5.1. Testing sequences

As explained in [Section A.2.3, “Multiple selections: sequences”](#), every result in XPath is a sequence, with none, one, or many values. It’s possible to perform tests on sequences directly: an empty sequence results in `false`, a non-empty one in `true`. For instance, [Example A.43](#) checks whether there are any `<message>` elements in a document.

Example A.43 – Using a sequence directly to test for `<message>` elements

```
<rule context="/">
  <assert test="//message">There are no messages</assert>
</rule>
```

If there are any `<message>` elements in the document, the expression `//message` returns a sequence with one or more results. This then results in the value `true`. If there are no `<message>` elements, the expression returns the empty sequence, resulting in `false`.

This feature is useful in many ways, including the following:

- To test whether a particular attribute is there:

Example A.44 – Testing for the presence of an attribute

```
<rule context="thing">
  <assert test="@code">The code attribute is missing</assert>
</rule>
```

- To test whether a particular attribute is not there:

Example A.45 – Testing for the absence of an attribute

```
<rule context="order">
  <assert test="not(@tax-percentage)">Tax information not permitted</assert>
</rule>
```

- To test whether a certain child element is there:

Example A.46 – Testing for the presence of a child element

```
<rule context="person">
  <assert test="address">Address information missing</assert>
</rule>
```

The `exists()` and `empty()` functions

XPath also contains two functions for testing sequences: `exists()` and `empty()`. As you would expect, `exists(...)` returns `true` when its argument is a non-empty sequence and `empty(...)` returns `true` when its argument is an empty sequence.

These functions are mainly present for cosmetic reasons: to make code more readable. For instance, [Example A.47](#) rewrites [Example A.44](#) using the `exists()` function.

Example A.47 – Testing for the presence of an attribute using `exists()`

```
<rule context="thing">
  <assert test="exists(@code)">The code attribute is missing</assert>
</rule>
```

And [Example A.45](#) using the `empty()` function:

Example A.48 – Testing for the absence of an attribute using `empty()`

```
<rule context="order">
  <assert test="empty(@tax-percentage)">Tax information not permitted</assert>
</rule>
```

Whether you want to use these functions or not is up to you; they're cosmetic. But they can make your intention clearer. Judge for yourself.

Besides testing for being empty or not, you can also check the number of values in a sequence. The `count()` function does just that. [Example A.49](#) rewrites [Example A.43](#) to check whether there are more than three `<message>` elements.

Example A.49 – Testing for more than three message elements

```
<rule context="/">
  <assert test="count(//message) gt 3">There are not enough messages</assert>
</rule>
```

You could also rewrite [Example A.44](#) using the `count()` function as [Example A.50](#) shows.

Example A.50 – Testing for the presence of an attribute using the `count()` function

```
<rule context="thing">
  <assert test="count(@code) ne 0">The code attribute is missing</assert>
</rule>
```

A.6.5.2. Testing string values

Testing string values is very common. For instance, [Example A.51](#) checks whether a code attribute contains the right value.

Example A.51 – Testing the string value of an attribute

```
<rule context="thing">
  <assert test="@code eq 'AB123'">The thing code is incorrect</assert>
</rule>
```

XPath string comparisons are case-sensitive. So 'XPath' is not the same as 'xpath'. And if the value of the `code` attribute in [Example A.51](#) is `code="ab123"`, the assertion will fail.

The cheap and easy way to make string value comparisons case-insensitive is to use `upper-case()` or `lower-case()`. These functions convert a string to upper or lower case. [Example A.52](#) modifies [Example A.51](#) to use `upper-case()`.

Example A.52 – Testing the string value of an attribute, case-insensitive

```
<rule context="thing">
  <assert test="upper-case(@code) eq 'AB123'">The thing code is incorrect</assert>
</rule>
```

Collations

For more fine-grained control on string value comparisons, you can use *collations*. Using the right collation you can, for instance, test independent of diacritics, so 'ü' tests equal to 'u' and 'ç' equal to 'c'. There's also a collation for case-insensitive compares, which would make using the `upper-case()` or `lower-case()` function for comparisons no longer necessary.

Working with collations is an advanced subject that I do not cover here. For more information see: <https://www.w3.org/TR/xpath-functions-31/#collations>.

There are several useful functions for doing partial string value comparisons:

- The `starts-with()` and `ends-with()` functions check whether a string starts or ends with a certain substring. [Example A.53](#) checks whether the value of `@code` ends with 'CQ':

Example A.53 – Testing the end of a string value

```
<rule context="thing">
  <assert test="ends-with(@code, 'CQ')">Codes must end with CQ</assert>
</rule>
```

- The `contains()` function checks for the presence of a string somewhere in another string. [Example A.54](#) checks whether a description contains the word product.

Example A.54 – Testing for a string containing another string

```
<rule context="description">
  <assert test="contains(., 'product')">
    Descriptions must contain the word product
  </assert>
</rule>
```

```
</assert>
</rule>
```

These comparisons are also case-sensitive. The assert in [Example A.54](#) will fail if you type `Product` instead of `product`. [Example A.55](#) makes the comparison in [Example A.54](#) case-insensitive.

Example A.55 – Testing for a string containing another string, case-insensitive

```
<rule context="description">
  <assert test="contains(lower-case(.), 'product')">
    Descriptions must contain the word product
  </assert>
</rule>
```

Comparing strings for greater than or less than

You can also compare strings using the `gt`, `ge`, `lt`, and `le` comparison operators. These comparisons are made based on the position of the individual characters in the *character set*, which is usually Unicode.

For simple strings this is straightforward: 'A' is less than 'Z', '9' is greater than '0', 'a' is greater than 'A', etc. This is exactly what you would expect, as long as you are comparing strings that contain simple characters (western languages, non-accented, non-symbols, etc.).

However, when your strings get more complicated, value comparisons might not behave so well. So be careful and test thoroughly if you need this feature. It might even be necessary to use a specific collation; see [Collations](#) for more about this feature.

A.6.6. Working with dates, times and durations

Schematron schemas often compare dates, times, and durations. For example, “this date must be more than 5 days before that date.” XPath is great for working with these data types.

Let’s start with how date, time, and duration values must officially be written. XPath uses the international standard ISO 8601 for this. [\[27\]](#)

Here are some highlights:

- A date must be written as `YYYY-MM-DD`, so use `2022-01-01` for January 1st 2022.
- A time must be written as `hh:mm:ss` using the 24 hour clock. So use `13:15:03` for fifteen minutes and three seconds past one in the afternoon.

- If you need to be more precise you can use a decimal fraction with the seconds, for instance 13:15:03.125, which means fifteen minutes, three seconds and 125 milliseconds past one in the afternoon.
- A time can be followed by a *time zone designator*. For instance, for my location (The Netherlands) a time including a time zone designator is 11:24:29+01:00. For us it's one hour later than the "Universal Time Coordinated" (UTC), also known as "Greenwich Mean Time" (GMT), therefore the +01:00.

As you can imagine, in widely distributed XML messages, timezones can be crucial. One o'clock in the afternoon in New York (13:00:00-05:00) is 6 hours behind one o'clock in The Netherlands (13:00:00+01:00). If these timestamps are compared they should not come out as equal; they are 6 hours apart. If your timezone designators are correct, XPath will take care of this.

- To combine date and time values, separate the two values with a capital τ , for instance 2022-02-02T13:15:03.
- According to ISO 8601, a duration is specified as P3Y6M4DT12H30M5S, which represents three years, six months, four days, twelve hours, thirty minutes, and five seconds. A duration always starts with a capital P . You can leave out parts when zero. For instance, a duration of five days is simply P5D and five minutes PT5M (note that you still need the τ separator).

Durations can also be negative, for instance -PT3H for minus three hours.

XPath has many data types that you can use to work with dates, times, and durations. The major ones are listed in [Table A.3](#). There are several others, like `xs:gYear` for a year and `xs:gMonthDay` for a month+day combination, that I don't cover in this book.

Table A.3 – XPath date, time and duration data types

Name	Description	Example
<code>xs:date</code>	A date.	2022-01-01
<code>xs:time</code>	A time.	13:15:00 13:15:00.125

Name	Description	Example
		13:15:00.125-05:00
xs:dateTime	A date and time combination.	2022-01-01T13:15:00 2022-01-01T13:15:00.125 2022-01-01T13:15:00.125-05:00
xs:dayTimeDuration	A duration.	P3Y6M4DT12H30M5S P1D PT1H3M

In calculations, these data types do exactly what you expect:

- Subtracting two dates/times results in a duration. For example, 2022-01-01T13:15:00 minus 2022-01-01T01:00:00 results in a duration of PT12H15M.
- Adding a duration to a date/time, or subtracting it, shifts this into the future or past. For example, 2022-01-01T13:15:00 minus PT5M becomes 2022-01-01T13:10:00.

But how does XPath know it's dealing with this type of data? How does it know it needs to treat the value of some element, attribute, or variable as a date, time, date+time, or duration? XPath doesn't attempt to guess; you need to be explicit.

[Section A.6.2.1, “Explicit data typing and data type conversions”](#) describes how to use the type name as a function to tell XPath the type of some data. This trick also works here.

Let's illustrate this with some examples.

- Assume your data contains, somewhere, an element about a payment, like [Example A.56](#).

Example A.56 – An element with date/time information

```
<payment timestamp="2022-02-01:23:11:89" due-timestamp="2022-01-31:12:00:00"/>
```

In a Schematron schema you want to check whether the payment registered in [Example A.56](#) was on time (it's not). This could be done as shown in [Example A.56](#).

Example A.57 – Schematron rule to check whether [Example A.56](#) payment is on time

```
<rule context="payment">
  <assert test="xs:dateTime(@timestamp) le xs:dateTime(@due-timestamp)">
    Payment too late
  </assert>
</rule>
```

The assertion's test explicitly tells XPath to treat the values of the `timestamp` and `due-timestamp` attributes as type `xs:dateTime` so the compare will be correct.

- Now assume your payment data is formatted differently. It has an order date and a payment term expressed in days.

Example A.58 – An element with date/time and payment term expressed in days

```
<payment timestamp="2022-02-01:23:11:89" order-timestamp="2022-02-11:12:00:00"
  payment-term="5"/>
```

To check whether the payment was on time you need to use a duration. However, the payment term is expressed as the number of days, 5, not as a duration, P5D.

Here is how to handle this:

1. Convert the number of days into a string that is a valid duration: 'P' || @payment-term || 'D'. For [Example A.58](#) this will result in 'P5D'.
2. Convert this into a duration using `xs:dayTimeDuration(...)`
3. Use this value in the assertion's test. [Example A.59](#) shows the end result.

Example A.59 – Schematron fragment that checks whether the payment of [Example A.58](#) was on time

```
<rule context="payment">
  <assert test="xs:dateTime(@timestamp) le
    xs:dateTime(@order-timestamp) + xs:dayTimeDuration('P' ||
      @payment-term || 'D')">
    Payment too late
  </assert>
</rule>
```

- As a final example, assume the payment term is a fixed 30 days. To make our schema more maintainable, we want to record this duration in a global variable. To tell XPath that this variable is a duration, we have to be explicit about it:

Example A.60 – Creating a variable with a duration data type

```
<let name="payment-term" value="xs:dayTimeDuration('P30D')"/>
```

[Example A.61](#) shows how to use this variable.

Example A.61 – Schematron fragment that uses the variable declared in [Example A.60](#) to check whether a payment was on time

```
<rule context="payment">
  <assert test="xs:dateTime(@timestamp) le
    xs:dateTime(@order-timestamp) + $payment-term">
    Payment too late
  </assert>
</rule>
```

A.6.6.1. Additional date, time, and duration functions

There are several additional functions for dates, times, and durations. Here are the most common ones. [Section A.6.1, “The XPath function library”](#) has details on how to find them all.

- The current date, time, or date+time can be retrieved with the functions `xs:current-date()`, `xs:current-time()` and `xs:current-dateTime()`. These functions have no parameters and return a value with the appropriate data type.

The times are not a-changing

Probably you'll never notice, but during execution the values returned by these functions never change. So even if you have a lengthy validation that takes minutes or more, these functions return exactly the same value at the beginning as at the end: the day and/or time the execution started.

- Converting a date or time into text can be done with the functions `xs:format-date()`, `xs:format-time()`, and `xs:format-dateTime()`. These functions use a format string to specify what the date/time must look like. For instance, in The Netherlands we usually write dates as day-month-year. To format the current date in this format, you can use this:

```
format-date(current-date(), '[D]-[M]-[Y]')
```

On February 1st 2022 the result would be 1-2-2022. The string that formats the output has many options. For instance you can format a month as a decimal number (2), Roman numeral (II), name (February), abbreviation (Feb), and several others. See <https://www.w3.org/TR/xpath-functions-31/#rules-for-datetime-formatting> for a full list of options and examples.

[26] You can find descriptions and examples of mixed content in [Section A.1.3, “Representing mixed content”](#).

[27] The Wikipedia article https://en.wikipedia.org/wiki/ISO_8601 has a good description of this standard.

Appendix B. An introduction to namespaces

XML namespaces are rather strange. In my humble opinion they're a great invention because they help keep XML dialects unambiguously apart. But their implementation leaves something to be desired, most importantly in comprehensibility. By that I mean not only comprehensibility for beginners but also comprehensibility when interpreting XML documents that use namespaces. Getting the namespace information you need can be like solving a puzzle.

However, don't let this discourage you. XML namespaces are here to stay, and even when you are writing a simple Schematron schema, you will run in to at least one: the Schematron namespace. So, let's take the bull by the horns.

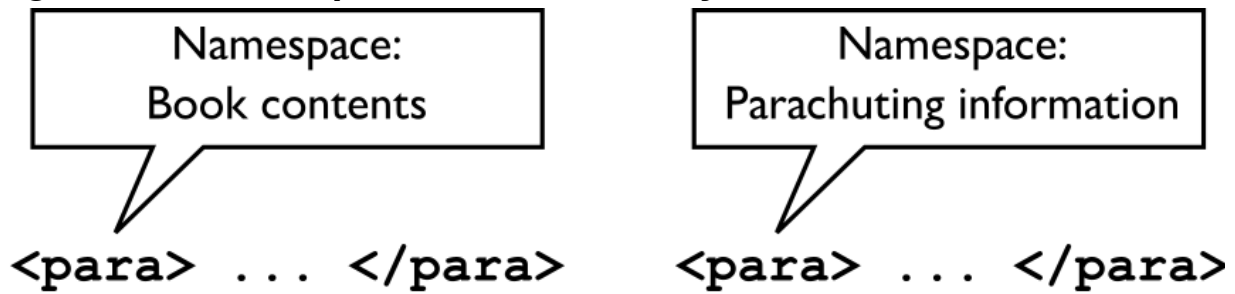
B.1. Why namespaces?

The main reason for namespaces is to provide uniquely named elements and attributes for XML documents. A namespace is a collection of elements and attributes that, together, form a *vocabulary*.

Assume you have an XML document that contains a `<para>` element. When this document contains book contents, `<para>` probably means a paragraph. But what if this is an XML document about parachuting? It could mean something completely different.

Namespaces attach a not-directly-visible label to elements and attributes: a label that says "this is a piece of text" on the book `<para>` element and a label that says something different on the parachuting `<para>` element. What's on this label is the namespace name. What these names are and how to attach them is the subject of this chapter.

Figure B.1. Namespaces as not-directly-visible labels



An application may be able to find out the meaning of a `<para>` element using the context. Book XML documents would probably start with a different root element than parachuting information, for instance `<book>` versus `<parachuting-info>`. And the application that processes the XML may assume it will receive XML in the right vocabulary.

However, sometimes it is difficult to determine the XML vocabulary of a document. For example, Schematron schemas start with a `<schema>` element, but so do W3C XML Schemas. They are two completely different languages with the same root element. And given that they both have to do with validation, it's not inconceivable to mix them up. A Schematron or XML Schema processor must know the schema it is working with. The processor can determine this by looking at the namespace.

Most XML standards use namespaces to define a vocabulary. For instance, DocBook is a standard for writing book content (used for the book you're reading now). DocBook contents must be in the DocBook namespace to be considered DocBook. Exactly the same XML in some other namespace wouldn't be DocBook.

Another reason for namespaces is to combine different XML vocabularies. For example, SVG (Scalable Vector Graphics) is an XML vocabulary for describing drawings. HTML is a completely different vocabulary used for web pages. Sometimes you want an SVG drawing on your web page. Using namespaces you can safely embed the SVG XML somewhere in the HTML. Because of the namespace labeling, the browser, or any XML-aware processor, can tell the vocabularies apart.

Bottom line: Namespaces add not-directly-visible labels, which contain the namespace name, to XML elements and attributes. These labels

unambiguously tell you that a particular XML document, or part of a document, is in a certain vocabulary. The vocabulary determines the goal, the meaning of the XML.

B.2. Namespace names

Namespaces have a name. This name is what sets one namespace apart from the other, and therefore, it must be unambiguous. To achieve this, the convention is to base namespace names on the web domain owned by the organization that issues them and format them as a URL (Uniform Resource Locator, the thing you see in the address bar of your browser).^[28] Domain names must be claimed, and the organization behind them makes sure they are unique. Therefore, you can be reasonably sure that a namespace name based on a domain that you own will not be used/invented/issued by somebody else. Here are some examples of namespaces:

- The World Wide Web consortium (W3C) publishes the XHTML standard, which is used for describing the contents of web pages in XML. The namespace name for XHTML is: `http://www.w3.org/1999/xhtml`.
- The DocBook standard is widely used for book contents, including this book. The namespace name for DocBook is: `http://docbook.org/ns/docbook`.
- When you've registered a domain for your company, it's safe to use it to define your own namespace names. For instance: `http://my-own-company.com/ns/xml-vocab-1`.

Be aware that the convention of using a domain name you own for namespace names is just that, a convention. Nothing stops you from inventing something completely different. However, as soon as your XML becomes public and is used outside your organization, I strongly advise you to follow along. And why not stick to this convention anyway, even for non-public XML?

Local namespace names

One exception I usually make for the “use the organization domain for namespace names” convention is when it really doesn't matter.

For example: XPath functions must be in some namespace. Self-defined functions in a program (for instance an XSLT stylesheet) are usually just local and cannot be used outside of it. You can go through the hassle of creating a full URL-based namespace name, but that's

hardly worthwhile. In these cases I simply use the namespace name `#local` or `#functions`. For an example of this in Schematron see [Section 7.2.2, “Using XSLT functions”](#).

Newcomers to the XML (namespace) world often find this convention extremely confusing. The name looks like a URL, but it isn't one? But it even has the `http` or `https` protocol in front. Despite that, there's no guarantee whatsoever that using it in a browser will lead you to a web page. Sometimes it does, sometimes it doesn't. There's no other choice than to get used to it: namespace names look like URLs of web pages, but they're not.

XML elements and attributes that do not use a namespace are in the so-called *null namespace*. They have no label (or an empty label) attached. This happens automatically if you don't use a namespace declaration, as explained in the next section.

QNames

You might come across the term QName (Qualified Name) when working with namespaces. A QName is a data type with two components:

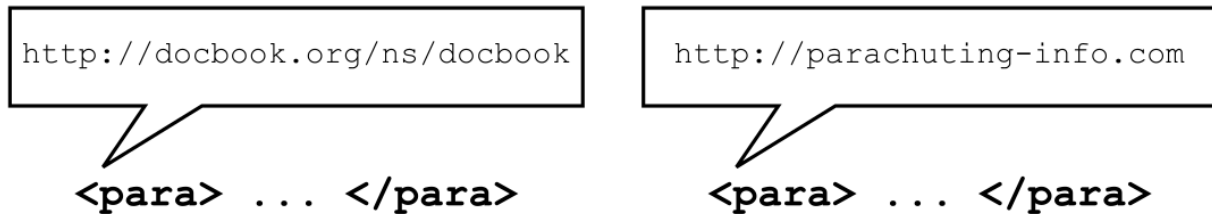
- The namespace of the element or attribute. For the null namespace this is empty.
- The local name of the element or attribute. This is what you would normally consider to be its name. For instance the local name of a `<para>` element is always `para`, regardless of the namespace.

XPath has several functions for working with QNames. See [Section A.6.1, “The XPath function library”](#) for more information.

B.3. Declaring namespaces

We now know that a namespace defines an XML vocabulary and attaches a not-directly-visible label, the namespace name, to elements and attributes. We also know that namespace names are formatted as URLs. With this knowledge we can redraw [Figure B.1](#) as [Figure B.2](#).

Figure B.2. Namespaces names, using the URL convention, as not directly visible labels



What we don't know yet is how to attach labels to declare that some element or attribute is in a specific namespace. You can do this by: defining a default namespace or using a namespace prefix.

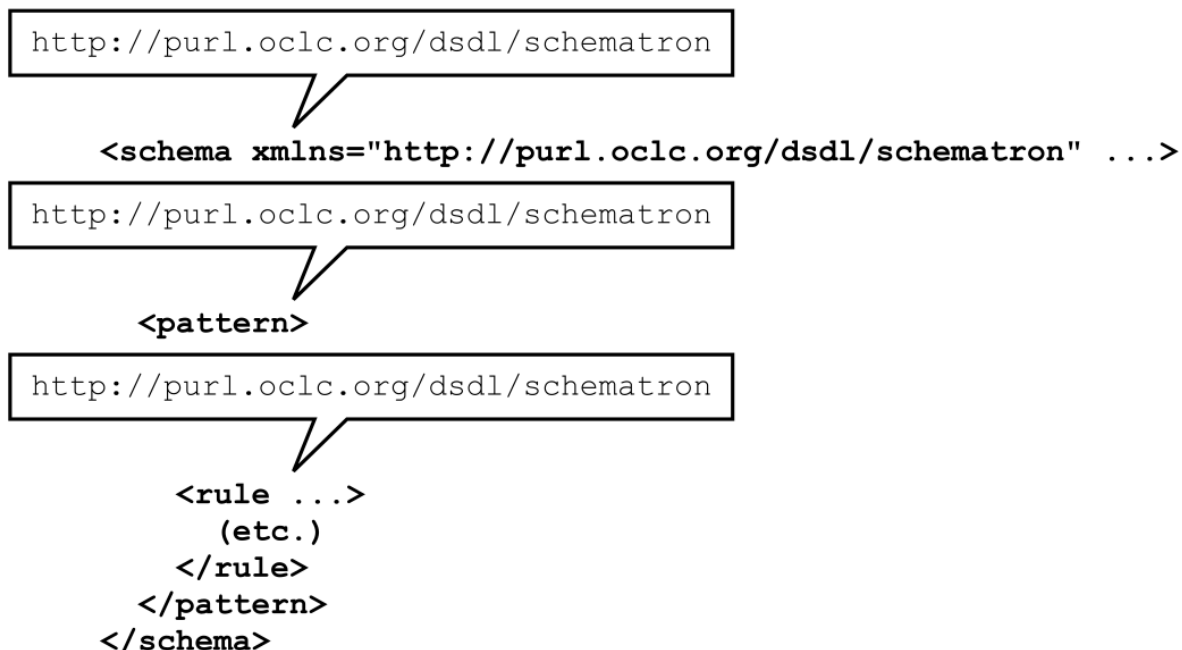
B.3.1. Defining a default namespace

A default namespace is a namespace that is declared on some element and stays in effect for its descendants (children, grandchildren, etc.) unless explicitly changed. You define a default namespace using the `xmlns` "attribute." This book has many examples, including [Example B.1](#).

Example B.1 – Defining the Schematron namespace as default for a Schematron schema using the `xmlns` "attribute". (schematron-book-code/examples/basics/check-type-1.sch)

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <pattern>
    <rule context="@type">
      <assert test="(. eq 'normal') or (. eq 'special')">
        The type must be normal or special
      </assert>
    </rule>
  </pattern>
</schema>
```

Figure B.3. The Schematron namespace, declared as the default namespace



As shown in [Figure B.3](#), the schema root element declares that it is in the Schematron namespace by specifying

`xmlns="http://purl.oclc.org/dsdl/schematron"`. This stays in effect for its descendants, so the `<pattern>`, `<rule>`, and `<assert>` elements are in the Schematron namespace, too.

The reason I write `xmlns` “attribute” in quotes is that, although it looks like an attribute, it’s not. It’s a namespace declaration. `xmlns` is a reserved name in XML, and you cannot use it as a normal attribute name.

But what about attributes: are the attributes in [Example B.1](#), like `context` and `test`, also in the Schematron namespace? The answer is no. Declaring a default namespace is for elements only. If you need to put attributes in a namespace, you have to use the namespace prefix mechanism, which is described in the next section.

A default namespace stays in effect until, on a child element, another default namespace declaration is encountered. For example, see [Example B.2](#).

Example B.2 – Overriding a default namespace declaration

```
<A xmlns="http://my-own-company.com/ns/1">
  <B>
    <C xmlns="http://my-own-company.com/ns/2">
      <D/>
    </C>
  </B>
<E/>
</A>
```

- The elements <A>, , and <E> are in the http://my-own-company.com/ns/1 namespace.
- The elements <C> and <D> are in the http://my-own-company.com/ns/2 namespace.

To put the <C> and <D> elements in the null namespace you can define the namespace as being an empty string, as shown in [Example B.3](#).

Example B.3 – Overriding a default namespace declaration with the null namespace

```
<A xmlns="http://my-own-company.com/ns/1">
  <B>
    <C xmlns="">
      <D/>
    </C>
  </B>
<E/>
</A>
```

B.3.2. Defining and using namespace prefixes

Another mechanism for declaring namespaces is namespace prefixes. A namespace prefix is a short string that serves as stand-in for the, usually much longer, namespace name. Declaring a namespace prefix is done using an xmlns:prefix “attribute”. For example, rewriting [Example B.1](#) using namespace prefixes looks like [Example B.4](#).

Example B.4 – Overriding a default namespace declaration with the null namespace

```
<sch:schema xmlns:sch="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3">
  <sch:pattern>
    <sch:rule context="@type">
      <sch:assert test="(. eq 'normal') or (. eq 'special')">
        Invalid type!
      </sch:assert>
    </sch:rule>
  </sch:pattern>
</sch:schema>
```

```
</sch:pattern>  
</sch:schema>
```

On the root element we declare a namespace prefix, `sch`, with the Schematron namespace name as its value, using the

`xmlns:sch="http://purl.oclc.org/dsdl/schematron"` “attribute” (again it’s officially not an attribute but a namespace declaration). We use the declared `sch` prefix on all elements that need to be in the Schematron namespace (for this example on all elements). This is done by prefixing the element’s name with `sch:.`

Despite the obvious textual differences between [Example B.4](#) and [Example B.1](#), [Example B.4](#) has the same meaning as [Example B.1](#). All elements are in the Schematron namespace. You can feed it to a Schematron processor, which will accept the input and process it the same as it would process [Example B.1](#).

A few details about namespace prefixes:

- A namespace prefix must be declared before it can be used.
- But you can declare as many namespace prefixes as you like, anywhere in your document. You don’t have to use them. A namespace prefix declaration just creates a link between a prefix and a namespace name. It says nothing about how to use the namespace.
- A namespace prefix declaration is in effect on the element it’s declared on and all its children, grandchildren, and so on.
- A namespace prefix stays defined until, on one of its descendant elements, another namespace prefix declaration is found with the same prefix.
- Namespace prefixes can be used to put an attribute in a namespace. To do this, prefix the attribute name with the namespace prefix.
- A namespace prefix must conform to the same rules as the name of an element or attribute.
- The default namespace and namespace prefix mechanisms can be mixed. It’s not unusual to do this. For example, in [Example 7.3](#), the Schematron XML is in the default namespace, and the XSLT key declaration uses the `xsl:` prefix, which puts it in the XSLT namespace (<http://www.w3.org/1999/XSL/Transform>).

Rewriting [Example B.2](#) using namespace prefixes instead of the default namespace mechanism looks like [Example B.5](#).

Example B.5 – Using the namespace prefix mechanism to create a semantically identical version of [Example B.2](#)

```
<ns1:A xmlns:ns1="http://my-own-company.com/ns/1"
  xmlns:ns2="http://my-own-company.com/ns/2">
  <ns1:B>
    <ns2:C>
      <ns2:D/>
    </ns2:C>
  </ns1:B>
  <ns1:E/>
</ns1:A>
```

Since it doesn't matter where you define the prefix (as long it's done before being used), you could also define the prefix as shown in [Example B.6](#).

Example B.6 – Another way of using the namespace prefix mechanism to create a semantically identical version of [Example B.2](#)

```
<ns1:A xmlns:ns1="http://my-own-company.com/ns/1">
  <ns1:B>
    <ns2:C xmlns:ns2="http://my-own-company.com/ns/2">
      <ns2:D/>
    </ns2:C>
  </ns1:B>
  <ns1:E/>
</ns1:A>
```

And what if you want to use the prefix mechanism to create a semantically identical version of [Example B.3](#)? In that example there is, in the middle of elements-in-a-namespace, an “island” of elements (<C> and <D>) in the null namespace. Luckily that's easy to do. If you do not add a prefix, the namespace goes back to the default namespace. In [Example B.7](#) no default namespace is defined, so those elements are in the null namespace. Thus, [Example B.7](#) is semantically identical to [Example B.3](#).

Example B.7 – Using the namespace prefix mechanism to create a semantically identical version of [Example B.3](#)

```
<ns1:A xmlns:ns1="http://my-own-company.com/ns/1">
  <ns1:B>
    <C>
      <D/>
```

```
</C>
</ns1:B>
<ns1:E/>
</ns1:A>
```

B.3.3. The XML namespace

All namespace prefixes must be declared before use, but there's one exception: the prefix `xml` is reserved and always bound to the `http://www.w3.org/XML/1998/namespace` namespace. You're not allowed to re-declare it.

This namespace defines a number of special attributes, ready to be used in any document without having to declare the `xml` prefix. For instance, you can specify the natural language for an element containing text, using the `xml:lang` attribute (see [Example B.8](#)).

Example B.8 – Example of using the `xml:lang` attribute

```
<texts>
  <text xml:lang="nl">Hallo daar, gezellig dat je er bent!</text>
  <text xml:lang="en">Hi there, nice to have you here!</text>
  <text xml:lang="fr">Salut, ravi de vous avoir ici!</text>
</texts>
```

Of course, what actually happens with these language settings is still up to the application that processes the XML. But you now have a standardized way to specify the language.

Notice that the `xml` prefix is not declared explicitly. We could have done this (`xmlns:xml="http://www.w3.org/XML/1998/namespace"`) but it's unnecessary.

You can find more information about the XML namespace and all the attributes it defines at <https://www.w3.org/XML/1998/namespace>.

B.4. Namespaces in Schematron

The rules for declaring and using namespaces in Schematron are unfortunately a bit of a hybrid:

- For the XML elements in a Schematron schema, Schematron follows the rules described in the sections above. It must follow those rules,

otherwise the XML wouldn't be well-formed and would, therefore, be unacceptable for XML parsers.

- Namespaces used in the XPath expressions in Schematron (which are in attributes such as `test`) must be defined using the Schematron `<ns>` element. This is explained in [Section 5.5, “Declaring namespaces”](#).

The `<ns>` element mechanism in Schematron doesn't allow you to define a default namespace. You can find an example in [Section 9.2, “Handling a default namespace”](#).

[\[28\]](#) Officially a namespace name must be a valid URI (Uniform Resource Identifier). The definition of URIs is broader than that of URLs and includes, for instance, official identifiers like `urn:isbn:0-486-27557-4`. However, in most cases, URLs are used for namespace names.

Appendix C. Schematron reference

This appendix provides a reference for the Schematron language. It describes all elements and attributes.

Some elements and attributes appear over and over again. Their definitions are gathered in the last section of this appendix ([Section C.10, “Standard attributes and elements”](#)).

Implementation-defined?

The Schematron language has lots of elements and attributes that are marked as “implementation defined.” What does that mean?

An implementation-defined element or attribute has a defined meaning in the standard, but what actually happens with it (if anything) is completely up to the Schematron processor.

For example, one of the elements marked as implementation defined is `<emph>`. As described in [Section C.9.2, “Emphasis: `<emph>`”](#), you can use this element to emphasize parts of your messages. [Example C.1](#) emphasizes the word *important*.

Example C.1 – Message using the `<emph>` element

```
<assert test="...">This is <emph>important</emph>: wrong value!</assert>
```

However, how this markup is handled is implementation defined. The Schematron processor might or might not pass this markup (or an equivalent) through to the SVRL, and a processor handling the SVRL has complete freedom as to whether it will recognize any markup passed through. If it does recognize the markup, how it chooses to render that markup is also implementation defined.

The bottom line is that all you can rely on with implementation-defined elements and attributes is that they are legal to insert into a Schematron schema.

C.1. XML structure overview notation

This appendix and [Appendix D](#) use a pseudo-code notation to define and explain XML structures. This notation looks like XML, but it’s not well-formed! So don’t copy/paste any of it into your code and expect it to work. For example, this is an XML structure for the fictitious `<some-element>` element:

```
<some-element attribute-1-optional? = (type)
               attribute-2-required = (type)
               attribute-3-fixed-values? = "value-1" | "value-2" >
  <elm-1-optional>?
  <elm-2-mandatory>
  <elm-3-multiple-optional>*
  <elm-4-multiple-mandatory>+
  ( <elm-in-group-1> |
    <elm-in-group-2> ) *
</some-element>
```

After this structure summary/overview, the attributes are listed in a table:

Table C.1 – Attributes of the <some-element> element

Attribute	#	Type	Description						
attribute-1-optional	?	(type)	Default: some-default Optional attribute (there is a ? after its name in the structure overview and in the # column).						
attribute-2-required	1	(type)	Required attribute (There is nothing after its name in the structure overview and a 1 in # column).						
attribute-3-fixed-values	?	(type)	Default: value-1 Optional attribute that must have one of these values: <table><tr><th>Value</th><th>Description</th></tr><tr><td>value-1</td><td>The first value you can use</td></tr><tr><td>value-2</td><td>The second value you can use</td></tr></table>	Value	Description	value-1	The first value you can use	value-2	The second value you can use
Value	Description								
value-1	The first value you can use								
value-2	The second value you can use								

Also the child elements are detailed in a table:

Table C.2 – Child elements of the `<some-element>` element

Child element	#	Description
elm-1-optional	?	This is an optional child element (there is a ? after its name in the structure example above and also in the occurrences column #).
elm-2-mandatory	1	This is a required child element (there is nothing after its name in the structure example above and a 1 in the occurrences column #).
elm-3-multiple-optional	*	This is an optional child element that can occur multiple times (there is a * after its name in the structure example above and also in the occurrences column #).
elm-4-multiple-mandatory	+	This is a required child element that can occur multiple times (there is a + after its name in the structure example above and also in the occurrences column #).
elm-in-group-1	*	An element in a group, see the explanation below
elm-in-group-2	*	Another element in a group, see the explanation below

- The # column says something about the occurrence of the attribute or element:

Nothing or 1	Exactly once (required, single)
?	Zero or once (optional, single)
*	Zero or more (optional, multiple)
+	One or more (required, multiple)

- Attributes are followed by a data type (for instance `xs:string` or `xs:boolean`) or by the list of values it can have (like for `attribute-3-fixed-values`).
- Sometimes child elements are in a group, like `<elm-in-group-1>` and `<elm-in-group-2>` in the example above. You can see this because the group is between parentheses and there's a pipe character (|) between the elements. This means you have a choice of elements and you can repeat this choice as often as the occurrence indicator on the group allows.

In the example above this occurrence indicator is * (zero-or-more), so any combination of both elements would be valid, including none at all.

C.2. The Schematron namespace

All elements that make up the Schematron language are in the namespace:

`http://purl.oclc.org/dsdl/schematron`

Schematron attributes are not in a namespace.

C.2.1. Using other namespaces

Additional elements and attributes in another (non-Schematron) namespace are allowed (almost) anywhere. For instance, this is completely valid (but what it means is up to you):

Example C.2 – Use of a foreign namespace in Schematron

```
<assert test="..." xmlns:nsx="http://some.namespace.definition" nsx:special="true">
  Invalid value for <nsx:fieldname>date</nsx:fieldname> field
</assert>
```

C.3. Root element: <schema>

The root element of a Schematron schema is always <schema>, and it must be in the Schematron namespace (<http://purl.oclc.org/dsdl/schematron>). This means that a basic Schematron schema will usually look like this (using the default namespace construction):

Example C.3 – A basic Schematron schema using a default namespace construction

```
<schema xmlns="http://purl.oclc.org/dsdl/schematron" ...>
...
</schema>
```

If you choose, you can use a prefixed namespace construction, as shown in [Example C.4](#). If you use a prefix, you are free to choose any prefix, but I suggest you use `sch` to avoid confusion.

Example C.4 – A basic Schematron schema using a prefixed namespace construction

```
<sch:schema xmlns:sch="http://purl.oclc.org/dsdl/schematron" ...>
...
</sch:schema>
```

The definition for the <schema> element is:

```
<schema defaultPhase? = xs:ID
  fpi? = xs:string
  icon? = xs:anyURI
  id? = xs:ID
  queryBinding? = xs:string
  schemaVersion? = xs:string
  see? = xs:anyURI
  xml:lang? = xs:language
  xml:space? = xs:string >
  ( <include> |
    <title> |
    <ns> |
    <p> |
    <let> |
    <phase> |
    <pattern> |
    <diagnostics> |
    <properties> )*
</schema>
```

Table C.3 – Attributes of the <schema> element

Attribute	#	Type	Description
defaultPhase	?	xs:ID	Reference to the default phase for this schema. See Section 6.2, “Selecting what patterns are active: <phase>” .
fpi	?	xs:string	A formal public identifier for this element. See Section C.10.2, “The fpi attribute” .
icon	?	xs:anyURI	Location of an appropriate graphics file for this element. See Section C.10.3, “The icon attribute” .
id	?	xs:ID	An identifier for this schema. The Schematron standard says nothing about this attribute, so its meaning and usage are implementation-defined.
queryBinding	?	xs:string	Default: xslt The query binding for this schema. See Chapter 7 .
schemaVersion	?	xs:string	The version of this schema. The value is not restricted (as long as it’s non-empty). Its use is implementation-defined, but it’s usually copied to the SVRL (as an attribute with the same name on the root element). Don’t confuse this with the Schematron version. It’s meant to be the version of this

Attribute	#	Type	Description
			particular schema, not the Schematron version used.
see	?	xs:anyURI	A URI pointing to external information for this element. See Section C.10.6, “The <code>see</code> attribute” .
xml:lang	?	xs:language	The default natural language for this element (and its children). See Section C.10.7, “The <code>xml:lang</code> attribute” .
xml:space	?	xs:string	How whitespace must be handled for this element. See Section C.10.8, “The <code>xml:space</code> attribute” .

Table C.4 – Child elements of the `<schema>` element

Child element	#	Description
include	*	Includes a document. See Section C.10.12, “The <code><include></code> element” .
title	*	Some title, usually meant for documenting (this part of) the schema. See Section C.10.11, “The <code><title></code> element” .
ns	*	A namespace declaration for use in this schema. See Section C.4, “Declaring a namespace: <code><ns></code>” .
p	*	A paragraph of text, usually meant for documenting (this part of) the schema. See Section C.10.10, “The <code><p></code>

Child element	#	Description
		element ".
let	*	A (top-level) variable declaration. See Section C.10.9, "The <let> element" .
phase	*	Defines a validation phase for this schema. See Section C.5, "Defining validation phases: <phase>" .
pattern	*	Defines a pattern for this schema. See Section C.6, "Creating validation patterns: <pattern>" .
diagnostics	*	Container element for diagnostics in this schema. See Section C.7, "Defining diagnostic messages: <diagnostics>". Officially only a single <diagnostics> element is allowed, but most Schematron processors accept multiple occurrences.
properties	*	Container element for properties for the asserts and reports in this schema. See Section C.8, "Defining additional properties: <properties>". Officially only a single <properties> element is allowed, but most Schematron processors accept multiple occurrences.

C.4. Declaring a namespace: <ns>

The `<ns>` element defines a namespace and its prefix for use in the schema. See [Section 5.5, “Declaring namespaces”](#) for more information.

```
<ns prefix = xs:NCName
      uri = xs:anyURI />
```

Table C.5 – Attributes of the `<ns>` element

Attribute	#	Type	Description
prefix	1	xs:NCName	The prefix to use for this namespace.
uri	1	xs:anyURI	The namespace’s URI.

You might expect to define namespaces (and their prefixes) with the usual `xmlns` “attribute” notation, but in Schematron you can’t. You have to use `<ns>` elements.

C.5. Defining validation phases: `<phase>`

A phase is a set of patterns that belong together. When you fire a Schematron validation containing phases you can specify which phase(s) should be active. Only patterns in active phases are checked (see [Section 6.2, “Selecting what patterns are active: `<phase>`”](#)).

```
<phase fpi? = xs:string
      icon? = xs:anyURI
      id = xs:ID
      see? = xs:anyURI
      xml:lang? = xs:language
      xml:space? = xs:string >
  ( <include> |
    <p> |
    <let> |
    <active> )*
</phase>
```

Table C.6 – Attributes of the `<phase>` element

Attribute	#	Type	Description
fpi	?	xs:string	A formal public identifier for this element. See Section C.10.2, “The fpi attribute” .
icon	?	xs:anyURI	Location of an appropriate graphics file for this element. See Section C.10.3, “The icon attribute” .
id	1	xs:ID	The identifier (name) of this phase.
see	?	xs:anyURI	A URI pointing to external information for this element. See Section C.10.6, “The see attribute” .
xml:lang	?	xs:language	The default natural language for this element (and its children). See Section C.10.7, “The xml:lang attribute” .
xml:space	?	xs:string	How whitespace must be handled for this element. See Section C.10.8, “The xml:space attribute” .

Table C.7 – Child elements of the <phase> element

Child element	#	Description
include	*	Includes a document. See Section C.10.12, “The <include> element” .
p	*	A paragraph of text, usually meant for documenting (this part of) the schema. See Section C.10.10, “The <p>

Child element	#	Description
		element ".
let	*	A variable declaration, in-scope for this phase. See Section C.10.9, "The <let> element" .
active	*	Reference to a pattern that belongs to this validation phase. See Section C.5.1, "Attaching a single pattern to a phase: <active>" .

C.5.1. Attaching a single pattern to a phase: <active>

The <active> element, which can occur only as child of <phase>, attaches a single pattern to its parent. The pattern is referenced by the value of its id attribute.

```
<active pattern = xs:IDREF >
  Mixed contents, can contain:
  <dir>*
  <emph>*
  <span>*
</active>
```

Table C.8 – Attributes of the <active> element

Attribute	#	Type	Description
pattern	1	xs:IDREF	The identifier of the referenced pattern (<pattern> element, id attribute. See Section C.6, "Creating validation patterns: <pattern>").

Table C.9 – Child elements of the <active> element

Child element	#	Description
dir	*	Text writing direction. See Section C.9.1, “Writing direction: <dir>” .
emph	*	Emphasize this text. See Section C.9.2, “Emphasis: <emph>” .
span	*	Text span. See Section C.9.4, “Spanning text: ” .

The text contents of an <active> element are for documentation purposes only.

C.6. Creating validation patterns: <pattern>

The <pattern> element defines a Schematron validation pattern. See [Section 5.2, “Patterns, rules, assertions, and reports”](#) for more information.

```
<pattern abstract? = xs:boolean
  documents? = XPath expression
  fpi? = xs:string
  icon? = xs:anyURI
  id? = xs:ID
  is-a? = xs:IDREF
  see? = xs:anyURI
  xml:lang? = xs:language
  xml:space? = xs:string >
  ( <include> |
    <title> |
    <p> |
    <let> |
    <rule> |
    <param> )*
</pattern>
```

Table C.10 – Attributes of the <pattern> element

Attribute	#	Type	Description
abstract	?	xs:boolean	Default: false Whether this is an abstract or a concrete pattern. See Section 6.4, “Reusing patterns: Abstract patterns”.
documents	?	XPath expression	XPath expression that must result in one or more references to documents (URIs/IRIs). If specified, the rules in this pattern will be applied to these documents and not the currently active document. See Section 8.5, “Validating documents referenced by XInclude”.
fpi	?	xs:string	A formal public identifier for this element. See Section C.10.2, “The fpi attribute”.
icon	?	xs:anyURI	Location of an appropriate graphics file for this element. See Section C.10.3, “The icon attribute”.
id	?	xs:ID	The identifier of this pattern. • An abstract pattern (abstract="true") must have an identifier. • If you want to use this pattern as part of a phase (see Section C.5, “Defining validation phases: <phase>”), you must specify an identifier as well.

Attribute	#	Type	Description
is-a	?	xs:IDREF	Specifies that this pattern is an instance of the abstract pattern as referenced by this attribute. See Section 6.4, “Reusing patterns: Abstract patterns” . Abstract patterns (abstract="true") must not specify the is-a attribute.
see	?	xs:anyURI	A URI pointing to external information for this element. See Section C.10.6, “The see attribute” .
xml:lang	?	xs:language	The default natural language for this element (and its children). See Section C.10.7, “The xml:lang attribute” .
xml:space	?	xs:string	How whitespace must be handled for this element. See Section C.10.8, “The xml:space attribute” .

Table C.11 – Child elements of the <pattern> element

Child element	#	Description
include	*	Includes a document. See Section C.10.12, “The <include> element” .
title	*	Some title, usually meant for documenting (this part of) the schema. See Section C.10.11, “The <title> element” .

Child element	#	Description
p	*	A paragraph of text, usually meant for documenting (this part of) the schema. See Section C.10.10, “The <p> element” .
let	*	A variable declaration, in-scope for this pattern. See Section C.10.9, “The <let> element” .
rule	*	A rule within a validation pattern, see Section C.6.1, “Rules in validation patterns: <rule>” .
param	*	Specifies a parameter for an abstract pattern. See Section C.6.2, “Parameters for abstract patterns: <param>” . Use only when this pattern is an instance of an abstract pattern (has an <code>is-a</code> attribute).

C.6.1. Rules in validation patterns: <rule>

A <rule> element specifies a rule in a pattern. See [Section 5.2, “Patterns, rules, assertions, and reports”](#) for more information.

```
<rule abstract? = xs:boolean
  context? = Match expression
  flag? = xs:string
  fpi? = xs:string
  icon? = xs:anyURI
  id? = xs:ID
  role? = xs:string
  see? = xs:anyURI
  subject? = XPath expression
  xml:lang? = xs:language
  xml:space? = xs:string >
( <include> |
  <p> |
  <let> |
```

```

    <assert> |
    <report> |
    <extends> )*
</rule>

```

Table C.12 – Attributes of the `<rule>` element

Attribute	#	Type	Description
abstract	?	xs:boolean	Default: false Whether this is an abstract or a concrete rule. See Section 6.3, “Reusing rules: abstract rules”
context	?	Match expression	Match expression that identifies on which node(s) this rule fires. Subsequently its assertions and reports are checked. When this is an abstract rule (<code>abstract="true"</code>), the <code>context</code> attribute must be absent. Otherwise it is required.
flag	?	xs:string	Name of a flag to be set to <code>true</code> when this element fires. See Section C.10.1, “The <code>flag</code> attribute”.
fpi	?	xs:string	A formal public identifier for this element. See Section C.10.2, “The <code>fpi</code> attribute”.
icon	?	xs:anyURI	Location of an appropriate graphics file for this element. See Section C.10.3, “The <code>icon</code> attribute”.

Attribute	#	Type	Description
id	?	xs:ID	The identifier of this rule.
role	?	xs:string	A name for the role of an assertion or context item. See Section C.10.4, “The role attribute” .
see	?	xs:anyURI	A URI pointing to external information for this element. See Section C.10.6, “The see attribute” .
subject	?	XPath expression	A path expression that allows a more precise specification of the node a rule, assert or report is about. See Section C.10.5, “The subject attribute” .
xml:lang	?	xs:language	The default natural language for this element (and its children). See Section C.10.7, “The xml:lang attribute” .
xml:space	?	xs:string	How whitespace must be handled for this element. See Section C.10.8, “The xml:space attribute” .

Table C.13 – Child elements of the <rule> element

Child element	#	Description
include	*	Includes a document. See Section C.10.12, “The <include> element” .

Child element	#	Description
p	*	A paragraph of text, usually meant for documenting (this part of) the schema. See Section C.10.10, “The <p> element” .
let	*	A variable declaration, in-scope for this rule. See Section C.10.9, “The <let> element” .
assert	*	An assertion to check when this rule is active. See Section C.6.1.1, “Defining assertions: <assert>” .
report	*	A report to check when this rule is active. See Section C.6.1.2, “Defining reports: <report>” .
extends	*	Specifies that this rule is an extension of the abstract rule specified by the <extends> element’s rule attribute. See Section C.6.1.3, “Referencing an abstract rule: <extends>” .

C.6.1.1. Defining assertions: <assert>

The <assert> element defines an assertion to check when the encompassing rule fires. See [Section 5.2, “Patterns, rules, assertions, and reports”](#) for more information.

```
<assert diagnostics? = xs:IDREFS
  flag? = xs:string
  fpi? = xs:string
  icon? = xs:anyURI
  id? = xs:ID
  properties? = xs:IDREFS
  role? = xs:string
  see? = xs:anyURI
```

subject? = XPath expression
 test = XPath expression
 xml:lang? = xs:language
 xml:space? = xs:string >
 Mixed contents, can contain:
 <dir>*
 <emph>*
 *
 <value-of>*
 <name>*
 </assert>

Table C.14 – Attributes of the <assert> element

Attribute	#	Type	Description
diagnostics	?	xs:IDREFS	Whitespace-separated list of identifiers of diagnostics. See Section C.7, “Defining diagnostic messages: <diagnostics>” .
flag	?	xs:string	Name of a flag to be set to <code>true</code> when this element fires. See Section C.10.1, “The flag attribute” .
fpi	?	xs:string	A formal public identifier for this element. See Section C.10.2, “The fpi attribute” .
icon	?	xs:anyURI	Location of an appropriate graphics file for this element. See Section C.10.3, “The icon attribute” .
id	?	xs:ID	The identifier of this assertion.
properties	?	xs:IDREFS	Whitespace separated list of identifiers of properties. See Section C.8, “Defining additional properties: <properties>” .

Attribute	#	Type	Description
role	?	xs:string	A name for the role of an assertion or context item. See Section C.10.4, “The role attribute” .
see	?	xs:anyURI	A URI pointing to external information for this element. See Section C.10.6, “The see attribute” .
subject	?	XPath expression	A path expression that allows a more precise specification of the node a rule, assert or report is about. See Section C.10.5, “The subject attribute” .
test	1	XPath expression	The test to perform. When the effective boolean value is false the assertion fires.
xml:lang	?	xs:language	The default natural language for this element (and its children). See Section C.10.7, “The xml:lang attribute” .
xml:space	?	xs:string	How whitespace must be handled for this element. See Section C.10.8, “The xml:space attribute” .

Table C.15 – Child elements of the <assert> element

Child element	#	Description
dir	*	Text writing direction. See Section C.9.1, “Writing direction: <dir>” .
emph	*	Emphasize this text. See Section C.9.2, “Emphasis: <emph>” .
span	*	Text span. See Section C.9.4, “Spanning text: ” .
value-of	*	Inserts the result(s) of an expression in the text. See Section C.9.5, “Value of an XPath expression: <value-of>” .
name	*	Inserts the name of (usually) the context item in the text. See Section C.9.3, “Retrieving the name of a node: <name>” .

C.6.1.2. Defining reports: <report>

The <report> element defines a report to check when the encompassing rule fires. See [Section 5.2, “Patterns, rules, assertions, and reports”](#) for more information.

```
<report diagnostics? = xs:IDREFS
  flag? = xs:string
  fpi? = xs:string
  icon? = xs:anyURI
  id? = xs:ID
  properties? = xs:IDREFS
  role? = xs:string
  see? = xs:anyURI
  subject? = XPath expression
  test = XPath expression
  xml:lang? = xs:language
  xml:space? = xs:string >
```

Mixed contents, can contain:
 <dir>*
 <emph>*
 *
 <value-of>*
 <name>*
 </report>

Table C.16 – Attributes of the <report> element

Attribute	#	Type	Description
diagnostics	?	xs:IDREFS	Whitespace-separated list of identifiers of diagnostics. See Section C.7, “Defining diagnostic messages: <diagnostics>” .
flag	?	xs:string	Name of a flag to be set to <code>true</code> when this element fires. See Section C.10.1, “The flag attribute” .
fpi	?	xs:string	A formal public identifier for this element. See Section C.10.2, “The fpi attribute” .
icon	?	xs:anyURI	Location of an appropriate graphics file for this element. See Section C.10.3, “The icon attribute” .
id	?	xs:ID	The identifier of this report.
properties	?	xs:IDREFS	Whitespace separated list of identifiers of properties. See Section C.8, “Defining additional properties: <properties>” .
role	?	xs:string	A name for the role of an assertion or context item. See Section C.10.4, “The role

Attribute	#	Type	Description
			attribute ".
see	?	xs:anyURI	A URI pointing to external information for this element. See Section C.10.6, "The see attribute" .
subject	?	XPath expression	A path expression that allows a more precise specification of the node a rule, assert or report is about. See Section C.10.5, "The subject attribute" .
test	1	XPath expression	The test to perform. When the effective boolean value is true the report fires.
xml:lang	?	xs:language	The default natural language for this element (and its children). See Section C.10.7, "The xml:lang attribute" .
xml:space	?	xs:string	How whitespace must be handled for this element. See Section C.10.8, "The xml:space attribute" .

Table C.17 – Child elements of the <report> element

Child element	#	Description
dir	*	Text writing direction. See Section C.9.1, "Writing direction: <dir>" .

Child element	#	Description
emph	*	Emphasize this text. See Section C.9.2, “Emphasis: <emph>” .
span	*	Text span. See Section C.9.4, “Spanning text: ” .
value-of	*	Inserts the result(s) of an expression in the text. See Section C.9.5, “Value of an XPath expression: <value-of>” .
name	*	Inserts the name of (usually) the context item in the text. See Section C.9.3, “Retrieving the name of a node: <name>” .

C.6.1.3. Referencing an abstract rule: <extends>

The <extends> element references an abstract rule. For more information see [Section 6.3, “Reusing rules: abstract rules”](#).

```
<extends href? = xs:anyURI
        rule? = xs:IDREF />
```

Table C.18 – Attributes of the <extends> element

Attribute	#	Type	Description
href	?	xs:anyURI	Reference to an external file that contains a rule definition. See below.

Attribute	#	Type	Description
rule	?	xs:IDREF	Reference by identifier to an abstract rule. See Section C.6.1, “Rules in validation patterns: <rule>” .

The <extends> element must either have a rule or an href attribute, not both.

C.6.2. Parameters for abstract patterns: <param>

The <param> element defines a parameter for an abstract pattern when instantiating it. For more information see [Section 6.4, “Reusing patterns: Abstract patterns”](#).

```
<param name? = xs:string
      value? = xs:string />
```

Table C.19 – Attributes of the <param> element

Attribute	#	Type	Description
name	?	xs:string	Name of the parameter.
value	?	xs:string	Value of the parameter.

C.7. Defining diagnostic messages: <diagnostics>

The <diagnostics> element is a container for zero or more <diagnostic> elements.

```
<diagnostics>
  ( <diagnostic> |
    <include> )*
</diagnostics>
```

Table C.20 – Child elements of the <diagnostics> element

Child element	#	Description
diagnostic	*	A single diagnostic message. See Section C.7.1, “Diagnostic message: <diagnostic>” .
include	*	Includes a document. See Section C.10.12, “The <include> element” .

Officially only a single <diagnostics> element is allowed in a Schematron schema, but most Schematron processors accept multiple occurrences.

C.7.1. Diagnostic message: <diagnostic>

A <diagnostic> element holds a generic, re-usable, diagnostic message. A diagnostic message can be attached to an assert or report, based on its identifier. When the assert or report fires, the diagnostic message will be issued. For more information see [Section 6.1, “Providing multiple messages: <diagnostics>”](#).

```
<diagnostic fpi? = xs:string
            icon? = xs:anyURI
            id = xs:ID
            role? = xs:string
            see? = xs:anyURI
            xml:lang? = xs:language
            xml:space? = xs:string >
Mixed contents, can contain:
<dir>*
<emph>*
<span>*
<value-of>*
</diagnostic>
```

Table C.21 – Attributes of the <diagnostic> element

Attribute	#	Type	Description
fpi	?	xs:string	A formal public identifier for this element. See Section C.10.2, “The fpi attribute” .

Attribute	#	Type	Description
icon	?	xs:anyURI	Location of an appropriate graphics file for this element. See Section C.10.3, “The icon attribute” .
id	1	xs:ID	The identifier of this diagnostic message.
role	?	xs:string	The role of this diagnostic. Typical values are warning, caution OR note. See Section 8.6, “Specifying a role: the role attribute” .
see	?	xs:anyURI	A URI pointing to external information for this element. See Section C.10.6, “The see attribute” .
xml:lang	?	xs:language	The default natural language for this element (and its children). See Section C.10.7, “The xml:lang attribute” .
xml:space	?	xs:string	How whitespace must be handled for this element. See Section C.10.8, “The xml:space attribute” .

Table C.22 – Child elements of the <diagnostic> element

Child element	#	Description
dir	*	Text writing direction. See Section C.9.1, “Writing direction: <dir>” .

Child element	#	Description
emph	*	Emphasize this text. See Section C.9.2, “Emphasis: <emph>” .
span	*	Text span. See Section C.9.4, “Spanning text: ” .
value-of	*	Inserts the result(s) of an expression in the text. See Section C.9.5, “Value of an XPath expression: <value-of>” .

Contrary to what you might expect (and for unknown reasons), you cannot use a <name> element in a diagnostic. Use <value-of select="name(.)"> instead.

C.8. Defining additional properties: <properties>

The <properties> element is a container for zero or more <property> elements.

```
<properties>
  <property>*
</properties>
```

Table C.23 – Child elements of the <properties> element

Child element	#	Description
property	*	A single property. See Section C.8.1, “Defining a property: <property>” .

Officially only a single <properties> element is allowed in a Schematron schema, but most Schematron processors accept multiple occurrences.

C.8.1. Defining a property: <property>

A property is an additional outcome of an assert or report. It allows you to attach arbitrary information when an assert or report fires. For more information see [Section 8.3, “Properties”](#).

```
<property id = xs:ID
          role? = xs:string
          scheme? = xs:string >
  Mixed contents, can contain:
  <dir>*
  <emph>*
  <span>*
  <value-of>*
  <name>*
</property>
```

Table C.24 – Attributes of the <property> element

Attribute	#	Type	Description
id	1	xs:ID	The identifier of this property.
role	?	xs:string	The role of this property.
scheme	?	xs:string	An IRI or other public identifier that specifies the notation used for its value.

Table C.25 – Child elements of the <property> element

Child element	#	Description
dir	*	Text writing direction. See Section C.9.1, “Writing direction: <dir>” .
emph	*	Emphasize this text. See Section C.9.2, “Emphasis: <emph>” .

Child element	#	Description
span	*	Text span. See Section C.9.4, “Spanning text: ” .
value-of	*	Inserts the result(s) of an expression in the text. See Section C.9.5, “Value of an XPath expression: <value-of>” .
name	*	Inserts the name of (usually) the context item in the text. See Section C.9.3, “Retrieving the name of a node: <name>” .

C.9. Message markup: mixed contents

There are several elements in Schematron that can be used to markup messages, resulting in mixed contents: text and markup elements mingled. See [Section 8.1, “Messages with markup: <emph>, , and <dir>”](#) for more information.

C.9.1. Writing direction: <dir>

The <dir> element specifies the reading direction of the enclosed text. For more information see [Section 8.1, “Messages with markup: <emph>, , and <dir>”](#).

```
<dir value? = "ltr" | "rtl" >
  (text only)
</dir>
```

Table C.26 – Attributes of the <dir> element

Attribute	#	Type	Description
value	?	xs:string	Specify the reading direction for the text inside the element.

Attribute	#	Type	Description	
			Value	Description
			ltr	Left-to-right
			rtl	Right-to-left

C.9.2. Emphasis: <emph>

The <emph> element specifies that the enclosed text should be rendered with emphasis. For more information see [Section 8.1, “Messages with markup: <emph>, , and <dir>”](#).

```
<emph>
  (text only)
</emph>
```

C.9.3. Retrieving the name of a node: <name>

The <name> element inserts the name of a node (usually the context item) in messages produced by <assert>, <report>, and <property> elements. See [Section 8.8, “The <name> element”](#) for more information.

```
<name path? = XPath expression />
```

Table C.27 – Attributes of the <name> element

Attribute	#	Type	Description
path	?	XPath expression	Default: . XPath expression that leads to the node you want to retrieve the name of.

C.9.4. Spanning text:

The `<span>` element indicates that the enclosed text should be rendered as indicated by the (required) `class` attribute. It could for instance refer to a definition in some external CSS file. Whether and how this mechanism works is implementation-defined. For more information see [Section 8.1, “Messages with markup: `<emph>`, `<span>`, and `<dir>`”](#).

```
<span class = xs:string >
  (text only)
</span>
```

Table C.28 – Attributes of the `<span>` element

Attribute	#	Type	Description
class	1	xs:string	Rendering class information for the enclosed text.

C.9.5. Value of an XPath expression: `<value-of>`

The `<value-of>` element inserts the result of an XPath expression in messages produced by `<assert>`, `<report>`, `<diagnostic>` and `<property>` elements. For more information see [Section 5.3, “More meaningful messages: `<value-of>`”](#).

```
<value-of select = XPath expression />
```

Table C.29 – Attributes of the `<value-of>` element

Attribute	#	Type	Description
select	1	XPath expression	XPath expression to be evaluated.

C.10. Standard attributes and elements

Some attributes and elements appear over and over in the Schematron specifications. Their definitions are gathered here.

C.10.1. The `flag` attribute

Data type: `xs:string`

The `flag` attribute contains the name of a flag. The Schematron specification states that the initial value of such a flag is `false` and that it becomes `true` when a `<rule>`, `<assert>`, OR `<report>` with that flag fires. How it's handled is implementation-defined. See [Section 8.2, “Flags”](#) for more information.

C.10.2. The `fpi` attribute

Data type: `xs:string`

The `fpi` (formal public identifier) attribute is meant to associate a public (globally unique) identifier with the element it's on. How it's handled, is implementation-defined. For more information see [Section 8.9, “The `icon`, `see`, and `fpi` attributes”](#).

C.10.3. The `icon` attribute

Data type: `xs:anyURI`

The `icon` attribute associates some graphics file with the element it's on. How it's handled, is implementation-defined. For more information see [Section 8.9, “The `icon`, `see`, and `fpi` attributes”](#).

C.10.4. The `role` attribute

Data type: `xs:string`

The `role` attribute can allows you to say something about the role of an assertion, report or property. How it's handled, is implementation-defined. See [Section 8.6, “Specifying a role: the `role` attribute”](#) for more information.

C.10.5. The `subject` attribute

Data type: XPath expression

The `subject` attribute allows you to specify an alternate location in the XML for the subject of an assert or report. See [Section 8.7, “Specify a different location: the `subject` attribute”](#) for more information.

C.10.6. The `see` attribute

Data type: `xs:anyURI`

The `see` attribute contains some URI to additional information regarding the element it's on. How it's handled, is implementation-defined. For more information see [Section 8.9, “The `icon`, `see`, and `fp:attributes`”](#).

C.10.7. The `xml:lang` attribute

Data type: `xs:language`

The `xml:lang` attribute sets the natural (human) language for the children of the element it's on. How it's handled, is implementation-defined.

Setting a language is often done in conjunction with using diagnostics ([Section 6.1, “Providing multiple messages: `<diagnostics>`”](#)).

C.10.8. The `xml:space` attribute

Data type: `xs:string`. Allowed values: `default`, `preserve`.

The `xml:space` attribute defines how to handle whitespace for the element it's on. Allowed values are `default` (as you might guess, its default value) and `preserve` only. How it's handled, is implementation-defined.

A reason outside of Schematron to use `xml:space="preserve"` is to stop the pretty-print function of an XML editor from formatting your carefully manually indented text contents. However, your editor may not recognize and respect `xml:space="preserve"`.

C.10.9. The `<let>` element

The `<let>` element declares a variable for use in the schema. See [Section 5.4, “Declaring and using variables: `<let>`”](#) for more information.

```
<let name = QName  
      value = XPath expression />
```

Table C.30 – Attributes of the <let> element

Attribute	#	Type	Description
name	1	QName	The name of the variable.
value	1	XPath expression	The XPath expression that provides the value for this variable.

C.10.10. The <p> element

The <p> element represents, according to the standard, “a paragraph of natural language text containing maintainer and user formation about the parent element.” How it’s handled is implementation-defined. For more information see [Section 8.4, “Adding structured comments: <title> and <p>”](#).

```
<p class? = xs:string
  icon? = xs:anyURI
  id? = xs:ID >
Mixed contents, can contain:
<dir>*
<emph>*
<span>*
</p>
```

Table C.31 – Attributes of the <p> element

Attribute	#	Type	Description
class	?	xs:string	A class to signify that this paragraph of text should be rendered in a specific way.
icon	?	xs:anyURI	Location of an appropriate graphics file for this element. See Section C.10.3, “The icon attribute” .
id	?	xs:ID	Some identifier for this paragraph.

Table C.32 – Child elements of the <p> element

Child element	#	Description
dir	*	Text writing direction. See Section C.9.1, “Writing direction: <dir>” .
emph	*	Emphasize this text. See Section C.9.2, “Emphasis: <emph>” .
span	*	Text span. See Section C.9.4, “Spanning text: ” .

C.10.11. The <title> element

The <title> element contains, according to the standard, “a summary of the purpose or role of the schema, pattern or rule for the purpose of documentation or a rich user interface”. How it’s handled, is implementation-defined. For more information see [Section 8.4, “Adding structured comments: <title> and <p>”](#).

```
<title>
  Mixed contents, can contain:
  <dir>*
</title>
```

Table C.33 – Child elements of the <title> element

Child element	#	Description
dir	*	Text writing direction. See Section C.9.1, “Writing direction: <dir>” .

C.10.12. The <include> element

The <include> element includes the contents of an external XML document. For more information see [Section 6.5, “Including documents: <include>”](#).

```
<include href = xs:anyURI />
```

Table C.34 – Attributes of the <include> element

Attribute	#	Type	Description
href	1	xs:anyURI	The location of the XML document to include.

Appendix D. SVRL reference

This appendix provides a reference for the SVRL vocabulary. SVRL stands for Schematron Validation Reporting Language. SVRL documents are produced by a Schematron processor as the result of a validation. See also [Section 4.4.1, “SVRL”](#).

The SVRL produced by the processor might be hidden from you. For instance, when you run a Schematron validation inside an IDE, you usually don't see any SVRL. Instead you get a nicely formatted list of messages and red squiggly lines. However, SVRL is used behind the scenes.

D.1. SVRL in the Schematron standard

The definition of SVRL in the official Schematron standard has (in my opinion) some serious flaws. It's underspecified and what is specified is often subtly inconsistent with Schematron itself.

- SVRL itself is not documented, except that the standard provides a (RELAX NG) schema. The meaning of elements and attributes is left to the imagination of the Schematron processor's implementer.
- There are no *production rules*. That is, the relation between what a Schematron schema specifies, the validation results, and the resulting SVRL is not specified. Sometimes this relation is obvious but not always.
- SVRL is inconsistent with Schematron in several places. For instance, the `<emph>` element (emphasis) in SVRL has a `class` attribute. In Schematron it does not. Also data types for attributes often differ.

I think leaving SVRL undocumented and its production rules unspecified is a serious flaw in the Schematron specification. It means you're at the mercy of the person who developed your Schematron processor when it comes to the SVRL produced. I don't doubt their good intentions, but SVRL's unspecified nature means running the same validation on the same input data could yield different results using different processors, and all would be correct. Interoperability of the SVRL output between processors is not guaranteed.

The current prevailing interpretation of SVRL and how it should be produced comes from the original skeleton Schematron processor (see [Section 4.5.1, “The “skeleton” XSLT Schematron processor”](#)). This interpretation has been adopted by the current version of the SchXslt processor (see [Section 4.5.2, “The SchXslt Schematron processor”](#)). Given the limited number of Schematron processors available, interoperability is not likely to become a serious problem any time soon.

This reference follows the prevailing interpretation of SVRL, as implemented by the current available processors. Despite this, there are places that are still unclear to me. I identify those places with the term [unclear].

This reference uses the same markup used for the Schematron schema in [Appendix C](#).

D.2. The SVRL namespace

The elements that make up an SVRL message are in the namespace:

`http://purl.oclc.org/dsdl/svrl`

The attributes are not in a namespace.

Some elements, most notably `<text>`, allow the use of attributes and elements in other namespaces.

D.3. Root element: `<schematron-output>`

The root element for an SVRL document is always `<schematron-output>`:

```
<schematron-output phase? = xs:NMTOKEN
                  schemaVersion? = (unspecified)
                  title? = (unspecified) >
  <text>*
  <ns-prefix-in-attribute-values>*
  <active-pattern>+
  <fired-rule>*
  <failed-assert>*
  <successful-report>*
</schematron-output>
```

Table D.1 – Attributes of the <schematron-output> element

Attribute	#	Type	Description
phase	?	xs:NMTOKEN	The Schematron phase that this SVRL is a result of. See Section 6.2, “Selecting what patterns are active: <phase>” .
schemaVersion	?	(unspecified)	A copy of the Schematron schema’s schemaVersion attribute (see Section C.3, “Root element: <schema>”).
title	?	(unspecified)	[unclear] Some title for this validation result. A Schematron schema has no corresponding attribute.

Table D.2 – Child elements of the <schematron-output> element

Child element	#	Description
text	*	[unclear] Some text about the schema and/or validation. See Section D.9, “Mixed text: <text>”. A Schematron schema has no corresponding element. Several ways of filling this element come to mind (for instance the contents of the schema’s main <title> and/or <p> elements), but none is specified.
ns-prefix-in-attribute-values	*	A namespace and prefix declaration, as declared with <ns> in the schema. See Section D.4, “Namespace declarations: <ns-prefix-in-attribute-values>” .

Child element	#	Description
active-pattern	+	Indicates that this pattern was active. See Section D.5, “Active patterns: <active-pattern>” .
fired-rule	*	Indicates that a rule fired. See Section D.6, “Fired rules: <fired-rule>” .
failed-assert	*	Indicates a failed assert. See Section D.7, “Failed asserts: <failed-assert>” .
successful-report	*	Indicates a successful report. See Section D.8, “Successful reports: <successful-report>” .

D.4. Namespace declarations: <ns-prefix-in-attribute-values>

The <ns-prefix-in-attribute-values> element is the counterpart of the <ns> element in the original schema (see [Section C.4, “Declaring a namespace: <ns>”](#)). It defines a namespace and corresponding prefix for use in expressions.

```
<ns-prefix-in-attribute-values prefix = xs:NMTOKEN
                               uri = (unspecified) />
```

Table D.3 – Attributes of the <ns-prefix-in-attribute-values> element

Attribute	#	Type	Description
prefix	1	xs:NMTOKEN	The prefix to use for this namespace.
uri	1	(unspecified)	The namespace’s URI.

Data type inconsistencies

The `prefix` attribute here is a typical example of data type inconsistencies in the SVRL definition. Schematron defines the `prefix` attribute on the `<ns>` element as data type `xs:NCName`. And that's correct: a namespace prefix can't contain a colon, which is the constraint the `xs:NCName` data type provides.

In contrast, the `prefix` attribute of the SVRL `<ns-prefix-in-attribute-values>` element is defined as having the data type `xs:NMTOKEN`, which is way too broad, since it allow characters, including colon, that are not allowed in a namespace prefix.

D.5. Active patterns: `<active-pattern>`

The `<active-pattern>` elements indicate which patterns are active. For more information see [Section 6.2, “Selecting what patterns are active: `<phase>`”](#).

```
<active-pattern documents? = (unspecified)
    id? = xs:NCName
    name? = (unspecified)
    role? = xs:string />
```

Table D.4 – Attributes of the `<active-pattern>` element

Attribute	#	Type	Description
documents	?	(unspecified)	The documents on which this pattern was active. This can be the primary document the schema validated or the documents specified in the <code>documents</code> attribute of the <code><pattern></code> element (see Section 8.5, “Validating documents referenced by <code>XInclude</code>”). The probable data type for this attribute is a whitespace separated list of <code>xs:anyURI</code> values, but no data type is specified.

Attribute	#	Type	Description
id	?	xs:NCName	The identifier for the pattern (a copy of the schema <pattern> element id attribute).
name	?	(unspecified)	[unclear] Some name for this pattern. A pattern in a Schematron schema has no corresponding attribute.
role	?	xs:string	[unclear] Some role indicator for this pattern. A pattern in the Schematron schema has no corresponding attribute.

If the <pattern> element of an active pattern in the schema has no id attribute, the only thing you'll see in the SVRL is:

```
<active-pattern documents="..." />
```

This is not very informative because it doesn't tell you which pattern was active. So if you're processing the SVRL and want to infer something useful from the list of active patterns, make sure to provide each pattern with an identifier.

D.6. Fired rules: <fired-rule>

The <fired-rule> element indicates that a rule has fired: that a node in the document was found for which the context attribute of the <rule> element in the schema matches. See [Section 5.2, “Patterns, rules, assertions, and reports”](#).

```
<fired-rule context = (unspecified)
  document? = xs:anyURI
  flag? = xs:string
  id? = xs:NCName
  name? = (unspecified)
  role? = xs:string />
```

Table D.5 – Attributes of the <fired-rule> element

Attribute	#	Type	Description
context	1	(unspecified)	The context for this rule (a copy of the schema <code><rule></code> element <code>context</code> attribute).
document	?	<code>xs:anyURI</code>	[unclear] An obvious interpretation is the URI of the document this rule fired against. However, no implementation I've tried does anything with this attribute.
flag	?	<code>xs:string</code>	A flag that was set to <code>true</code> when this rule fired (a copy of the schema <code><rule></code> element <code>flag</code> attribute).
id	?	<code>xs:NCName</code>	The identifier for the rule (a copy of the schema <code><rule></code> element <code>id</code> attribute).
name	?	(unspecified)	[unclear] Some name for this rule. A rule in the Schematron schema has no corresponding attribute.
role	?	<code>xs:string</code>	The role for this rule (a copy of the schema <code><rule></code> element <code>role</code> attribute).

D.7. Failed asserts: `<failed-assert>`

The `<failed-assert>` element indicates that an assert has failed: the expression in the schema `<assert>` element's `test` attribute evaluated to `false`.

```
<failed-assert flag? = xs:string
              id? = xs:NCName
              location = (unspecified)
              role? = xs:string
              test = (unspecified) >
```

```

<text>
<diagnostic-reference>*
<property-reference>*
</failed-assert>

```

Table D.6 – Attributes of the <failed-assert> element

Attribute	#	Type	Description
flag	?	xs:string	A flag that was set to true when this assert/report fired (a copy of the schema <assert> element flag attribute).
id	?	xs:NCName	The identifier for the assert/report (a copy of the schema <assert> element id attribute).
location	1	(unspecified)	The location of the node in the document (as an XPath expression) where the assert/report failed on.
role	?	xs:string	The role for this assert/report (a copy of the schema <assert> element role attribute).
test	1	(unspecified)	The test expression for this assert/report (a copy of the schema <assert> element test attribute).

Table D.7 – Child elements of the <failed-assert> element

Child element	#	Description
text	1	The resulting text (with optional markup) for this assert/report. See Section D.9, “Mixed text: <text>” .

Child element	#	Description
diagnostic-reference	*	A diagnostic message that was linked to this assert/report. See Section D.7.1, “Diagnostic reference: <diagnostic-reference>” .
property-reference	*	A property that was linked to this assert/report. See Section D.7.2, “Property reference: <property-reference>” .

D.7.1. Diagnostic reference: <diagnostic-reference>

The <diagnostic-reference> element references a diagnostic connected to the assert/report. See [Section 6.1, “Providing multiple messages: <diagnostics>”](#).

```
<diagnostic-reference diagnostic = xs:NCName >
  <text>
</diagnostic-reference>
```

Table D.8 – Attributes of the <diagnostic-reference> element

Attribute	#	Type	Description
diagnostic	1	xs:NCName	The identifier of the diagnostic (a copy of the schema <diagnostic> element id attribute).

Table D.9 – Child elements of the <diagnostic-reference> element

Child element	#	Description
text	1	The resulting text (with optional markup) for this diagnostic. See Section D.9, “Mixed text: <text>” .

D.7.2. Property reference: <property-reference>

The `<property-reference>` element references a property connected to the assert/report. See [Section 8.3, “Properties”](#).

```
<property-reference property = xs:NCName
                    role? = (unspecified)
                    scheme? = (unspecified) >
  <text>
</property-reference>
```

Table D.10 – Attributes of the `<property-reference>` element

Attribute	#	Type	Description
property	1	xs:NCName	The identifier of the property (a copy of the schema <code><property></code> element id attribute).
role	?	(unspecified)	The role for this property (a copy of the schema <code><property></code> element role attribute).
scheme	?	(unspecified)	The scheme for this property (a copy of the schema <code><property></code> element scheme attribute).

Table D.11 – Child elements of the `<property-reference>` element

Child element	#	Description
text	1	The resulting text (with optional markup) for this property. See Section D.9, “Mixed text: <code><text></code>” .

D.8. Successful reports: `<successful-report>`

The `<successful-report>` element indicates that a report was successful: the expression in the schema `<report>` test attribute evaluated to true. Its attributes and child elements are exactly the same as for the `<failed-assert>` element. Therefore please refer to [Section D.7, “Failed asserts: `<failed-assert>`”](#) for details.

D.9. Mixed text: <text>

A <text> element contains the resulting text of an assert, report, diagnostic, or property. The term *mixed* means that text and certain markup elements can be intermingled. See [Section C.9, “Message markup: mixed contents”](#) for more information.

The main difference between Schematron and SVRL is that SVRL does not have the <value-of> and <name> elements. These schema elements are replaced by the evaluated expression or node name.

```
<text fpi? = (unspecified)
      icon? = (unspecified)
      see? = (unspecified)
      xml:lang? = xs:language
      xml:space? = xs:string >
  Mixed contents, can contain:
  <dir>*
  <span>*
  <emph>*
</text>
```

Table D.12 – Attributes of the <text> element

Attribute	#	Type	Description
fpi	?	(unspecified)	A formal public identifier for this element (a copy of the Schematron fpi attribute).
icon	?	(unspecified)	Location of an appropriate graphics file for this element (a copy of the Schematron icon attribute).
see	?	(unspecified)	A URI pointing to some external information for this element (a copy of the Schematron <fpi> attribute).
xml:lang	?	xs:language	The default natural language for this element (a copy of the Schematron xml:lang

Attribute	#	Type	Description
			attribute).
xml:space	?	xs:string	How whitespace must be handled for this element (a copy of the Schematron xml:space attribute).

Table D.13 – Child elements of the <text> element

Child element	#	Description
dir	*	Text writing direction. See Section D.9.1, “Writing direction: <dir>” .
span	*	Generic span for text markup. See Section D.9.3, “Spanning text: ” .
emph	*	Emphasize this text. See Section D.9.2, “Emphasis: <emph>” .

D.9.1. Writing direction: <dir>

The <dir> element specifies the reading direction of the enclosed text.

```
<dir class? = (unspecified)
    dir? = "ltr" | "rtl" >
  (text only)
</dir>
```

Table D.14 – Attributes of the <dir> element

Attribute	#	Type	Description						
class	?	(unspecified)	[unclear] Rendering class information for the enclosed text. A <dir> element in the Schematron schema has no class attribute.						
dir	?	xs:string	Specify the reading direction for the text inside the element (a copy of the schema <dir> element <value> attribute). <table><tr><th>Value</th><th>Description</th></tr><tr><td>ltr</td><td>Left-to-right</td></tr><tr><td>rtl</td><td>Right-to-left</td></tr></table>	Value	Description	ltr	Left-to-right	rtl	Right-to-left
Value	Description								
ltr	Left-to-right								
rtl	Right-to-left								

D.9.2. Emphasis: <emph>

The <emph> element specifies that the enclosed text should be rendered with emphasis.

```
<emph class? = (unspecified) >
  (text only)
</emph>
```

Table D.15 – Attributes of the <emph> element

Attribute	#	Type	Description
class	?	(unspecified)	[unclear] Rendering class information for the enclosed text. An <emph> element in the Schematron schema has no class attribute.

D.9.3. Spanning text:

The `<span>` element indicates that the enclosed text should be rendered as indicated by the (required) `class` attribute. It could for instance refer to a definition in some external CSS file, but whether and how this mechanism works is completely implementation-defined.

```
<span class = xs:string >  
  (text only)  
</span>
```

Table D.16 – Attributes of the `<span>` element

Attribute	#	Type	Description
<code>class</code>	1	<code>xs:string</code>	Rendering class information for the enclosed text (a copy of the corresponding schema element <code>class</code> attribute)..

Appendix E. Schematron QuickFix

At the XML Prague conference in 2016, Nico Kutscherauer (data2type) and Octavian Nadolu (Syncro Soft, the company that develops the oXygen IDE) introduced an extension to Schematron: Schematron QuickFix or SQF. This extension allows you to embed so-called QuickFixes in Schematron schemas. SQF-enabled user interfaces can pick up these fixes and offer them to the user. For instance, when an element seems to be missing, an SQF QuickFix can tell the user interface to offer an “insert ... element” option. SQF is based on the idea that a Schematron validation should not only tell the user what is wrong, it should also offer a solution.

SQF is a promising and useful standard, but support is limited:

- There is a [W3C draft specification](#) from 2015.
- A nearly complete implementation is available in the oXygen IDE.
- Another implementation is [Escali](#), but as of this writing (2022) development seems to be stalled at version 0.2, last updated in 2019.

This appendix provides an example using the oXygen IDE. More information and examples can be found on the [SQF website](#) or in the oXygen IDE help pages (search for SQF).

Let’s start by validating the document shown in [Example E.1](#).

Example E.1 – XML document to use for demonstrating the QuickFix capabilities (schematron-book-code/data/things-for-sqf.xml)

```
<things>
  <thing name="thing 1" type="normal"/>
  <thing name="thing 2" type="outstanding"/>
  <thing name="thing 3" type="special"/>
  <thing type="normal"/>
</things>
```

The rules for this document are:

- The value of the type attribute on <thing> must be either normal or special.
- The <thing> element must have a name attribute.

As you can see, the second and fourth <thing> elements violate the rules. Let's validate this document and also offer the user ways to resolve the issues.

A Schematron schema to do this using SQF looks like [Example E.2](#).

Example E.2 – Schematron schema with QuickFixes (schematron-book-code/examples/sqf/check-things-for-sqf.sch)

```
<!-- 1 - Define the QuickFix namespace: -->
<schema xmlns="http://purl.oclc.org/dsdl/schematron" queryBinding="xslt3"
  xmlns:sqf="http://www.schematron-quickfix.com/validator/process">

  <pattern>
    <rule context="thing">

      <!-- 2 - Check that things have a correct type: -->
      <assert test="@type eq 'normal' or @type eq 'special'"
        sqf:fix="change-type delete-thing">
        Type must be normal or special
      </assert>

      <!-- 3 - Check that things have a name: -->
      <assert test="@name" sqf:fix="add-name delete-thing">
        A thing must have a name
      </assert>

      <!-- 4 - Define a QuickFix to delete a thing:-->
      <sqf:fix id="delete-thing">
        <sqf:description>
          <sqf:title>Remove thing</sqf:title>
          <sqf:p>Remove this thing from the list</sqf:p>
        </sqf:description>
        <sqf:delete/>
      </sqf:fix>

      <!-- 5 - Define a QuickFix to change the type of a thing: -->
      <sqf:fix id="change-type" use-for-each="('normal', 'special')">
        <sqf:description>
          <sqf:title>
            Set the thing's type to <value-of select="$sqf:current"/>
          </sqf:title>
        </sqf:description>
        <sqf:replace match="@type" node-type="attribute" target="type"
          select="$sqf:current"/>
      </sqf:fix>

      <!-- 6 - Define a QuickFix to add a name: -->
      <sqf:fix id="add-name">
        <sqf:description>
          <sqf:title>Add a name</sqf:title>
        </sqf:description>
        <sqf:user-entry name="newname">
          <sqf:description>
            <sqf:title>What is the name of the thing?</sqf:title>
          </sqf:description>
        </sqf:user-entry>
      </sqf:fix>
    </rule>
  </pattern>
</schema>
```

```

        </sqf:user-entry>
        <sqf:add node-type="attribute" target="name" select="$newname"/>
    </sqf:fix>

</rule>
</pattern>

</schema>

```

1. Define the SQF namespace with the prefix `sqf`. The namespace URL is <http://www.schematron-quickfix.com/validator/process>.
2. Define a Schematron assertion that checks the value of the `type` attribute.

Link this assertion, using the `sqf:fix` attribute, with two SQF fixes: `change-type` and `delete-thing`. These identifiers refer to the QuickFixes described in point 4 and 5 below.

3. Define a Schematron assertion that checks for the presence of the `name` attribute.

Link this assertion, using the `sqf:fix` attribute, with two SQF fixes: `add-name` and `delete-thing`. These identifiers refer to the QuickFixes described in point 4 and 6 below.

4. Define an SQF QuickFix that deletes a `<thing>` and give it the identifier `delete-thing`. This identifier is used in the `sqf:fix` attributes on the `<assert>` elements to reference this QuickFix.

The QuickFix contains a description (`<sqf:description>`) that the processor displays in the user interface.

Finally, it references the SQF action `<sqf:delete>`, which deletes the context item (the `<thing>` element that failed the test) from the document.

5. Define an SQF QuickFix that changes the type of a thing and give it the identifier `change-type`. This identifier is used to reference it from assert 2 in its `sqf:fix` attribute.

This QuickFix gets repeated because of its `use-for-each` element. It creates two actions for the user to choose from: one for changing the type to `normal` and one for changing it to `special`.

The QuickFix contains a description (`<sqf:description>`) that the processor displays in the user interface. This description references the internal variable `$sqf:current` that takes its value from the `use-for-each` attribute.

Finally, it references the SQF action `<sqf:replaces>`, which replaces the value `type` attribute of the context item (the `<thing>` element that failed the test) with the correct value chosen by the user.

6. Define an SQF QuickFix that adds a name to a thing and give it the identifier `add-name`. This identifier is used to reference it from assert 3 in its `sqf:fix` attribute.

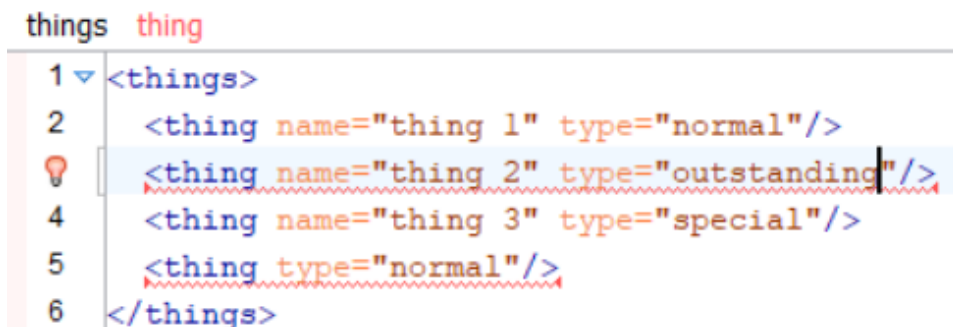
The QuickFix contains a description (`<sqf:description>`) that the processor displays in the user interface.

It then defines a user-entry interaction using the `<sqf:user-entry>` element. This causes a message box with a text entry field to pop up for the user. The user's entry will be stored in the `$newname` variable.

Finally, it references the SQF action `<sqf:add>`, which adds the `name` attribute to the context item (the `<thing>` element that failed the test), with the value entered by the user.

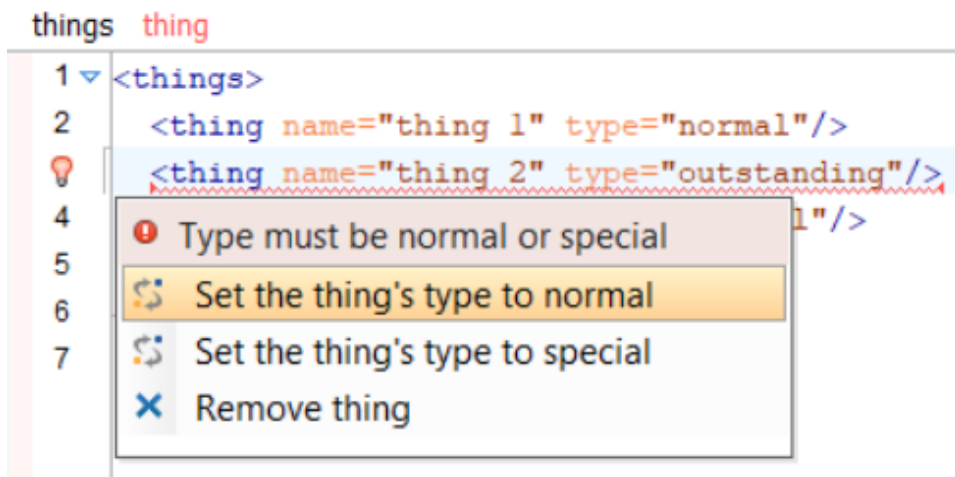
The following screenshots show how this looks in the oXygen version 24.0 user interface. After being validated against [Example E.2](#), [Example E.1](#) looks like [Figure E.1](#).

Figure E.1. [Example E.1](#) after validation with [Example E.2](#).



When the cursor is placed on the second, invalid, thing a light bulb appears in the left margin. When you click on this, the following appears ([Figure E.2](#)):

Figure E.2. [Figure E.1](#) after clicking on the light bulb.

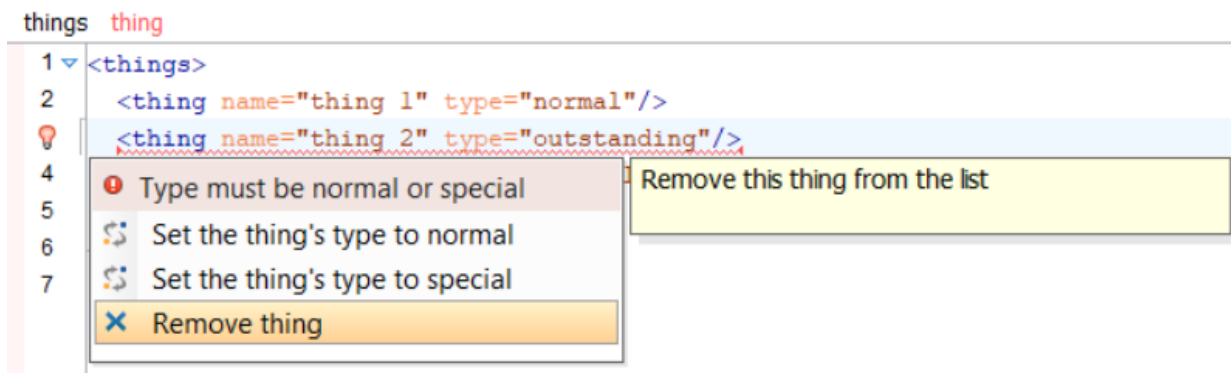


This dropdown menu shown in [Figure E.2](#) has four entries:

- The first entry is the Schematron schema error message, as defined in assert 2 of [Example E.2](#).
- The second entry allows the user to set the type to `normal`. This comes from the QuickFix 5 `change-type` fix.
- The third entry allows the user to set the type to `special`. This also comes from the QuickFix 5 `change-type` fix.
- The fourth entry, which comes from the QuickFix 6, `delete-thing` fix allows the user to delete the `<thing>` element.

If you hover the mouse over the last entry you see an additional message pop-up (see [Figure E.3](#)). This message is defined in the `<sqf:p>` element, which is a child of `<sqf:description>` in this QuickFix in [Example E.2](#).

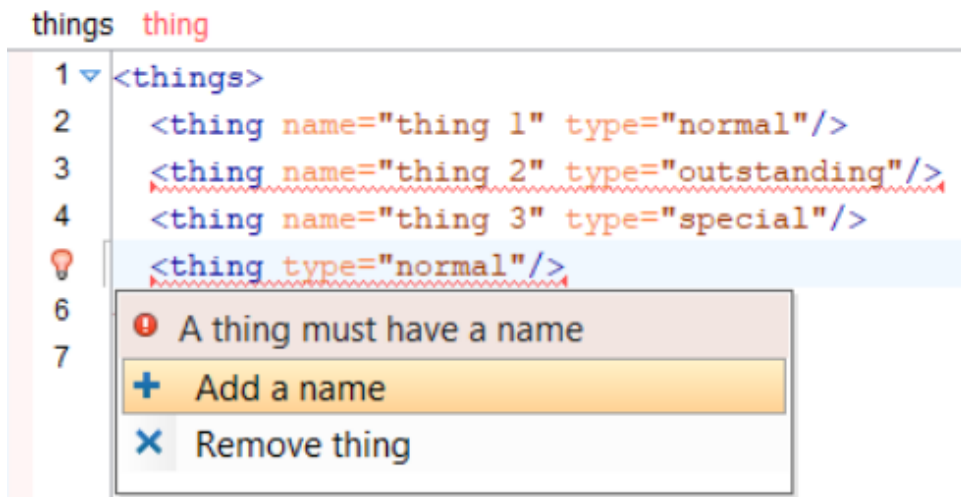
Figure E.3. [Figure E.2](#) after selecting the last menu entry.



Clicking one of the menu entries executes the associated action.

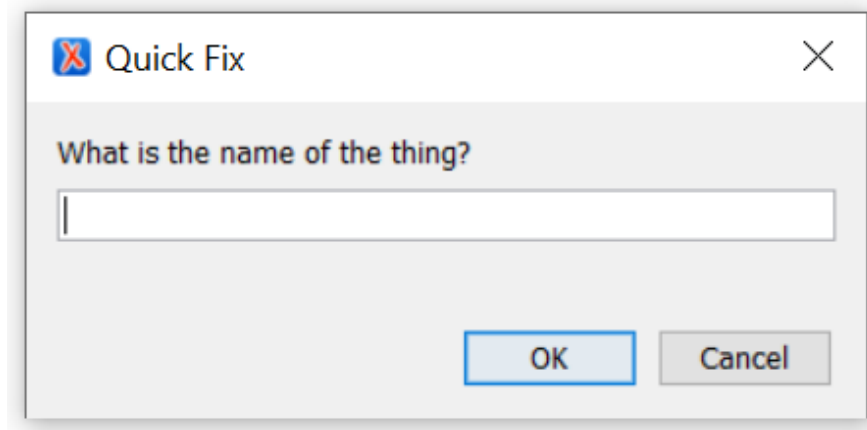
When you click on the light bulb for the fourth thing, which lacks a `name` attribute, you get a different menu because a different assertion was triggered ([Figure E.4](#)).

Figure E.4. The QuickFix menu after selecting the fourth thing



Clicking the `Add a name` entry pops up a message box that lets you enter a name for the thing

Figure E.5. The QuickFix message box for entering a name



Since this is just an SQF appetizer, don't worry if you don't immediately grasp all the details. But I hope you agree that SQF is a valuable addition to Schematron. A schema designer can now offer users an automated way to fix

validation problems. SQF currently (mid-2022) works only with oXygen, but if you are using oXygen, it can make it easier for users to create valid documents.

Appendix F. Additional resources

This appendix contains suggestions for additional information resources. It's compiled using the personal preferences of the author, combined with suggestions from others. Although incomplete and biased, it's nevertheless a good place to start exploring.

F.1. Schematron

- ***Schematron* by Eric van der Vlist (O'Reilly, 2007)**

A short book (around 50 pages), describing an older version of the Schematron standard.

- **Schematron website:** <https://schematron.com/>

A website about Schematron, maintained by the original designer of the language, Rick Jelliffe.

- **Awesome Schematron:** <https://github.com/Schematron/awesome-schematron>

As it says itself: “A curated list of awesome Schematron tools and applications.” Contains links to (all versions of) the specification, interesting articles, and more.

F.2. XPath

XPath is the underlying technology for many XML-related languages, including Schematron. Given its importance, there's a whole appendix in this book ([Appendix A](#)) devoted to this subject.

There are, as far as I know, no books that cover just XPath. However, it does feature prominently in some books about other subjects, including the following:

- ***XSLT 2.0 and XPath 2.0: Programmer's Reference* by Michael Kay (Wiley, 2008)**

Although outdated (it covers XSLT and XPath version 2.0, but in 2022 we're up to XSLT version 3.0 and XPath version 3.1), it's still a valuable introduction to XPath. It doesn't cover the latest features, but it does teach you the fundamentals of the language in a way I still haven't seen equaled anywhere else.

- ***XQuery: Search Across a Variety of XML Data* by Priscilla Walmsley (O'Reilly, 2016)**

This book about XQuery covers the XPath 3.1 standard. It provides a function overview and has separate sections about advanced topics such as maps, arrays, JSON, and higher-order functions. Unfortunately, it doesn't clearly distinguish XQuery and XPath from each other.

Besides the information in these books, there are also the official standards:

- **XML Path Language (XPath) 3.1:** <https://www.w3.org/TR/xpath-31/>

The official language W3C recommendation dated 21 March 2017.

- **XPath and XQuery Functions and Operators 3.1:** <https://www.w3.org/TR/xpath-functions-31/>

The XPath functions and operators W3C recommendation dated 21 March 2017. This resource is especially useful if you are looking for information about a specific function.

F.3. XML Schema

XML Schema is a schema language for validating XML documents. You can find an example in [Section 3.2.2, “W3C XML Schema”](#).

- ***Definitive XML Schema* by Priscilla Walmsley (Prentice Hall, 2012)**

Covers XML Schema in detail. Includes information on the 1.1 version.

F.4. RELAX NG

RELAX NG is a schema language for validating XML documents. You can find an example in [Section 3.2.3, “RELAX NG”](#).

- ***RELAX NG* by Eric van der Vlist (O’Reilly, 2003)**

Covers RELAX NG in detail.

- **RELAX NG website:** <https://relaxng.org/>

Contains links to the standard itself and other related information.

F.5. XSLT

The programming language XSLT is mentioned a few times in this book. It’s a language for transforming XML documents.

- ***XSLT 2.0 and XPath 2.0: Programmer’s Reference* by Michael Kay (Wiley, 2008)**

Although outdated (it covers XSLT and XPath version 2.0, but in 2022 we’re up to XSLT version 3.0 and XPath version 3.1), it is still an excellent resource when programming XSLT. It is also a very good place to start when you need information on XPath.

- ***XSLT, 2nd Edition* by Doug Tidwell (O’Reilly, 2008)**

A good book to learn and explore XSLT. Like Michael Kay’s book, listed above, it’s a bit outdated but can still be used to study the basics.

- ***XSLT Cookbook* by Sal Mangano (O’Reilly, 2006)**

A cookbook with solutions for various XSLT programming problems. Like the preceding two books, it’s outdated with regards to the current version of XSLT, but it is still useful.

- **The XSLT 3.0 standard:** <https://www.w3.org/TR/xslt-30/>

This is a standard and, therefore, sometimes hard to follow for mere mortals. However, since there are not many information sources (yet) on XSLT 3.0, you need to look here if you want to learn about the more advanced features, such as streaming, or interesting new instructions such as `<xsl:iterate>` and `<xsl:where-populated>`.

F.6. XQuery

XQuery is a programming language designed for querying XML resources (like XML databases), but it can also be used as a general-purpose programming language.

- ***XQuery: Search Across a Variety of XML Data* by Priscilla Walmsley (O'Reilly, 2016)**

Considered the authoritative resource for programming XQuery, this book is also a good place to start when you need information about the XPath standard. It is up-to-date with the most recent XPath standard, so it is a good resource for XPath, even if you're not interested in XQuery.

- ***XQuery for Humanists* by Clifford B. Anderson and Joseph C. Wicentowski (Texas A&M University Press, 2020)**

This book focuses on text processing, assuming the reader has no programming background. It also covers setting up XQuery processing environments such as BaseX and eXist.

F.7. XProc

The programming language XProc is mentioned a few times in this book. It's a language for transforming XML (and other) documents using a sequence of separate smaller steps: a pipeline.

- ***XProc 3.0 Programmer Reference* by Erik Siegel (XML Press, 2020)**

This book describes the 3.0 version of the XProc standard. If you like this Schematron book, you will probably also enjoy this one, by the same author ;)

- **XProc website:** <https://xproc.org/>

Site with links to all kinds of information about the XProc standard, including the standard itself and a list of learning materials.

F.8. Other information

- **W3C:** <https://www.w3.org>

The W3C is the body that manages, among other things, the XML standards. Their website is easy to use and informative.

- **xml.com:** <https://www.xml.com/>

xml.com is a website with articles and news about XML and related technologies.

- **W3 Schools:** <https://w3schools.com>

W3Schools is a site for developers with tutorials and references on web development languages. Among them XSLT, XQuery, and XPath (unfortunately, not Schematron).^[29]

^[29] W3Schools is considered a controversial information resource within the XML community. It has been faulted for having a history of errors and is infamous for its heavy advertising. Nonetheless, I consult it regularly and find the information useful and well-presented. Judge for yourself.

Colophon

About the Author

Erik Siegel is a content engineer and XML specialist who runs [Xatapult](#) consultancy in the Netherlands. He specializes in content design and conversions, XML Schemas and transformations, eXist and XProc applications, and XML-related training.

Since 2017, he has been part of the XProc 3.0 editorial team.

About XML Press

XML Press (<https://xmlpress.net>) was founded in 2008 to publish content that helps technical communicators be more effective. Our publications support managers, social media practitioners, technical communicators, and content strategists and the engineers who support their efforts.

Our publications are available through most retailers, and discounted pricing is available for volume purchases for business, educational, or promotional use. For more information, send email to orders@xmlpress.net or call us at (970) 231-3624.

Appendix G. Copyright and Legal Notices

Erik Siegel

Copyright © 2022 Erik Siegel

Except as noted in the Credits section below, all rights are reserved. No part of this book may be reproduced or transmitted in any form or by any means without the prior written permission of the copyright holder, except for the inclusion of brief quotations in a review.

Disclaimer

The information in this book is provided on an “as is” basis, without warranty. While every effort has been taken by the author and XML Press in the preparation of this book, the author and XML Press shall have neither liability nor responsibility to any person or entity with respect to any loss or damages arising from the information contained herein.

This book contains links to third-party web sites that are not under the control of the author or XML Press. The author and XML Press are not responsible for the content of any linked site. Inclusion of a link in this book does not imply that the author or XML Press endorses or accepts any responsibility for the content of that third-party site.

Credits

Schematroll mascot image:

Cody Chang

Author photo:

Maaïke Siegel

Trademarks

XML Press and the XML Press logo are trademarks of XML Press.

All terms mentioned in this book that are known to be trademarks or service marks have been capitalized as appropriate. Use of a term in this book should not be regarded as affecting the validity of any trademark or service mark.

ISBN: 978-1-937434-80-9 (print)

ISBN: 978-1-937434-81-6 (ebook)

XML Press

Denver, CO

<https://xmlpress.net>

Table of Contents

1. Preface	8
1.1. Who is this book for?	10
1.2. How to use this book	12
1.3. Using and finding the code examples	13
1.4. The oXygen IDE	13
1.5. Contact information	14
1.6. Acknowledgements	14
2. Introduction to Schematron	15
2.1. What is Schematron?	15
2.2. Why Schematron?	15
2.3. The history of Schematron in a nutshell	16
2.4. The Schematron standard	17
2.4.1. A critical note on the standard	17
2.5. The Schematroll	18
2.6. An illustrative example	19
2.6.1. The example document	19
2.6.2. Running the examples	20
2.6.3. Checking the value of the code attribute	20
2.6.4. Improving the message	23
2.6.5. Using variables	24
3. Schematron in context	27
3.1. Validating XML documents	27
3.1.1. Restricting XML freedom	27
3.1.2. How to define XML formats	29
3.1.3. Validating XML documents	30
3.2. The main schema languages	33
3.2.1. Document Type Definition (DTD)	34
3.2.2. W3C XML Schema	35

3.2.2.1. W3C XML Schema version 1.1	36
3.2.3. RELAX NG	38
3.2.4. Schematron	39
4. Applying Schematron	41
4.1. Creating Schematron schemas	42
4.2. IDE-based Schematron validation	42
4.3. Raw Schematron validation	45
4.4. Validation results	46
4.4.1. SVRL	47
4.4.2. XVRL	48
4.5. Schematron validation processors	48
4.5.1. The “skeleton” XSLT Schematron processor	49
4.5.2. The SchXslt Schematron processor	49
4.5.2.1. Running SchXslt from the command line	50
5. Schematron basics	53
5.1. Setting up a Schematron schema	53
5.2. Patterns, rules, assertions, and reports	55
5.2.1. The Schematron processing algorithm	57
5.2.2. Rule processing	58
5.2.3. Assert and Report processing	60
5.2.3.1. Using reports instead of assertions	63
5.2.3.2. The message texts	65
5.3. More meaningful messages: <value-of>	65
5.4. Declaring and using variables: <let>	68
5.4.1. Ground rules for variables	69
5.4.2. Variable usage example	73
5.5. Declaring namespaces	76
6. Advanced Schematron	79
6.1. Providing multiple messages: <diagnostics>	79
6.1.1. Multiple messages	81
6.1.2. Localization of messages	83

6.2. Selecting what patterns are active: <phase>	85
6.3. Reusing rules: abstract rules	90
6.3.1. Abstract rules in external documents	93
6.3.2. Alternative to abstract rules	94
6.4. Reusing patterns: Abstract patterns	95
6.5. Including documents: <include>	98
7. Query language binding and using XSLT	101
7.1. Introduction to query language binding	101
7.2. Using XSLT in Schematron	106
7.2.1. Using XSLT keys	106
7.2.2. Using XSLT functions	109
7.2.3. Using other XSLT features	112
8. Additional features	114
8.1. Messages with markup: <emph>, , and <dir>	114
8.2. Flags	115
8.3. Properties	116
8.4. Adding structured comments: <title> and <p>	120
8.5. Validating documents referenced by XInclude	122
8.6. Specifying a role: the role attribute	125
8.7. Specify a different location: the subject attribute	126
8.8. The <name> element	129
8.9. The icon, see, and fpi attributes	130
9. Schematron examples and recipes	132
9.1. Validating a Schematron schema	132
9.2. Handling a default namespace	135
9.3. Using Schematron for schema validation	138
9.4. Checking multiple identifier references	143
9.5. Validating processing instructions and comments	148
9.6. Validating doubled elements in mixed content	151
A. XPath technology primer	156
A.1. XML as a tree	156

A.1.1. Basic trees: documents, elements, and text	157
A.1.2. Attributes in the tree	161
A.1.3. Representing mixed content	162
A.2. Basic tree navigation	164
A.2.1. Using navigation expressions	166
A.2.2. Basic navigation to attributes	168
A.2.3. Multiple selections: sequences	168
A.3. Tree navigation and the context item	169
A.4. Some special operators	171
A.4.1. The context item single dot . operator	172
A.4.2. The parent double dot .. operator	172
A.4.3. The * and @* wildcard operators	173
A.4.4. The “search” // operator	174
A.5. Predicates in tree navigation	175
A.6. Expressions on simple data	181
A.6.1. The XPath function library	181
A.6.2. XPath data types	182
A.6.2.1. Explicit data typing and data type conversions	185
A.6.3. Numerical expressions	187
A.6.4. String expressions	188
A.6.5. Boolean expressions and comparisons	190
A.6.5.1. Testing sequences	192
A.6.5.2. Testing string values	194
A.6.6. Working with dates, times and durations	196
A.6.6.1. Additional date, time, and duration functions	200
B. An introduction to namespaces	202
B.1. Why namespaces?	202
B.2. Namespace names	204
B.3. Declaring namespaces	205
B.3.1. Defining a default namespace	206
B.3.2. Defining and using namespace prefixes	208

B.3.3. The XML namespace	211
B.4. Namespaces in Schematron	211
C. Schematron reference	213
C.1. XML structure overview notation	213
C.2. The Schematron namespace	216
C.2.1. Using other namespaces	216
C.3. Root element: <schema>	217
C.4. Declaring a namespace: <ns>	220
C.5. Defining validation phases: <phase>	221
C.5.1. Attaching a single pattern to a phase: <active>	223
C.6. Creating validation patterns: <pattern>	224
C.6.1. Rules in validation patterns: <rule>	227
C.6.1.1. Defining assertions: <assert>	230
C.6.1.2. Defining reports: <report>	233
C.6.1.3. Referencing an abstract rule: <extends>	236
C.6.2. Parameters for abstract patterns: <param>	237
C.7. Defining diagnostic messages: <diagnostics>	237
C.7.1. Diagnostic message: <diagnostic>	238
C.8. Defining additional properties: <properties>	240
C.8.1. Defining a property: <property>	240
C.9. Message markup: mixed contents	242
C.9.1. Writing direction: <dir>	242
C.9.2. Emphasis: <emph>	243
C.9.3. Retrieving the name of a node: <name>	243
C.9.4. Spanning text: 	243
C.9.5. Value of an XPath expression: <value-of>	244
C.10. Standard attributes and elements	244
C.10.1. The flag attribute	244
C.10.2. The fpi attribute	245
C.10.3. The icon attribute	245
C.10.4. The role attribute	245

C.10.5. The subject attribute	245
C.10.6. The see attribute	246
C.10.7. The xml:lang attribute	246
C.10.8. The xml:space attribute	246
C.10.9. The <let> element	246
C.10.10. The <p> element	247
C.10.11. The <title> element	248
C.10.12. The <include> element	248
D. SVRL reference	250
D.1. SVRL in the Schematron standard	250
D.2. The SVRL namespace	251
D.3. Root element: <schematron-output>	251
D.4. Namespace declarations: <ns-prefix-in-attribute-values>	253
D.5. Active patterns: <active-pattern>	254
D.6. Fired rules: <fired-rule>	255
D.7. Failed asserts: <failed-assert>	256
D.7.1. Diagnostic reference: <diagnostic-reference>	258
D.7.2. Property reference: <property-reference>	258
D.8. Successful reports: <successful-report>	259
D.9. Mixed text: <text>	260
D.9.1. Writing direction: <dir>	261
D.9.2. Emphasis: <emph>	262
D.9.3. Spanning text: 	262
E. Schematron QuickFix	264
F. Additional resources	271
F.1. Schematron	271
F.2. XPath	271
F.3. XML Schema	272
F.4. RELAX NG	273
F.5. XSLT	273
F.6. XQuery	274

F.7. XProc	274
F.8. Other information	275
G. Copyright and Legal Notices	277